

Q1) A company is hosting their high-frequency trading application in AWS which serves millions of investors around the globe. The application is hosted in an Auto Scaling Group of EC2 instances behind an Application Load Balancer with an Amazon DynamoDB database. The architecture was deployed using a CloudFormation template with a Route 53 record. There recently was a production deployment that had caused system degradation and outage, costing the company a significant monetary loss due to their application's unavailability. As a result, the company instructed their DevOps engineer to implement an efficient strategy for deploying updates to their web application with the ability to perform an immediate rollback of the stack. All deployments should maintain the normal number of active EC2 instances to keep the performance of the application. Which of the following should the DevOps engineer implement to satisfy these requirements?

● Configure the UpdatePolicy of the AWS::AutoScaling::DeploymentUpdates resource in the CloudFormation template to use the AutoScalingReplacingUpdate policy. Update the required properties for the new Auto Scaling group policy. Set the WillReplace property to false.

● Configure the UpdatePolicy of the AWS::AutoScaling::AutoscalingGroup resource in the CloudFormation template to use the AutoScalingRollingUpdate policy. Update the required properties for the new Auto Scaling group policy. Set the MinSuccessfulInstancesPercent property, as well as its corresponding WaitOnResourceSignals and PauseTime properties.

● Configure the UpdatePolicy of the AWS::AutoScaling::AutoscalingGroup resource in the CloudFormation template to use the AutoScalingReplacingUpdate policy. Set the WillReplace property to false. Also, specify the AutoScalingRollingUpdate policy to update instances that are in an Auto Scaling group in batches.

✔ Configure the UpdatePolicy of the AWS::AutoScaling::AutoscalingGroup resource in the CloudFormation template to use the AutoScalingReplacingUpdate policy. Update the required properties for the new Auto Scaling group policy. Set the WillReplace property to true.

Explanation: -If you plan to launch an Auto Scaling group of EC2 instances, you can configure the AWS::AutoScaling::AutoScalingGroup resource type reference in your CloudFormation template to define an Amazon EC2 Auto Scaling group with the specified name and attributes. To configure Amazon EC2 instances launched as part of the group, you can specify a launch template or a launch configuration. It is recommended that you use a launch template to make sure that you can use the latest features of Amazon EC2, such as T2 Unlimited instances.

You can add an UpdatePolicy attribute to your Auto Scaling group to perform rolling updates (or replace the group) when a change has been made to the group.

To specify how AWS CloudFormation handles replacement updates for an Auto Scaling group, use the AutoScalingReplacingUpdate policy. This policy enables you to specify whether AWS CloudFormation replaces an Auto Scaling group with a new one or replaces only the instances in the Auto Scaling group.

During replacement, AWS CloudFormation retains the old group until it finishes creating the new one. If the update fails, AWS CloudFormation can roll back to the old Auto Scaling group and delete the new Auto Scaling group. While AWS CloudFormation creates the new group, it doesn't detach or attach any instances. After successfully creating the new Auto Scaling group, AWS CloudFormation deletes the old Auto Scaling group during the cleanup process.

When you set the WillReplace parameter, remember to specify a matching CreationPolicy. If the minimum number of instances (specified by the MinSuccessfulInstancesPercent property) don't signal success within the Timeout period (specified in the CreationPolicy policy), the replacement update fails and AWS CloudFormation rolls back to the old Auto Scaling group.

Hence, the correct answer is: Configure the UpdatePolicy of the AWS::AutoScaling::AutoscalingGroup resource in the CloudFormation template to use the AutoScalingReplacingUpdate policy. Update the required properties for the new Auto Scaling group policy. Set the WillReplace property to true.

The option that says: Configure the UpdatePolicy of the AWS::AutoScaling::AutoscalingGroup resource in the CloudFormation template to use the AutoScalingReplacingUpdate policy. Set the WillReplace property to false. Also, specify the AutoScalingRollingUpdate policy to update instances that are in an Auto Scaling group in batches is incorrect because if both the AutoScalingReplacingUpdate and AutoScalingRollingUpdate policies are specified, setting the WillReplace property to true gives AutoScalingReplacingUpdate precedence. But since this property is set to false, then the AutoScalingRollingUpdate policy will take precedence instead.

The option that says: Configure the UpdatePolicy of the AWS::AutoScaling::AutoscalingGroup resource in the CloudFormation template to use the AutoScalingRollingUpdate policy. Update the required properties for the new Auto Scaling group policy. Set the MinSuccessfulInstancesPercent property, you must also enable the WaitOnResourceSignals and PauseTime properties is incorrect because this type of deployment will affect the existing compute capacity of your application. The rolling update doesn't maintain the total number of active EC2 instances during deployment. A better solution is to use the AutoScalingReplacingUpdate policy instead that will create a separate Auto Scaling group and is able to perform an immediate rollback of the stack in an event of an update failure.

The option that says: Configure the UpdatePolicy of the AWS::AutoScaling::DeploymentUpdates resource in the CloudFormation template to use the AutoScalingReplacingUpdate policy. Update the required properties for the new Auto Scaling group policy. Set the WillReplace property to false is incorrect because there is no AWS::AutoScaling::DeploymentUpdates resource. You have to use the AWS::AutoScaling::AutoscalingGroup resource instead and set the WillReplace property to true.

References:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-properties-as-group.html>

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-attribute-updatepolicy.html#cfn-attributes-updatepolicy-replacingupdate>

<https://aws.amazon.com/premiumsupport/knowledge-center/auto-scaling-group-rolling-updates/>

Check out this AWS CloudFormation Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-cloudformation/>

Q2) A leading technology company is planning to build a document management portal that will utilize an existing Amazon DynamoDB table. The portal will be hosted in AWS Elastic Beanstalk for easier deployment. The table has an attribute of DocumentName that acts as the partition key and another attribute called Category as its sort key. A DevOps Engineer was instructed to develop a feature that queries the DocumentName attribute yet uses a different sort key other than the existing one.

To fetch the latest data, strong read consistency must be used in the database tier. Which of the following solutions below should the engineer implement?

● Add a Global Secondary Index that uses the DocumentName attribute and a different sort key.

● Add a Local Secondary Index that uses the DocumentName attribute and a different sort key.

● Add a Global Secondary Index which uses the DocumentName attribute and use an alternative sort key as projected attributes.

✔ Set up a new DynamoDB table with a Local Secondary Index that uses the DocumentName attribute with a different sort key. Migrate the data from the existing table to the new table.

Explanation: -A local secondary index maintains an alternate sort key for a given partition key value. A local secondary index also contains a copy of some or all of the attributes from its base table; you specify which attributes are projected into the local secondary index when you create the table.

The data in a local secondary index is organized by the same partition key as the base table, but with a different sort key. This lets you access data items efficiently across this different dimension. For greater query or scan flexibility, you can create up to five local secondary indexes per table.

Suppose that an application needs to find all of the threads that have been posted within the last three months. Without a local secondary index, the application would have to Scan the entire Thread table and discard any posts that were not within the specified time frame. With a local secondary index, a Query operation could use LastPostDateTime as a sort key and find the data quickly.

To create a Local Secondary Index, make sure that the primary key of the index is the same as the primary key/partition key of the table, just as shown below. Then you must select an alternative sort key which is different from the sort key of the table.

When you request a strongly consistent read, DynamoDB returns a response with the most up-to-date data, reflecting the updates from all prior write operations that were successful. A strongly consistent read might not be available if there is a network delay or outage. Strongly consistent reads are not supported on global secondary indexes.

The primary key of a local secondary index must be composite (partition key and sort key). A local secondary index lets you query over a single partition, as specified by the partition key value in the query.

Local secondary indexes are created at the same time that you create a table. You cannot add a local secondary index to an existing table, nor can you delete any local secondary indexes that currently exist.

Hence, the correct answer is: Set up a new DynamoDB table with a Local Secondary Index that uses the DocumentName attribute with a different sort key. Migrate the data from the existing table to the new table.

The option that says: Add a Global Secondary Index that uses the DocumentName attribute and a different sort key is incorrect because the scenario is still using the same partition key, but with an alternate sort key which warrants the use of a local secondary index instead of a global secondary index.

The option that says: Add a Global Secondary Index which uses the DocumentName attribute and use an alternative sort key as projected attributes is incorrect because using a local secondary index is a more appropriate solution to be used in this scenario just as explained above. Moreover, projected attributes are just attributes stored in the index that can be returned by queries and scans performed on the index, hence, these are not useful in satisfying the provided requirement.

The option that says: Add a Local Secondary Index that uses the DocumentName attribute and a different sort key is incorrect because although it is using the correct type of index, you cannot add a local secondary index to an already existing table.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/SecondaryIndexes.html>

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/LSI.html>

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-amazon-dynamodb/>

Tutorials Dojo's AWS Certified DevOps Engineer Professional Exam Study Guide:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-certified-devops-engineer-professional/>

Q3) You are deploying a critical web application with Elastic Beanstalk using the “Rolling” deployment policy. Your Elastic Beanstalk environment configuration has an RDS DB instance attached to it and used by your application servers. The deployment failed when you deployed a major version. And it took even more time to rollback changes because you have to manually redeploy the old version.

Which of the following options will you implement to prevent this from happening in future deployments?

- ☐ Configure Rolling with additional batch as the deployment policy in your Elastic Beanstalk environment for future deployments of your web application.
- ☐ Implement a Blue/green deployment strategy in your Elastic Beanstalk environment for future deployments of your web application. Ensure that the RDS DB instance is still tightly coupled with the environment.
- ☒ Configure Immutable as the deployment policy in your Elastic Beanstalk environment for future deployments of your web application.

Explanation:-Immutable environment updates are an alternative to rolling updates. Immutable environment updates ensure that configuration changes that require replacing instances are applied efficiently and safely. If an immutable environment update fails, the rollback process requires only terminating an Auto Scaling group. A failed rolling update, on the other hand, requires performing an additional rolling update to roll back the changes.

To perform an immutable environment update, Elastic Beanstalk creates a second, temporary Auto Scaling group behind your environment's load balancer to contain the new instances. First, Elastic Beanstalk launches a single instance with the new configuration in the new group. This instance serves traffic alongside all of the instances in the original Auto Scaling group that are running the previous configuration.

When the first instance passes health checks, Elastic Beanstalk launches additional instances with the new configuration, matching the number of instances running in the original Auto Scaling group. When all of the new instances pass health checks, Elastic Beanstalk transfers them to the original Auto Scaling group, and terminates the temporary Auto Scaling group and old instances.

Refer to the table below for the characteristics of each deployment method as well as the amount of time it takes to do the deployment, as seen in the Deploy Time column:

Hence, the correct answer is: Configure Immutable as the deployment policy in your Elastic Beanstalk environment for future deployments of your web application.

The option that says: Configure Rolling with additional batch as the deployment policy in your Elastic Beanstalk environment for future deployments of your web application is incorrect because this deployment type is just similar to rolling deployments and hence, this will not help alleviate the root cause of the issue in this scenario.

The option that says: Configure All at once as the deployment policy in your Elastic Beanstalk environment for future deployments of your web application is incorrect because this will cause a brief downtime during deployment and hence, this is not ideal for deploying your critical production applications.

The option that says: Implement a Blue/green deployment strategy in your Elastic Beanstalk environment for future deployments of your web application. Ensure that the RDS DB instance is still tightly coupled with the environment is incorrect because a Blue/green deployment requires that your environment runs independently of your production database. This means that you have to decouple your RDS database from your environment. If your Elastic Beanstalk environment has an attached Amazon RDS DB instance, the data will be lost if you terminate the original (blue) environment.

References:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/environmentmgmt-updates-immutable.html>

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.rolling-version-deploy.html>

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.rollingupdates.html>

Check out this AWS Elastic Beanstalk Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-elastic-beanstalk/>

- ☐ Configure All at once as the deployment policy in your Elastic Beanstalk environment for future deployments of your web application.

Q4) A web application is hosted on an Auto Scaling group (ASG) of On-Demand EC2 instances. You have a separate Linux EC2 instance that is used primarily for batch processing. This particular instance needs to update its configuration file based on the list of the active IP addresses of the instances within the ASG. This is needed in order to run a batch job properly.

Which of the following options will let you effectively update the configuration file whenever the ASG scales-in/scales-out?

- Create a CloudWatch Events rule scheduled to run every minute. Set the target to a Lambda function to query the ASG instances and update the configuration file on an S3 bucket. Automatically establish an SSH connection to the Linux EC2 instances by using another Lambda function and then download the master configuration file from S3 to update the local configuration file stored in the instance.
- Create a CloudWatch Logs to monitor the Launch/Terminate events of the ASG. Set the target to a Lambda function that will update the configuration file inside the EC2 instance. Add a proper IAM permission to Lambda to access the EC2 instance.
- Develop a custom script that runs on the background to query active EC2 instances and IP addresses of the ASG. Then the script will update the configuration whenever it detects a change.
- ✔ Create a CloudWatch Events rule for the Launch/Terminate events of the ASG. Set the target to an SSM Run Command that will update the configuration file on the target EC2 instance.

Explanation:-You can use Amazon CloudWatch Events to invoke AWS Systems Manager Run Command and perform actions on Amazon EC2 instances when certain events happen.

You can use SSM Run Command to configure instances without having to login to an instance. This setup requires the SSM agent to be installed on your EC2 instance and that proper IAM permission has been granted. You need to set the CloudWatch Event rule that will watch for the launch/terminate action of the auto-scaling group. Then set the Systems Manager Run command as the target of that event.

On the parameters of SSM Run Command, define the commands you need to update the configuration file. It is important that your target EC2 instance is properly tagged and defined on SSM Run command as it will be the basis of SSM to identify your instance.

Hence, the correct answer is: Create a CloudWatch Events rule for the Launch/Terminate events of the ASG. Set the target to an SSM Run Command that will update the configuration file on the target EC2 instance.

The option that says: Develop a custom script that runs on the background to query active EC2 instances and IP addresses of the ASG. Then the script will update the configuration whenever it detects a change is incorrect because although this may be a possible solution, you will still have to write and maintain your own script and that creates an unnecessary operational overhead. Also, running a script with regular intervals is not a good approach if you are just waiting for events.

The option that says: Create a CloudWatch Logs to monitor the Launch/Terminate events of the ASG. Set the target to a Lambda function that will update the configuration file inside the EC2 instance. Add a proper IAM permission to Lambda to access the EC2 instance is incorrect because you can't monitor the Auto Scaling Group events using CloudWatch Logs. You have to use CloudWatch Events instead.

The option that says: Create a CloudWatch Events rule scheduled to run every minute. Set the target to a Lambda function to query the ASG instances and update the configuration file on an S3 bucket. Automatically establish an SSH connection to the Linux EC2 instances by using another Lambda function and then download the master configuration file from S3 to update the local configuration file stored in the instance is incorrect because it is better to use a CloudWatch Events rule that tracks the Launch/Terminate events of the Auto Scaling group (ASG) instead of running it every minute. AWS Lambda can't directly establish an SSH connection and run a command inside an EC2 instance to update the configuration file, even with proper IAM permissions. A better solution would be to use the Systems Manager Run Command to let you remotely and securely manage the configuration of your managed instances.

References:

https://docs.aws.amazon.com/AmazonCloudWatch/latest/events/EC2_Run_Command.html

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-prereqs.html>

<https://docs.aws.amazon.com/systems-manager/latest/userguide/rc-console.html>

Check out these Amazon CloudWatch and AWS Systems Manager Cheat Sheets:

<https://tutorialsdojo.com/aws-cheat-sheet-amazon-cloudwatch/>

<https://tutorialsdojo.com/aws-cheat-sheet-aws-systems-manager/>

Q5) A company is planning to launch its Node.js application to AWS to better serve its clients around the globe. A hybrid deployment is required to be implemented wherein the application will run on both on-premises application servers and On-Demand Amazon EC2 instances. The application instances require secure access to database credentials, which must be encrypted both at rest and in transit.

As a DevOps Engineer, how can you automate the deployment process of the application in the MOST secure manner?

- Store database credentials in the appspec.yml configuration file. Create an IAM policy for allowing access to only the database credentials.

Attach the IAM policy to the role associated to the instance profile for CodeDeploy-managed instances and the IAM role used for on-premises instances registration on CodeDeploy. Deploy the application packages to the EC2 instances and on-premises servers using AWS CodeDeploy.

- Using AWS Systems Manager Parameter Store, upload and manage the database credentials with a Secure String data type. Create an IAM Policy that allows access and decryption of the database credentials. Attach the IAM policy to the instance profile for CodeDeploy-managed instances as well as to the on-premises instances. Deploy the application packages to the EC2 instances and on-premises servers using AWS CodeDeploy.

- ✔ Using AWS Systems Manager Parameter Store, upload and manage the database credentials with a Secure String data type. Create an IAM role with an attached policy that allows access and decryption of the database credentials. Associate this role to all the on-premises servers as well as the EC2 instances. Deploy the application packages to the EC2 instances and on-premises servers using AWS CodeDeploy.

Explanation:-AWS Systems Manager Parameter Store provides secure, hierarchical storage for configuration data management and secrets management. You can store data such as passwords, database strings, and license codes as parameter values. You can store values as plain text or encrypted data. You can then reference values by using the unique name that you specified when you created the parameter. Highly scalable, available, and durable, Parameter Store is backed by the AWS Cloud.

Servers and virtual machines (VMs) in a hybrid environment require an IAM role to communicate with the Systems Manager service. The role grants AssumeRole trust to the Systems Manager service. You only need to create the service role for a hybrid environment once for each AWS account. Users in your company or organization who will use Systems Manager on your hybrid machines must be granted permission in IAM to call the SSM API.

Hence, the correct answer is: Using AWS Systems Manager Parameter Store, upload and manage the database credentials with a Secure String data type. Create an IAM role with an attached policy that allows access and decryption of the database credentials. Associate this role to all the on-premises servers as well as the EC2 instances. Deploy the application packages to the EC2 instances and on-premises servers using AWS CodeDeploy.

The option that says: Using AWS Systems Manager Parameter Store, upload and manage the database credentials with a Secure String data type.

Create an IAM role that allows access and decryption of the database credentials. Associate this role to all the Amazon EC2 instances. Upload the application in AWS Elastic Beanstalk with a Node.js platform configuration and deploy the application revisions to both on-premises servers and EC2 instances using blue/green deployment is incorrect because you can't deploy an application to your on-premises servers using Elastic Beanstalk.

This is only applicable to your Amazon EC2 instances.

The option that says: Using AWS Systems Manager Parameter Store, upload and manage the database credentials with a Secure String data type. Create an IAM policy that allows access and decryption of the database credentials. Attach the IAM policy to the instance profile for CodeDeploy-managed instances as well as to the on-premises instances. Deploy the application packages to the EC2 instances and on-premises servers using AWS CodeDeploy is incorrect because you have to use an IAM Role and not an IAM Policy to grant access to AWS Systems Manager Parameter Store.

The option that says: Store database credentials in the appspec.yml configuration file. Create an IAM policy for allowing access to only the database credentials. Attach the IAM policy to the role associated to the instance profile for CodeDeploy-managed instances and the IAM role used for on-

premises instances registration on CodeDeploy. Deploy the application packages to the EC2 instances and on-premises servers using AWS CodeDeploy is incorrect because it is not secure to store sensitive database credentials in an appspec.yml configuration file. You have to use AWS Systems Manager Parameter Store instead.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html>

<https://docs.aws.amazon.com/systems-manager/latest/userguide/sysman-service-role.html>

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-managedinstances.html>

Check out this AWS Systems Manager Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-systems-manager/>

- Using AWS Systems Manager Parameter Store, upload and manage the database credentials with a Secure String data type. Create an IAM role that allows access and decryption of the database credentials. Associate this role to all the Amazon EC2 instances. Upload the application in AWS Elastic Beanstalk with a Node.js platform configuration and deploy the application revisions to both on-premises servers and EC2 instances using blue/green deployment.

Q6) A leading software company is testing an application that is hosted on an Auto Scaling group of EC2 instances across multiple Availability Zones behind an Application Load Balancer. For its deployment process, the company uses the blue/green deployment process with immutable instances to avoid any service degradation. Recently, the DevOps team noticed that the users are being automatically logged out of the application intermittently. Whenever a new version of the application is deployed, all users are automatically logged out that affects the overall user experience. The DevOps team needs a solution to ensure users remain logged in across scaling events and application deployments. Which among the following options is the MOST efficient way to solve this issue?

- Launch a Memcached session store on a large EC2 instance on a single Availability Zone. Configure the application to store user-session information in its corresponding Memcached node.
- Launch a Redis session store on a large EC2 instance for each supported Availability Zone that are independent from each other. Configure the application to store user-session information in its corresponding Redis node.
- ✔ Launch a new Amazon ElastiCache for Redis cluster with nodes across multiple Availability Zones. Configure the application to store user-session information in ElastiCache.

Explanation:- There are various ways to manage user sessions including storing those sessions locally to the node responding to the HTTP request or designating a layer in your architecture which can store those sessions in a scalable and robust manner. Common approaches used include utilizing Sticky sessions or using a Distributed Cache for your session management.

In order to address scalability and to provide a shared data storage for sessions that can be accessible from any individual web server, you can abstract the HTTP sessions from the web servers themselves. A common solution to for this is to leverage an In-Memory Key/Value store such as Redis and Memcached.

While Key/Value data stores are known to be extremely fast and provide sub-millisecond latency, the added network latency and added cost are the drawbacks. An added benefit of leveraging Key/Value stores is that they can also be utilized to cache any data, not just HTTP sessions, which can help boost the overall performance of your applications.

A consideration when choosing a distributed cache for session management is determining how many nodes may be needed in order to manage the user sessions. Generally speaking, this decision can be determined by how much traffic is expected and/or how much risk is acceptable. In a distributed session cache, the sessions are divided by the number of nodes in the cache cluster. In the event of a failure, only the sessions that are stored on the failed node are affected. If reducing risk is more important than cost, adding additional nodes to further reduce the percent of stored sessions on each node may be ideal even when fewer nodes are sufficient.

Another consideration may be whether or not the sessions need to be replicated or not. Some key/value stores offer replication via read replicas. In the event of a node failure, the sessions would not be entirely lost. Whether replica nodes are important in your individual architecture may inform as to which key/value store should be used. ElastiCache offerings for In-Memory key/value stores include ElastiCache for Redis, which can support replication, and ElastiCache for Memcached which does not support replication.

Hence, the correct answer is: Launch a new Amazon ElastiCache for Redis cluster with nodes across multiple Availability Zones. Configure the application to store user-session information in ElastiCache.

The option that says: Launch a Memcached session store on a large EC2 instance on a single Availability Zone. Configure the application to store user-session information in its corresponding Memcached node is incorrect as this will not solve the problem - the session store is located in a single instance and on a single Availability Zone. You should set up a distributed cache for session management using an ElastiCache cluster that spans across multiple AZs.

The option that says: Launch a Redis session store on a large EC2 instance for each supported Availability Zone that are independent from each other. Configure the application to store user-session information in its corresponding Redis node is incorrect because you have to create several session store nodes that are related to each other and are not stand-alone nodes.

The option that says: Enable sticky sessions in the Application Load Balancer. Configure the application to store and retrieve the user sessions from the load balancer is incorrect because, in the event of a failure, you are likely to lose the sessions that reside on the failed node. In addition, in the event the number of your web servers change, for example a scale-up scenario, it's possible that the traffic may be unequally spread across the web servers as active sessions may exist on particular servers.

References:

<https://aws.amazon.com/elasticache/>

<https://aws.amazon.com/caching/session-management/>

Check out this Amazon ElastiCache Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-amazon-elasticache/>

- Enable sticky sessions in the Application Load Balancer. Configure the application to store and retrieve the user sessions from the load balancer.

Q7) A company has a PROD, DEV, and TEST environment in its software development department, each contains hundreds of EC2 instances and other AWS services. There was a series of security patches that have been released on the official Ubuntu operating system for a critical flaw that was recently discovered. Although this is an urgent matter, there is no guarantee that these patches will be bug-free and production-ready. This is why a DevOps engineer was instructed to immediately patch all of their affected EC2 instances in all the environments, except for the PROD environment. The EC2 instances in their PROD environment will only be patched after the initial patches have been verified to work effectively in their non-PROD environments. Each environment also has different baseline patch requirements that you will need to satisfy. How should the DevOps engineer perform this task with the LEAST amount of effort?

- Use the AWS Systems Manager Maintenance Windows to set up a scheduled maintenance period for each environment, where the period is after business hours so as not to affect daily operations. The Systems Manager will execute a cron job that will install the required patches for each Amazon EC2 instance in each environment during the maintenance period. Use the Systems Manager Managed Instances to verify that your environments are fully patched and compliant.

- Develop various shell scripts for each environment that specifies which patch will serve as its baseline. Tag each instance based on its

environment, business unit, and operating system. Add the Amazon EC2 instances into Target Groups using the AWS Systems Manager Run Command and then execute the script corresponding to each Target Group.

- Set up a new patch baseline in AWS Systems Manager Patch Manager for each environment. Tag each Amazon EC2 instance based on its operating system. Categorize EC2 instances based on their tags using Patch Groups. Apply the patches specified in their corresponding patch baseline to each Patch Group. Use Patch Compliance to ensure that the patches have been installed correctly. Using AWS Config, record all of the changes to patch and association compliance statuses.

- ✔ Tag each instance based on its environment, business unit, and operating system. Set up a patch baseline in AWS Systems Manager Patch Manager for each environment. Categorize each Amazon EC2 instance based on its tags using Patch Groups. Apply the required patches specified in the corresponding patch baseline to each Patch Group.

Explanation:- AWS Systems Manager Patch Manager automates the process of patching managed instances with security-related updates. For Linux-based instances, you can also install patches for non-security updates. You can patch fleets of Amazon EC2 instances or your on-premises servers and virtual machines (VMs) by operating system type.

Patch Manager uses patch baselines, which include rules for auto-approving patches within days of their release, as well as a list of approved and rejected patches. You can install patches on a regular basis by scheduling patching to run as a Systems Manager Maintenance Window task. You can also install patches individually or to large groups of instances by using Amazon EC2 tags. For each auto-approval rule that you create, you can specify an auto-approval delay. This delay is the number of days to wait after the patch was released before the patch is automatically approved for patching.

A patch group is an optional means of organizing instances for patching. For example, you can create patch groups for different operating systems (Linux or Windows), different environments (Development, Test, and Production), or different server functions (web servers, file servers, databases). Patch groups can help you avoid deploying patches to the wrong set of instances. They can also help you avoid deploying patches before they have been adequately tested.

You create a patch group by using Amazon EC2 tags. Unlike other tagging scenarios across Systems Manager, a patch group must be defined with the tag key: Patch Group. After you create a patch group and tag instances, you can register the patch group with a patch baseline. By registering the patch group with a patch baseline, you ensure that the correct patches are installed during the patching execution.

Hence, the correct answer is: Tag each instance based on its environment, business unit, and operating system. Set up a patch baseline in AWS Systems Manager Patch Manager for each environment. Categorize each Amazon EC2 instance based on its tags using Patch Groups. Apply the required patches specified in the corresponding patch baseline to each Patch Group.

The option that says: Develop various shell scripts for each environment that specifies which patch will serve as its baseline. Tag each instance based on its environment, business unit, and operating system. Add the Amazon EC2 instances into Target Groups using the AWS Systems Manager Run Command and then execute the script corresponding to each Target Group is incorrect as this option takes more effort to perform because you are using Systems Manager Run Command instead of Patch Manager. The Run Command service enables you to automate common administrative tasks and perform ad hoc configuration changes at scale, however, it takes a lot of effort to implement this solution. You can use Patch Manager instead to perform the task required by the scenario since you need to perform this task with the least amount of effort.

The option that says: Set up a new patch baseline in AWS Systems Manager Patch Manager for each environment. Tag each Amazon EC2 instance based on its operating system. Categorize EC2 instances based on their tags using Patch Groups. Apply the patches specified in their corresponding patch baseline to each Patch Group. Use Patch Compliance to ensure that the patches have been installed correctly. Using AWS Config, record all of the changes to patch and association compliance statuses is incorrect because you should be tagging instances based on the environment and its OS type in which they belong and not just its OS type. This is because the type of patches that will be applied varies between the different environments. With this option, the Ubuntu EC2 instances in all of your environments, including in production, will automatically be patched.

The option that says: Use the AWS Systems Manager Maintenance Windows to set up a scheduled maintenance period for each environment, where the period is after business hours so as not to affect daily operations. The Systems Manager will execute a cron job that will install the required patches for each Amazon EC2 instance in each environment during the maintenance period. Use the Systems Manager Managed Instances to verify that your environments are fully patched and compliant is incorrect because this is not the simplest way to address the issue using AWS Systems Manager. The AWS Systems Manager Maintenance Windows feature lets you define a schedule for when to perform potentially disruptive actions on your instances such as patching an operating system, updating drivers, or installing software or patches. Each Maintenance Window has a schedule, a maximum duration, a set of registered targets (the instances that are acted upon), and a set of registered tasks. Although this solution may work, it entails a lot of configuration and effort to implement.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-patch.html>

<https://aws.amazon.com/blogs/mt/patching-your-windows-ec2-instances-using-aws-systems-manager-patch-manager/>

Check out this AWS Systems Manager Cheat Sheet:

<https://tutorialsdodo.com/aws-cheat-sheet-aws-systems-manager/>

Q8) There are times that the DevOps engineers are unable to connect to certain Amazon EC2 Windows instances and experiences boot issues. This process involves a lot of manual steps before they could regain access. The team was assigned to design a solution to automatically recover impaired Amazon EC2 instances in the company's AWS VPC. To meet the compliance requirements, It should automatically fix an EC2 instance that has become unreachable due to network misconfigurations, RDP issues, firewall settings, and many others.

Which is the MOST suitable solution that the DevOps engineers should implement to satisfy this requirement?

- Diagnose and troubleshoot problems on your Amazon EC2 Linux and Windows Server instances using the EC2Rescue tool. Run the tool automatically by using AWS OpsWorks Chef Automate and the AWSSupport-ExecuteEC2Rescue document.

- Integrate AWS Config and the AWS Systems Manager State Manager to self diagnose and troubleshoot problems on your Amazon EC2 Linux and Windows Server instances. Use AWS Systems Manager Maintenance Windows to automate the recovery process.

- ✔ Diagnose and troubleshoot problems on your Amazon EC2 Linux and Windows Server instances using the EC2Rescue tool. Automatically run the tool by using the Systems Manager Automation and the AWSSupport-ExecuteEC2Rescue document.

Explanation:- EC2Rescue can help you diagnose and troubleshoot problems on Amazon EC2 Linux and Windows Server instances. You can run the tool manually or you can run the tool automatically by using Systems Manager Automation and the AWSSupport-ExecuteEC2Rescue document. The AWSSupport-ExecuteEC2Rescue document is designed to perform a combination of Systems Manager actions, AWS CloudFormation actions, and Lambda functions that automate the steps normally required to use EC2Rescue.

Systems Manager Automation simplifies common maintenance and deployment tasks of Amazon EC2 instances and other AWS resources.

Automation enables you to do the following:

- Build Automation workflows to configure and manage instances and AWS resources.
- Create custom workflows or use pre-defined workflows maintained by AWS.
- Receive notifications about Automation tasks and workflows by using Amazon CloudWatch Events.
- Monitor Automation progress and execution details by using the Amazon EC2 or the AWS Systems Manager console.

Hence, the correct answer is: Diagnose and troubleshoot problems on your Amazon EC2 Linux and Windows Server instances using the EC2Rescue tool. Automatically run the tool by using the Systems Manager Automation and the AWSSupport-ExecuteEC2Rescue document.

The option that says: Integrate AWS Config and the AWS Systems Manager State Manager to self diagnose and troubleshoot problems on your

Amazon EC2 Linux and Windows Server instances. Use AWS Systems Manager Maintenance Windows to automate the recovery process is incorrect because AWS Config is a service that is primarily used to assess, audit, and evaluate the configurations of your AWS resources but not to diagnose and troubleshoot problems in your EC2 instances. In addition, AWS Systems Manager State Manager is primarily used as a secure and scalable configuration management service that automates the process of keeping your Amazon EC2 and hybrid infrastructure in a state that you define but does not help you in troubleshooting your EC2 instances.

The option that says: Use a combination of AWS Config and the AWS Systems Manager Session Manager to self diagnose and troubleshoot problems on your Amazon EC2 Linux and Windows Server instances. Automate the recovery process by setting up a monitoring system using CloudWatch, AWS Lambda and the AWS Systems Manager Run Command that will automatically monitor and recover impaired EC2 instances is incorrect because just like the previous option, AWS Config does not help you in troubleshooting the problems in your EC2 instances. Moreover, the AWS Systems Manager Sessions Manager simply provides secure and auditable instance management without the need to open inbound ports, maintain bastion hosts, or manage SSH keys for your EC2 instances but it does not provide the capability of helping you diagnose and troubleshoot problems in your instance like what the EC2Rescue tool can do. In addition, setting up a CloudWatch, AWS Lambda and the AWS Systems Manager Run Command that will automatically monitor and recover impaired EC2 instances is an operational overhead that can be easily done by using AWS Systems Manager Automation.

The option that says: Diagnose and troubleshoot problems on your Amazon EC2 Linux and Windows Server instances using the EC2Rescue tool. Run the tool automatically by using AWS OpsWorks Chef Automate and the AWSSupport-ExecuteEC2Rescue document is incorrect because Chef Automate is a suite of automation tools from Chef which is mainly used for configuration management, compliance and security, and continuous deployment.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/automation-ec2rescue.html>

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-automation.html>

Check out this AWS Systems Manager Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-systems-manager/>

● Use a combination of AWS Config and the AWS Systems Manager Session Manager to self diagnose and troubleshoot problems on your Amazon EC2 Linux and Windows Server instances. Automate the recovery process by setting up a monitoring system using CloudWatch, AWS Lambda, and the AWS Systems Manager Run Command that will automatically monitor and recover impaired EC2 instances.

Q9) A company has a mission-critical website hosted on-premises that is written in .NET and Angular. Its DevOps team is planning to host the site to AWS Elastic Beanstalk. The application should maintain its availability at all times to avoid any loss of revenue for the company. The existing EC2 instances should remain in service during the deployment of the succeeding application versions. A new fleet of EC2 instances should be provisioned to host the new application version. The new instances should be placed in service and the old ones should be removed after the new application version is successfully deployed in the new fleet of instances. No DNS change should also be made on the underlying resources of the environment especially the Elastic Beanstalk DNS CNAME. In the event of deployment failure, the new fleet of instances should be terminated and the current instances should continue serving traffic as normal.

As a DevOps Engineer, what deployment strategy should you implement to satisfy the requirements?

- Configure the deployment setting of the Elastic Beanstalk environment to use rolling deployments with additional batch.
- Configure the deployment setting of the Elastic Beanstalk environment to use rolling deployments.
- ✔ Configure the deployment setting of the Elastic Beanstalk environment to use immutable environment updates.

Explanation:-In ElasticBeanstalk, you can choose from a variety of deployment methods:

All at once – Deploy the new version to all instances simultaneously. All instances in your environment are out of service for a short time while the deployment occurs. This is the method that provides the least amount of time for deployment.

Rolling – Deploy the new version in batches. Each batch is taken out of service during the deployment phase, reducing your environment's capacity by the number of instances in a batch.

Rolling with additional batch – Deploy the new version in batches, but first launch a new batch of instances to ensure full capacity during the deployment process.

Immutable – Deploy the new version to a fresh group of instances by performing an immutable update.

Blue/Green - Deploy the new version to a separate environment, and then swap CNAMEs of the two environments to redirect traffic to the new version instantly.

Refer to the table below for the characteristics of each deployment method as well as the amount of time it takes to do the deployment, as seen in the Deploy Time column:

Immutable environment updates are an alternative to rolling updates. Immutable environment updates ensure that configuration changes that require replacing instances are applied efficiently and safely. If an immutable environment update fails, the rollback process requires only terminating an Auto Scaling group. A failed rolling update, on the other hand, requires performing an additional rolling update to roll back the changes.

To perform an immutable environment update, Elastic Beanstalk creates a second, temporary Auto Scaling group behind your environment's load balancer to contain the new instances. First, Elastic Beanstalk launches a single instance with the new configuration in the new group. This instance serves traffic alongside all of the instances in the original Auto Scaling group that are running the previous configuration.

When the first instance passes health checks, Elastic Beanstalk launches additional instances with the new configuration, matching the number of instances running in the original Auto Scaling group. When all of the new instances pass health checks, Elastic Beanstalk transfers them to the original Auto Scaling group, and terminates the temporary Auto Scaling group and old instances.

Hence, the correct answer is: Configure the deployment setting of the Elastic Beanstalk environment to use immutable environment updates.

The option that says: Configure the deployment setting of the Elastic Beanstalk environment to use rolling deployments is incorrect because it doesn't launch a fresh group of EC2 instances to host the new application version.

The option that says: Configure the deployment setting of the Elastic Beanstalk environment to use rolling deployments with additional batch is incorrect because the rollback process of this configuration does not meet the specified requirement. In the event of deployment failure, the existing instances are affected and could be unavailable.

The option that says: Configure the deployment setting of the Elastic Beanstalk environment to use blue/green deployment. Swap the CNAME records of the old and new environments to redirect the traffic from the old version to the new version is incorrect because the scenario explicitly mentioned that there should be no DNS change made on the underlying resources of the environment especially the Elastic Beanstalk DNS CNAME.

References:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/environmentmgmt-updates-immutable.html>

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.rolling-version-deploy.html>

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.CNAMEswap.html>

Check out this AWS Elastic Beanstalk Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-elastic-beanstalk/>

● Configure the deployment setting of the Elastic Beanstalk environment to use blue/green deployment. Swap the CNAME records of the old and new environments to redirect the traffic from the old version to the new version.

Q10) You are using CloudFormation Stacks to provision your cloud infrastructure. It is hosted on a Git repository on AWS CodeCommit. Since the stacks are an integral part of your infrastructure, you have designated your team leader as the only authority to commit changes to the master branch of your repository. You want to give full access permissions to team members on the DEV and STG branches while the team leader approves each merge request to the master branch. Which of the following options will you implement to achieve this?

- Create an IAM group for team members and another IAM group for the team leader, both with `AWSCodeCommitReadOnly` policy attached. Attach another IAM policy to the team leaders' group that allows Push, Delete, and Merge APIs of CodeCommit on the master branch.
- Create an IAM group for team members with `AWSCodeCommitReadOnly` policy attached and another IAM group for the team leader with `AWSCodeCommitFullAccess` policy attached. Attach another IAM policy to the team members' group that allows only Pull, Push, Delete and Merge APIs of CodeCommit on the DEV and STG branch.
- Create an IAM group for team members with `AWSCodeCommitPowerUser` policy attached and another IAM group for the team leader with `AWSCodeCommitFullAccess` policy attached. By default, the `AWSCodeCommitPowerUser` does not allow Delete and Merge actions on the master branch.

✔ Create an IAM group for team members and another IAM group for the team leader, both with `AWSCodeCommitPowerUser` policy attached. Attach another IAM policy to the team members' group that denies Push, Delete, and Merge APIs of CodeCommit on the master branch.

Explanation:-You want to create a policy in IAM that will deny API actions if certain conditions are met. You want to prevent team members from updating the master branch, but you don't want to prevent them from viewing the branch, cloning the repository, or creating pull requests that will merge to that branch.

Additionally, you want the team members to have full access to the DEV and STG branches. For this scenario, it is recommended to start with `FullAccess` or `PowerUser` policy which gives necessary permissions to the users for all repository/branches and then includes an additional policy that restrict certain APIs that you don't want them to perform such as Push, Delete and Merge APIs.

Hence, the correct answer is: Create an IAM group for team members and another IAM group for the team leader, both with `AWSCodeCommitPowerUser` policy attached. Attach another IAM policy to the team members' group that denies Push, Delete and Merge APIs of CodeCommit on the master branch.

The option that says: Create an IAM group for team members and another IAM group for the team leader, both with `AWSCodeCommitReadOnly` policy attached. Attach another IAM policy to the team leaders' group that allows Push, Delete and Merge APIs of CodeCommit on the master branch is incorrect because the `AWSCodeCommitReadOnly` will grant read-only permission to the team members group which restricts their permission on the repository and its branches.

The option that says: Create an IAM group for team members with `AWSCodeCommitReadOnly` policy attached and another IAM group for the team leader with `AWSCodeCommitFullAccess` policy attached. Attach another IAM policy to the team members' group that allows only Pull, Push, Delete and Merge APIs of CodeCommit on the DEV and STG branch is incorrect because this will only allow specific actions which may not be sufficient for full access of the team members on the STG and DEV branch. There are more actions needed to allow full access for the team members.

The option that says: Create an IAM group for team members with `AWSCodeCommitPowerUser` policy attached and another IAM group for the team leader with `AWSCodeCommitFullAccess` policy attached. By default, the `AWSCodeCommitPowerUser` does not allow Delete and Merge actions on the master branch is incorrect because technically, the `AWSCodeCommitPowerUser` policy does not allow repository deletion at all. Moreover, it also allows merge actions to your branch.

References:

<https://aws.amazon.com/blogs/devops/refining-access-to-branches-in-aws-codecommit/>

<https://aws.amazon.com/about-aws/whats-new/2018/05/aws-codecommit-supports-branch-level-permissions/>

<https://docs.aws.amazon.com/codecommit/latest/userguide/auth-and-access-control-permissions-reference.html>

Q11) A company has its on-premises data network connected to their AWS VPC via a Direct Connect connection. Their DevOps team is maintaining their Media Asset Management (MAM) system which uses a repository of over 50-TB digital videos and media files that are stored on their on-premises tape library. Due to the sheer size of their data, they want to implement an automated catalog system that will enable them to search their files using facial recognition. A catalog will store the faces of the people who are present in these videos including a still image of each person. Eventually, the media company would like to migrate these media files to AWS including the MAM video contents.

Which of the following provides a solution which uses the LEAST amount of ongoing management overhead and will cause MINIMAL disruption to the existing system?

● Move all of the media files from the on-premises library into an EBS volume mounted on a large Amazon EC2 instance. Set up an open-source facial recognition tool in the instance. Process the media files to retrieve the metadata and store this information into the MAM solution. Copy the media files to an Amazon S3 bucket.

● Using Amazon Kinesis Video Streams, create a video ingestion stream and build a collection of faces with Amazon Rekognition. Stream the media files from the MAM solution into Kinesis Video Streams. Configure Amazon Rekognition to process the streamed files. Set up a stream consumer to retrieve the required metadata, and store them into the MAM solution. Configure the stream to store the files in an Amazon S3 bucket.

● Launch a tape gateway appliance in your on-premises data center and connect it to your AWS Storage Gateway service. Set up the MAM solution to fetch the media files from the current archive and store them into the tape gateway in the AWS Cloud. Build a collection from the catalog of faces using Amazon Rekognition. Set up an AWS Lambda function which invokes the Rekognition Javascript SDK to have Amazon Rekognition process the video directly from the tape gateway in real-time. Retrieve the required metadata and store them into the MAM solution.

✔ Connect the on-premises file system to AWS Storage Gateway by setting up a file gateway appliance on-premises. Use the MAM solution to extract the media files from the current data store and send them into the file gateway. Populate a collection using Amazon Rekognition by building a catalog of faces from the processed media files. Launch a Lambda function to invoke Amazon Rekognition Javascript SDK to have it fetch the media files from the S3 bucket which is backing the file gateway. Retrieve the needed metadata using the Lambda function and store the information into the MAM solution.

Explanation:-Amazon Rekognition can store information about detected faces in server-side containers known as collections. You can use the facial information that's stored in a collection to search for known faces in images, stored videos, and streaming videos. Amazon Rekognition supports the `IndexFaces` operation. You can use this operation to detect faces in an image and persist information about facial features that are detected into a collection. This is an example of a storage-based API operation because the service persists information on the server.

AWS Storage Gateway offers file-based, volume-based, and tape-based storage solutions. With a tape gateway, you can cost-effectively and durably archive backup data in GLACIER or DEEP_ARCHIVE. A tape gateway provides a virtual tape infrastructure that scales seamlessly with your business needs and eliminates the operational burden of provisioning, scaling, and maintaining a physical tape infrastructure.

You can run AWS Storage Gateway either on-premises as a VM appliance, as a hardware appliance, or in AWS as an Amazon Elastic Compute Cloud (Amazon EC2) instance. You deploy your gateway on an EC2 instance to provision iSCSI storage volumes in AWS. You can use gateways hosted on EC2 instances for disaster recovery, data mirroring, and providing storage for applications hosted on Amazon EC2.

Hence, the correct answer is: Connect the on-premises file system to AWS Storage Gateway by setting up a file gateway appliance on-premises.

Use the MAM solution to extract the media files from the current data store and send them into the file gateway. Populate a collection using Amazon Rekognition by building a catalog of faces from the processed media files. Launch a Lambda function to invoke Amazon Rekognition Javascript SDK to have it fetch the media files from the S3 bucket which is backing the file gateway. Retrieve the needed metadata using the Lambda function and store the information into the MAM solution.

The option that says: Move all of the media files from the on-premises library into an EBS volume mounted on a large Amazon EC2 instance. Set up an open-source facial recognition tool in the instance. Process the media files to retrieve the metadata and store this information into the MAM solution. Copy the media files to an Amazon S3 bucket is incorrect because it entails a lot of ongoing management overhead instead of just using Amazon Rekognition. Moreover, it is more suitable to use the AWS Storage Gateway service rather than an EBS Volume.

The option that says: Launch a tape gateway appliance in your on-premises data center and connect it to your AWS Storage Gateway service. Set up the MAM solution to fetch the media files from the current archive and store them into the tape gateway in the AWS Cloud. Build a collection from the catalog of faces using Amazon Rekognition. Set up an AWS Lambda function which invokes the Rekognition Javascript SDK to have Amazon Rekognition process the video directly from the tape gateway in real-time. Retrieve the required metadata and store them into the MAM solution is incorrect because although this is using the right combination of AWS Storage Gateway and Amazon Rekognition, take note that you can't directly fetch the media files from your tape gateway in real-time since this is backed up using Glacier. Although the on-premises data center is using a tape gateway, you can still set up a solution to use a file gateway in order to properly process the videos using Amazon Rekognition. Keep in mind that the tape gateway in AWS Storage Gateway service is primarily used as an archive solution.

The option that says: Using Amazon Kinesis Video Streams, create a video ingestion stream and build a collection of faces with Amazon Rekognition. Stream the media files from the MAM solution into Kinesis Video Streams. Configure Amazon Rekognition to process the streamed files. Set up a stream consumer to retrieve the required metadata, and store them into the MAM solution. Configure the stream to store the files in an Amazon S3 bucket is incorrect because you won't be able to connect your tape gateway directly to your Kinesis Video Streams service. You need to use the AWS Storage Gateway first.

References:

<https://docs.aws.amazon.com/rekognition/latest/dg/collections.html>

<https://aws.amazon.com/storagegateway/file/>

Check out this Amazon Rekognition Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-amazon-rekognition/>

Q12) You are developing a marketplace app for emoji stickers hosted in Docker containers deployed on Amazon ECS. For each deployment, you need to verify that the set of core APIs are working before continuing to allow production traffic to flow on the new app version. If the API tests are unsuccessful, the deployment will automatically rollback to the older version. Your validation scripts are a set of functions on AWS Lambda which should run for each deployment. How can you implement this verification effectively?

- Define your Lambda functions in the AfterAllowTraffic lifecycle hook of the AppSpec.yaml file. Configure the Lambda functions to validate the deployment using production traffic and rollback if the tests fail.
- Define your Lambda functions in the AfterInstall lifecycle hook of the AppSpec.yaml file. Configure the Lambda functions to validate the deployment after installing the new version and rollback if the tests fail.
- Define your Lambda functions in the BeforeAllowTraffic lifecycle hook of the AppSpec.yaml file. Configure the Lambda functions to validate the deployment before allowing production traffic and rollback if the tests fail.
- ✔ Define your Lambda functions in the AfterAllowTestTraffic lifecycle hook of the AppSpec.yaml file. Configure the Lambda functions to validate the deployment using the test traffic and rollback if the tests fail.

Explanation:-You can use a Lambda function to validate part of the deployment of an updated Amazon ECS application. To add a validation test, you first implement the test in a Lambda function. Next, in your deployment AppSpec file, you specify the Lambda function for the lifecycle hook you want to test. If a validation test fails, the deployment stops, is rolled back, and marked failed. If the test succeeds, the deployment continues to the next deployment lifecycle event or hook. During an Amazon ECS deployment with validation tests, CodeDeploy uses a load balancer that is configured with two target groups: one production traffic listener and one test traffic listener.

The content in the 'hooks' section of the AppSpec file varies, depending on the compute platform for your deployment. The 'hooks' section for an EC2/On-Premises deployment contains mappings that link deployment lifecycle event hooks to one or more scripts. The 'hooks' section for a Lambda or an Amazon ECS deployment specifies Lambda validation functions to run during a deployment lifecycle event. If an event hook is not present, no operation is executed for that event.

When the deployment starts, the deployment lifecycle events start to execute one at a time. Some lifecycle events are hooks that only execute Lambda functions specified in the AppSpec file. An AWS Lambda hook is one Lambda function specified with a string on a new line after the name of the lifecycle event. On the AfterAllowTestTraffic hook, you can specify Lambda functions that can validate the deployment using the test traffic. Hence, the correct answer is: Define your Lambda functions in the AfterAllowTestTraffic lifecycle hook of the AppSpec.yaml file. Configure the Lambda functions to validate the deployment using the test traffic and rollback if the tests fail.

The option that says: Define your Lambda functions in the AfterInstall lifecycle hook of the AppSpec.yaml file. Configure the Lambda functions to validate the deployment after installing the new version and rollback if the tests fail is incorrect because in the AfterInstall lifecycle hook, the new task version is not yet attached to the test listener and application load balancer so no traffic is flowing yet on this cluster.

The option that says: Define your Lambda functions in the BeforeAllowTraffic lifecycle hook of the AppSpec.yaml file. Configure the Lambda functions to validate the deployment before allowing production traffic and rollback if the tests fail is incorrect because, in the BeforeAllowTraffic lifecycle hook, validation has succeeded and you can use this hook to perform additional actions before allowing production traffic to flow to the new task version.

The option that says: Define your Lambda functions in the AfterAllowTraffic lifecycle hook of the AppSpec.yaml file. Configure the Lambda functions to validate the deployment using production traffic and rollback if the tests fail is incorrect because in the AfterAllowTraffic lifecycle hook, production traffic is rerouted from the old task set to the new task set. You want to verify the application "before" opening it to production traffic, and not "after".

References:

<https://docs.aws.amazon.com/codedeploy/latest/userguide/tutorial-ecs-deployment-with-hooks.html>

<https://docs.aws.amazon.com/codedeploy/latest/userguide/reference-appspec-file-structure-hooks.html#appspec-hooks-ecs>

<https://docs.aws.amazon.com/codedeploy/latest/userguide/deployment-steps.html#deployment-steps-what-happens>

Check out this AWS CodeDeploy Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-codedeploy/>

Tutorials Dojo's AWS Certified DevOps Engineer Professional Exam Study Guide:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-certified-devops-engineer-professional/>

Q13) You are using a cluster of Amazon Linux EC2 instances to host an online forum. AWS CodeDeploy is used for deploying the new version of the application. The AppSpec file has been modified to use custom, non-default settings to fit your application deployment needs. You configured a deployment script during the lifecycle event to download the application code and install a few packages needed for operation. This deployment script completes in about 1 to 2 minutes. During deployment, CodeDeploy reports that the deployment has failed but upon checking the generated logs, the deployment script has successfully finished.

Which of the following could be the possible cause of the failed deployment?

- CodeDeploy has reached timeout because of misconfigured health check on ALB.
- The deployment script returns a non-zero value signaling CodeDeploy that it failed.

✔ CodeDeploy has reached timeout while waiting for the long-running script to finish.

Explanation:-For deployments to Amazon Linux, Ubuntu Server, and RHEL instances, if you have a deployment script that starts a long-running process, CodeDeploy might spend a long time waiting in the deployment lifecycle event and then fail the deployment. This is because if the process runs longer than the foreground and background processes in that event, CodeDeploy stops and fails the deployment, even if the process is still running as expected.

You can enable CodeDeploy to continue after the process (or script) starts running by redirecting Standard Output and Standard Input to /dev/null. Alternatively, you can also use /dev/null as Standard Input.

In CodeDeploy, you can configure a timeout as part of your deployment lifecycle event hook. The timeout value is a specific amount of time for the script to complete its execution before the deployment is considered failed. The default value is 3600 seconds (1 hour). If the script exceeds this limit, the deployment stops and the deployment to the instance fails.

The scenario mentioned that the AppSpec file has been modified to use custom, non-default settings. This means that the default timeout value has been changed and possibly set to a shorter time span (e.g. 1 minute). Therefore, this could cause an issue in this scenario in which CodeDeploy reports that the deployment has failed even though it successfully finished.

Hence, the correct answer is: CodeDeploy has reached timeout while waiting for the long-running script to finish.

The option that says: CodeDeploy has reached timeout because of misconfigured health check on ALB is incorrect because even if the ALB health checks are configured properly, the deployment will still fail if CodeDeploy reaches the custom timeout.

The option that says: The deployment script returns a non-zero value signaling CodeDeploy that it failed is incorrect because the scenario explicitly mentioned that the deployment script takes too long to execute. This has nothing to do with ALB. If the script finished running as expected but returned a non-zero value at the end, CodeDeploy may report the deployment as failed.

The option that says: The deployment script returns a zero value signaling CodeDeploy that it failed is incorrect as a return value of zero means the script finished successfully.

References:

<https://docs.aws.amazon.com/codedeploy/latest/userguide/troubleshooting-deployments.html#troubleshooting-long-running-processes>

<https://docs.aws.amazon.com/codedeploy/latest/userguide/instances.html>

Check out this AWS CodeDeploy Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-codedeploy/>

● The deployment script returns a zero value signaling CodeDeploy that it failed.

Q14) An insurance firm has a hybrid cloud architecture where its on-premises data center and VPC are connected using an AWS Direct Connect connection. They have a patch management system hosted on-premises that automatically applies the required operating system patches of their Windows and Linux servers. A DevOps Engineer was given the task to synchronize the patch baselines being used on-premises to all of the EC2 instances in your VPC, as well as to automate the patching schedule. Which of the following methods should the Engineer implement to meet the above requirement with the LEAST amount of effort?

● Set up scheduled jobs using AWS Lambda and AWS Systems Manager Run Command to automate the patching schedule. Set up AWS Systems Manager Patch Manager to manage and deploy the security patches of your Amazon EC2 instances based on the patch baselines from your on-premises network.

● Use the AWS Systems Manager Maintenance Windows to automate the patching schedule. Set up AWS Systems Manager Session Manager to manage and deploy the security patches of your Amazon EC2 instances based on the patch baselines from your on-premises network.

● Automate the patching schedule to package and distribute the required patches to your EC2 instances using AWS Systems Manager Distributor. Set up AWS Systems Manager State Manager to automate the process of keeping your Amazon EC2 and hybrid infrastructure in a state that you define, which includes the OS patches that should be applied in each EC2 instance.

✔ Install the SSM Agent to all of your Amazon EC2 instances. Use AWS Systems Manager Maintenance Windows to automate the patching schedule of your resources. Use AWS Systems Manager Patch Manager to manage and deploy the security patches of your EC2 instances based on the patch baselines from your on-premises network.

Explanation:-AWS Systems Manager Patch Manager automates the process of patching managed instances with security-related updates. For Linux-based instances, you can also install patches for non-security updates. You can patch fleets of Amazon EC2 instances or your on-premises servers and virtual machines (VMs) by operating system type. This includes supported versions of Windows, Ubuntu Server, Red Hat Enterprise Linux (RHEL), SUSE Linux Enterprise Server (SLES), Amazon Linux, and Amazon Linux 2. You can scan instances to see only a report of missing patches, or you can scan and automatically install all missing patches.

A managed instance is a machine that has been configured for use with Systems Manager. Systems Manager also helps you configure and maintain your managed instances. Supported machine types include Amazon EC2 instances, on-premises servers, and virtual machines (VMs), including VMs in other cloud environments. Supported operating system types include Windows Server, multiple distributions of Linux, and Raspbian.

AWS Systems Manager Maintenance Windows let you define a schedule for when to perform potentially disruptive actions on your instances such as patching an operating system, updating drivers, or installing software or patches. Each Maintenance Window has a schedule, a maximum duration, a set of registered targets (the instances that are acted upon), and a set of registered tasks. You can also specify dates that a Maintenance Window should not run before or after, and you can specify the international time zone on which to base the Maintenance Window schedule.

Hence, the correct answer is: Install the SSM Agent to all of your Amazon EC2 instances. Use AWS Systems Manager Maintenance Windows to automate the patching schedule of your resources. Use AWS Systems Manager Patch Manager to manage and deploy the security patches of your EC2 instances based on the patch baselines from your on-premises network.

The option that says: Use the AWS Systems Manager Maintenance Windows to automate the patching schedule. Set up AWS Systems Manager Session Manager to manage and deploy the security patches of your Amazon EC2 instances based on the patch baselines from your on-premises network is incorrect because the Session Manager is primarily used to comply with corporate policies that require controlled access to instances, strict security practices, and fully auditable logs with instance access details but not for applying OS patches. Using the AWS Systems Manager Patch Manager is a more appropriate solution to implement.

The option that says: Set up scheduled jobs using AWS Lambda and AWS Systems Manager Run Command to automate the patching schedule. Set up AWS Systems Manager Patch Manager to manage and deploy the security patches of your Amazon EC2 instances based on the patch baselines from your on-premises network is incorrect because although it properly uses AWS Systems Manager Patch Manager, it is still better to use AWS Systems Manager Maintenance Windows instead of manually creating scheduled jobs using AWS Lambda and AWS Systems Manager Run Command. Take note that the scenario specifies that you have to meet the requirement with the LEAST amount of effort, which can be met by using the AWS Systems Manager Maintenance Windows feature. In addition, installing the SSM Agent to all of your instances is also required when using the AWS Systems Manager Patch Manager, which is not mentioned in this option.

The option that says: Automate the patching schedule to package and distribute the required patches to your EC2 instances using AWS Systems Manager Distributor. Set up AWS Systems Manager State Manager to automate the process of keeping your Amazon EC2 and hybrid infrastructure in a state that you define, which includes the OS patches that should be applied in each EC2 instance is incorrect because the AWS Systems Manager State Manager is primarily used as a secure and scalable configuration management service that automates the process of keeping your Amazon EC2 and hybrid infrastructure in a state that you define. This does not handle patch management, unlike AWS Systems Manager Patch Manager. With the State Manager, you can configure your instances to boot with a specific software at start-up; download and update agents on a defined schedule; configure network settings and many others, but not the patching of your EC2 instances.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/what-is-systems-manager.html>

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-patch.html>

Check out this AWS Systems Manager Cheat Sheet:

<https://tutorialsdodo.com/aws-cheat-sheet-aws-systems-manager/>

Quick introduction on AWS Systems Manager Patch Manager:

<https://youtu.be/tnyDHqC8nDM>

Q15) A local animation film production company stores its video animations in multiple Amazon S3 buckets. A DevOps Engineer was instructed to set up a monitoring system that will notify the IT Operations team if there are any existing or new buckets that have public read or public write access. The Security Team will verify if the detected S3 buckets are indeed intended to be accessed publicly or not.

Which of the following is the MOST suitable solution that you can do in order to meet this requirement with the LEAST amount of effort?

● Develop a script that queries the settings of all buckets using the S3 API. Upload the script in a new AWS Lambda function and integrate it with CloudWatch Events in order to run it on a daily basis. Configure the script to ensure that the Amazon S3 Block Public Access option is always enabled in all of the S3 buckets. Send an email notification to the IT Operations team for all publicly accessible buckets using Amazon SNS.

✔ Set up AWS Config to monitor Amazon S3 bucket ACLs and policies for compliance violations. Create an IAM Role with the required policies that grant a Lambda function permissions to read S3 bucket policies and send alerts through SNS. Configure a CloudWatch Events rule to trigger the Lambda function when AWS Config detects an S3 bucket ACL or policy violation. Launch a Lambda function that uses the IAM role to review S3 bucket ACLs and policies. Configure the function to notify the IT Operations team of out-of-compliance policies.

Explanation:-When you turn on AWS Config, it first discovers the supported AWS resources that exist in your account and generates a configuration item for each resource. AWS Config also generates configuration items when the configuration of a resource change, and it maintains historical records of the configuration items of your resources from the time you start the configuration recorder. By default, AWS Config creates configuration items for every supported resource in the region. If you don't want AWS Config to create configuration items for all supported resources, you can specify the resource types that you want to track.

You can use AWS Config to monitor your S3 bucket ACLs and policies for violations that allow public read or public write access. If AWS Config finds a policy violation, it will trigger an Amazon CloudWatch Event rule that will then trigger an AWS Lambda function which either corrects the S3 bucket ACL or notifies you via Amazon Simple Notification Service (Amazon SNS) that the policy is in violation and allows public read or public write access. This can be done in five simple steps:

1. Enable AWS Config to monitor Amazon S3 bucket ACLs and policies for compliance violations.
2. Create an IAM Role and Policy that grants a Lambda function permissions to read S3 bucket policies and send alerts through SNS.
3. Create and configure a CloudWatch Events rule that triggers the Lambda function when AWS Config detects an S3 bucket ACL or policy violation.
4. Create a Lambda function that uses the IAM role to review S3 bucket ACLs and policies, correct the ACLs, and notify your team of out-of-compliance policies.

Hence, the correct answer is Set up AWS Config to monitor Amazon S3 bucket ACLs and policies for compliance violations. Create an IAM Role with the required policies that grant a Lambda function permissions to read S3 bucket policies and send alerts through SNS. Configure a CloudWatch Events rule to trigger the Lambda function when AWS Config detects an S3 bucket ACL or policy violation. Launch a Lambda function that uses the IAM role to review S3 bucket ACLs and policies. Configure the function to notify the IT Operations team of out-of-compliance policies.

The option that says: Develop a script that queries the settings of all buckets using the S3 API. Upload the script in a new AWS Lambda function and integrate it with CloudWatch Events in order to run it on a daily basis. Configure the script to ensure that the Amazon S3 Block Public Access option is always enabled in all of the S3 buckets. Send an email notification to the IT Operations team for all publicly accessible buckets using Amazon SNS is incorrect because it entails a lot of effort to develop a custom script. A better solution is to use AWS Config instead.

The option that says: Consolidate all of the company's AWS accounts using AWS Organizations. Create a Service Control Policy that will restrict all users from uploading public objects to any S3 bucket. Launch a Lambda function to detect if there is an attempt made by a user to upload a public object. Create an Amazon SNS topic that will notify the IT Operations team is incorrect because a service control policy (SCP) simply determines what services and actions can be delegated by administrators to the users and roles in the accounts that the SCP is applied to. In addition, setting up AWS Organizations and SCPs are not suitable in this scenario since they do not provide proper compliance monitoring as required in the scenario. The best solution here is to use AWS Config.

The option that says: Create s3-bucket-public-read-prohibited and s3-bucket-public-write-prohibited managed rules in AWS Config that will automatically notify the IT Operations team if someone uploaded publicly accessible and writable files to the Amazon S3 bucket is incorrect because although the use of these AWS Config Managed rules is correct, this solution does not automatically send a notification to your IT Operations team. You should still set up CloudWatch Events and Amazon SNS.

References:

<https://aws.amazon.com/blogs/security/how-to-use-aws-config-to-monitor-for-and-respond-to-amazon-s3-buckets-allowing-public-access/>

<https://docs.aws.amazon.com/config/latest/developerguide/WhatIsConfig.html>

Check out this AWS Config Cheat Sheet:

<https://tutorialsdodo.com/aws-cheat-sheet-aws-config/>

● Consolidate all of the company's AWS accounts using AWS Organizations. Create a Service Control Policy that will restrict all users from uploading public objects to any S3 bucket. Launch a Lambda function to detect if there is an attempt made by a user to upload a public object. Create an Amazon SNS topic that will notify the IT Operations team.

● Create s3-bucket-public-read-prohibited and s3-bucket-public-write-prohibited managed rules in AWS Config that will automatically notify the IT Operations team if someone uploaded publicly accessible and writable files to the Amazon S3 bucket.

Q16) A mobile phone manufacturer hosts a suite of enterprise resource planning (ERP) solutions to several Amazon EC2 instances in their AWS VPC. Its DevOps team is using AWS CloudFormation templates to design, launch, and deploy resources to their cloud infrastructure. Each template is manually updated to map the latest AMI IDs of the ERP solution. This process takes a significant amount of time to execute, which is why the team was tasked to automate this process.

In this scenario, which of the following options is the MOST suitable solution that can satisfy the requirement?

● Integrate AWS Service Catalog with AWS Config to automatically fetch the latest AMI and use it for succeeding deployments.

● Integrate AWS CloudFormation with AWS Service Catalog to fetch the latest AMI IDs and automatically use them for succeeding deployments.

✔ Use Systems Manager Parameter Store in conjunction with CloudFormation to retrieve the latest AMI IDs for your template. Call the update-stack API in CloudFormation in your template whenever you decide to update the Amazon EC2 instances.

Explanation:-You can use the existing Parameters section of your CloudFormation template to define Systems Manager parameters, along with other parameters. Systems Manager parameters are a unique type that is different from existing parameters because they refer to actual values in the Parameter Store. The value for this type of parameter would be the Systems Manager (SSM) parameter key instead of a string or other value. CloudFormation will fetch values stored against these keys in Systems Manager in your account and use them for the current stack operation. If the parameter being referenced in the template does not exist in Systems Manager, a synchronous validation error is thrown. Also, if you have

defined any parameter-value validations (AllowedValues, AllowedPattern, etc.) for Systems Manager parameters, they will be performed against SSM keys which are given as input values for template parameters, not actual values stored in Systems Manager.

Parameters stored in Systems Manager are mutable. Any time you use a template containing Systems Manager parameters to create/update your stacks, CloudFormation uses the values for these Systems Manager parameters at the time of the create/update operation. So, as parameters are updated in Systems Manager, you can have the new value of the parameter take effect by just executing a stack update operation. The Parameters section in the output for Describe API will show an additional 'ResolvedValue' field that contains the resolved value of the Systems Manager parameter that was used for the last stack operation.

Hence, the correct answer Use Systems Manager Parameter Store in conjunction with CloudFormation to retrieve the latest AMI IDs for your template. Call the update-stack API in CloudFormation in your template whenever you decide to update the Amazon EC2 instances.

The option that says: Set up and configure the Systems Manager State Manager service to store the latest AMI IDs and integrate it with your AWS CloudFormation template. Call the update-stack API in CloudFormation whenever you decide to update the EC2 instances in your CloudFormation template is incorrect because the Systems Manager State Manager service simply automates the process of keeping your Amazon EC2 and hybrid infrastructure in a state that you define. This can't be used as a parameter store that refers to the latest AMI of your application.

The option that says: Integrate AWS Service Catalog with AWS Config to automatically fetch the latest AMI and use it for succeeding deployments is incorrect because using AWS Service Catalog is not suitable in this scenario. This service just allows organizations to create and manage catalogs of IT services that are approved for use on AWS.

The option that says: Integrate AWS CloudFormation with AWS Service Catalog to fetch the latest AMI IDs and automatically use them for succeeding deployments is incorrect because AWS Service Catalog just allows organizations to create and manage catalogs of IT services that are approved for use on AWS. A better solution is to use Systems Manager Parameter Store in conjunction with CloudFormation to retrieve the latest AMI IDs for your template.

References:

<https://aws.amazon.com/blogs/mt/integrating-aws-cloudformation-with-aws-systems-manager-parameter-store/>

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html>

Check out this AWS Systems Manager Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-systems-manager/>

● Set up and configure the Systems Manager State Manager service to store the latest AMI IDs and integrate it with your AWS CloudFormation template. Call the update-stack API in CloudFormation whenever you decide to update the EC2 instances in your CloudFormation template.

Q17) A leading e-commerce company has a payment portal that handles the payment and refund transactions of its online platform. The portal is hosted in an Auto Scaling group of On-Demand EC2 instances across three multiple Availability Zones in the US West (N. California) region. There is a new requirement to improve the system monitoring of the application as well to track the number of payment and refund transactions being done every minute. The DevOps team should also be notified if this metric breached the specified threshold.

Which of the following options provides the MOST cost-effective and automated solution that will satisfy the above requirement?

● Configure the instances to push the entire log of each payments and refund transactions to CloudWatch Events as a custom metric. Set up a CloudWatch alarm to notify the DevOps team using SNS when the threshold is breached. View statistical graphs of your published metrics with the AWS Management Console.

● Set up an ELK stack that is composed of Elasticsearch, Logstash, and Kibana for log processing. Store the payments and refund transactions in each instance and configure Logstash to send the logs to Amazon ES. Set up a Kibana dashboard to view the data and the metric graphs.

✔ Configure the instances to push the number of payments and refund transactions to CloudWatch as a custom metric. Set up a CloudWatch alarm to notify the DevOps team using SNS when the threshold is breached. View statistical graphs of your published metrics with the AWS Management Console.

Explanation:-Metrics are data about the performance of your systems. By default, several services provide free metrics for resources (such as Amazon EC2 instances, Amazon EBS volumes, and Amazon RDS DB instances). You can also enable detailed monitoring for some resources, such as your Amazon EC2 instances, or publish your own application metrics. Amazon CloudWatch can load all the metrics in your account (both AWS resource metrics and application metrics that you provide) for search, graphing, and alarms.

Metric data is kept for 15 months, enabling you to view both up-to-the-minute data and historical data. You can publish your own metrics to CloudWatch using the AWS CLI or an API. You can view statistical graphs of your published metrics with the AWS Management Console.

CloudWatch stores data about a metric as a series of data points. Each data point has an associated timestamp. You can even publish an aggregated set of data points called a statistic set.

You can aggregate your data before you publish it to CloudWatch. When you have multiple data points per minute, aggregating data minimizes the number of calls to put-metric-data. For example, instead of calling put-metric-data multiple times for three data points that are within 3 seconds of each other, you can aggregate the data into a statistic set that you publish with one call, using the --statistic-values parameter.

Hence, the correct answer is: Configure the instances to push the number of payments and refund transactions to CloudWatch as a custom metric. Set up a CloudWatch alarm to notify the DevOps team using SNS when the threshold is breached. View statistical graphs of your published metrics with the AWS Management Console.

The option that says: Set up an ELK stack that is composed of Elasticsearch, Logstash, and Kibana for log processing. Store the payments and refund transactions in each instance and configure Logstash to send the logs to Amazon ES. Set up a Kibana dashboard to view the data and the metric graphs is incorrect because although this solution may work, it entails a lot of effort to set up and costs more to maintain. A more cost-effective solution is to use custom metrics in CloudWatch instead.

The option that says: Configure the instances to push the number of payments and refund transactions to Amazon CloudWatch Logs as a custom metric. Develop a custom monitoring application using a Python-based Flask web application to view the metrics and host it in an Amazon EC2 instance is incorrect because it will take you a considerable amount of time to set up the Flask web app. Running this solution also entails an added monthly cost.

The option that says: Configure the instances to push the entire log of each payment and refund transactions to CloudWatch Events as a custom metric. Set up a CloudWatch alarm to notify the DevOps team using SNS when the threshold is breached. View statistical graphs of your published metrics with the AWS Management Console is incorrect because you can't push a custom metric using CloudWatch Events.

References:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/publishingMetrics.html>

https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/working_with_metrics.html

Check out this Amazon CloudWatch Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-amazon-cloudwatch/>

● Configure the instances to push the number of payments and refund transactions to Amazon CloudWatch Logs as a custom metric. Develop a custom monitoring application using a Python-based Flask web application to view the metrics and host it in an Amazon EC2 instance.

Q18) You have developed a web portal for gathering Census data within your city. Each household information uploaded on the portal is stored in encrypted files on S3, and the indexes are saved on a DynamoDB table. Data security is important so you have enabled S3 access logs as well as CloudTrail to keep track of who and what accesses the S3 objects. For added security,

you want to verify that data access meets compliance standards and be alerted if there are any risk of unauthorized access or inadvertent data leaks.

Which of the following AWS services enables you to do this?

- Use Amazon GuardDuty to monitor malicious activity on your S3 data.
- Use Amazon Inspector to alert you whenever a security violation is detected on your S3 data.
- Use Amazon Rekognition to monitor and recognize patterns on your S3 data.
- ✔ Use Amazon Macie to monitor and detect usage patterns on your S3 data.

Explanation:-Amazon Macie is an ML-powered security service that helps you prevent data loss by automatically discovering, classifying, and protecting sensitive data stored in Amazon S3. Amazon Macie uses machine learning to recognize sensitive data such as personally identifiable information (PII) or intellectual property, assigns a business value, and provides visibility into where this data is stored and how it is being used in your organization.

Amazon Macie continuously monitors data access activity for anomalies, and delivers alerts when it detects risk of unauthorized access or inadvertent data leaks. Amazon Macie has ability to detect global access permissions inadvertently being set on sensitive data, detect uploading of API keys inside source code, and verify sensitive customer data is being stored and accessed in a manner that meets their compliance standards.

Hence, the correct answer is: Use Amazon Macie to monitor and detect usage patterns on your S3 data.

The option that says: Use Amazon Rekognition to monitor and recognize patterns on your S3 data is incorrect because Rekognition is simply a service that can identify the objects, people, text, scenes, and activities, as well as detect any inappropriate content on your images or videos.

The option that says: Use Amazon GuardDuty to monitor malicious activity on your S3 data is incorrect because GuardDuty is just a threat detection service that continuously monitors for malicious activity and unauthorized behavior to protect your AWS accounts and workloads.

The option that says: Amazon Inspector to alert you whenever a security violation detected on your S3 data is incorrect because Inspector is basically an automated security assessment service that helps improve the security and compliance of applications deployed on AWS.

References:

<https://docs.aws.amazon.com/macie/latest/userguide/what-is-macie.html>

<https://aws.amazon.com/macie/faq/>

<https://docs.aws.amazon.com/macie/index.html>

Check out this Amazon Macie Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-amazon-macie/>

Q19) A company is planning to implement a monitoring system that will track the cost-effectiveness of their EC2 resources across the multiple AWS accounts that they own. All of their existing resources have appropriate tags that map the corresponding environment, department, and business unit for cost allocation purposes. They instructed their DevOps engineer to automate infrastructure cost optimization across multiple shared environments and accounts such as the detection of low utilization of EC2 instances.

Which is the MOST suitable solution that the DevOps engineer should implement in this scenario?

- Set up AWS Systems Manager to track the instance utilization of all of your EC2 instances and report underutilized instances to CloudWatch. Filter the data in CloudWatch based on tags for each environment, department, and business unit. Create triggers in CloudWatch that will invoke an AWS Lambda function that will terminate underutilized EC2 instances.
- Develop a custom shell script on an EC2 instance which runs periodically to report the instance utilization of all instances and store the result into a DynamoDB table. Use a QuickSight dashboard with DynamoDB as the source data to monitor and identify the underutilized EC2 instances. Integrate Amazon QuickSight and AWS Lambda to trigger an EC2 termination command for the underutilized instances.
- Set up a CloudWatch dashboard for EC2 instance tags based on the environment, department, and business unit in order to track the instance utilization. Create a trigger using a CloudWatch Events rule and AWS Lambda to terminate the underutilized EC2 instances.
- ✔ Integrate CloudWatch Events rule and AWS Trusted Advisor to detect the EC2 instances with low utilization. Create a trigger with an AWS Lambda function that filters out the reported data based on tags for each environment, department, and business unit. Create a second trigger that will invoke another Lambda function to terminate the underutilized EC2 instances.

Explanation:-AWS Trusted Advisor draws upon best practices learned from serving hundreds of thousands of AWS customers. Trusted Advisor inspects your AWS environment, and then makes recommendations when opportunities exist to save money, improve system availability and performance, or help close security gaps. All AWS customers have access to five Trusted Advisor checks. Customers with a Business or Enterprise support plan can view all Trusted Advisor checks.

AWS Trusted Advisor is integrated with the Amazon CloudWatch Events and Amazon CloudWatch services. You can use Amazon CloudWatch Events to detect and react to changes in the status of Trusted Advisor checks. And you can use Amazon CloudWatch to create alarms on Trusted Advisor metrics for check status changes, resource status changes, and service limit utilization.

You can use Amazon CloudWatch Events to detect and react to changes in the status of Trusted Advisor checks. Then, based on the rules that you create, CloudWatch Events invokes one or more target actions when a check status changes to the value you specify in a rule. Depending on the type of status change, you might want to send notifications, capture status information, take corrective action, initiate events, or take other actions.

Hence, the correct answer is to: Integrate CloudWatch Events rule and AWS Trusted Advisor to detect the EC2 instances with low utilization. Create a trigger with an AWS Lambda function that filters out the reported data based on tags for each environment, department, and business unit. Create a second trigger that will invoke another Lambda function to terminate the underutilized EC2 instances.

The option that develops a custom shell script on an EC2 instance is incorrect because it takes time to build a custom shell script to track the EC2 instance utilization. A more suitable way is to just use AWS Trusted Advisor instead.

The option that sets up a CloudWatch dashboard for EC2 instance tags based on the environment, department, and business unit is incorrect because CloudWatch alone can't provide the instance utilization of all of your EC2 servers. You have to use AWS Trusted Advisor to get this specific data.

The option that sets up AWS Systems Manager to track the instance utilization of all of your EC2 instances is incorrect because the Systems Manager service is primarily used to manage your EC2 instances. It doesn't provide an easy way to provide the list of under or over-utilized EC2 instances like what Trusted Advisor can.

References:

<https://docs.aws.amazon.com/awssupport/latest/user/cloudwatch-events-ta.html>

<https://docs.aws.amazon.com/awssupport/latest/user/trustedadvisor.html>

<https://aws.amazon.com/premiumsupport/technology/trusted-advisor/best-practice-checklist/>

Check out this AWS Trusted Advisor Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-trusted-advisor/>

Q20) A company is using AWS OpsWorks to manage the deployments of its Auto Scaling group of EC2 instances as well as other resources. The Systems Support Team discovered that some EC2 instances in the production environment have been restarting without reason. The DevOps team has found that those instances were restarted by OpsWorks, based upon the information shown on the AWS CloudTrail logs. The Lead DevOps Engineer now wants to receive an automated email notification to improve system monitoring. The DevOps team should receive an email whenever AWS OpsWorks restarts an

instance that is considered unhealthy or unable to communicate with the service endpoint.
How can the DevOps team satisfy this requirement?

- ☐ Option 2
- ☒ Option 1

Explanation:-You can configure rules in Amazon CloudWatch Events to alert you to changes in AWS OpsWorks Stacks resources, and direct CloudWatch Events to take actions based on event contents.

Amazon CloudWatch Events delivers a near real-time stream of system events that describe changes in Amazon Web Services (AWS) resources. Using simple rules that you can quickly set up, you can match events and route them to one or more target functions or streams. CloudWatch Events becomes aware of operational changes as they occur. CloudWatch Events responds to these operational changes and takes corrective action as necessary, by sending messages to respond to the environment, activating functions, making changes, and capturing state information.

The `initiated_by` field is only populated when the instance is in the requested, terminating, or stopping states. The `initiated_by` field can contain one of the following values.

user - A user requested the instance state change by using either the API or AWS Management Console.

auto-scaling - The AWS OpsWorks Stacks automatic scaling feature initiated the instance state change.

auto-healing - The AWS OpsWorks Stacks automatic healing feature initiated the instance state change.

Hence, the correct answer is: Set up an Amazon SNS topic and create a subscription that will notify the DevOps team via email. Associate this SNS topic to new Amazon CloudWatch rule topic, which has the following configuration:

```
{
  "source": [
    "aws.opsworks"
  ],
  "detail": {
    "initiated_by": [
      "auto-healing"
    ]
  }
}
```

The option that uses a CloudWatch rule configuration with a value of "user" in its "detail.initiated_by" field is incorrect because with this configuration, the notification will only be triggered in the event that the user requested the instance state to change by using either the API or AWS Management Console.

The option that uses a CloudWatch rule configuration with a value of "auto-scaling" in its "detail.initiated_by" field is incorrect because the notification will only be triggered in the event that AWS OpsWorks Stacks automatic scaling feature initiated the instance state change. Remember that the scenario says that the DevOps team should receive an email whenever AWS OpsWorks restarts an instance that is considered unhealthy or unable to communicate with the service endpoint. Hence, this scenario is more related to auto-healing rather than not auto-scaling. Moreover, you should use SNS instead of SES.

The option that uses a CloudWatch rule configuration with a value of "auto-termination" in its "detail.initiated_by" field is incorrect because "auto-termination" is not a valid option. You should use set a value of "auto-healing" instead.

References:

<https://aws.amazon.com/premiumsupport/knowledge-center/opsworks-unexpected-start-instance/>

https://docs.aws.amazon.com/AmazonCloudWatch/latest/events/EventTypes.html#opsworks_event_types

<https://aws.amazon.com/blogs/mt/how-to-set-up-aws-opsworks-stacks-auto-healing-notifications-in-amazon-cloudwatch-events/>

Check out this Amazon CloudWatch Cheat Sheet:

<https://tutorialsdodo.com/aws-cheat-sheet-amazon-cloudwatch/>

- ☐ Option 3
- ☐ Option 4

Q21) A leading commercial bank has its online banking application hosted in AWS. It uses an encrypted Amazon S3 bucket to store the confidential files of its customers. Their DevOps team has configured federated access to a particular Active Directory user group from their on-premises network to allow access to the S3 bucket. For audit purposes, there is a new requirement to automatically detect any policy changes that are related to the restricted federated access of the bucket and to have the ability to revert any accidental changes made by the administrators.

Which of the following options provides the FASTEST way to detect configuration changes?

- ☐ Set up an AWS Config rule with a periodic trigger that runs every hour which will detect any changes in the S3 bucket configuration. Associate a Lambda function in the rule that will revert any recent changes made in the bucket.
- ☒ Set up an AWS Config rule with a configuration change trigger that will detect any changes in the S3 bucket configuration and which will also invoke an AWS Systems Manager Automation document with a Lambda function that will revert any changes.

Explanation:-AWS Config provides a detailed view of the configuration of AWS resources in your AWS account. This includes how the resources are related to one another and how they were configured in the past so that you can see how the configurations and relationships change over time. When you add a rule to your account, you can specify when you want AWS Config to run the rule; this is called a trigger. AWS Config evaluates your resource configurations against the rule when the trigger occurs.

There are two types of triggers:

- Configuration changes
- Periodic

If you choose both configuration changes and periodic triggers, AWS Config invokes your Lambda function when it detects a configuration change and also at the frequency that you specify.

Configuration changes

AWS Config runs evaluations for the rule when certain types of resources are created, changed, or deleted. You choose which resources trigger the evaluation by defining the rule's scope. The scope can include the following:

- One or more resource types
- A combination of a resource type and a resource ID
- A combination of a tag key and value
- When any recorded resource is created, updated, or deleted

AWS Config runs the evaluation when it detects a change to a resource that matches the rule's scope. You can use the scope to constrain which resources trigger evaluations. Otherwise, evaluations are triggered when any recorded resource changes.

Periodic

AWS Config runs evaluations for the rule at a frequency that you choose (for example, every 24 hours).

Hence, the correct answer is: Set up an AWS Config rule with a configuration change trigger that will detect any changes in the S3 bucket configuration and which will also invoke an AWS Systems Manager Automation document with a Lambda function that will revert any changes.

The option that says: Integrate Amazon CloudWatch Events, integrate an Event Bus with AWS CloudTrail API in order to trigger an AWS Lambda function that will detect and revert any particular changes is incorrect because although you can track all changes to your configuration using CloudTrail API, it would be difficult to integrate it with CloudWatch Events in order to monitor the changes. There is no direct way of integrating these two services and you have to create a custom mapping in order for this to work.

The option that says: Integrate CloudWatch Events and a Lambda function to create a scheduled job that runs every hour to scan the IAM policy attached to the federated access role. Configure the function to detect as well as revert any recent changes made in the current configuration is incorrect because although this solution may work, there would be a significant delay since the Lambda function is only run every one hour. So if the new S3 bucket configuration was applied at 12:05 PM, the change will only be detected at 1:00 PM. Moreover, this entails a lot of overhead since you have to develop a custom function that will scan your IAM policies.

The option that says: Set up an AWS Config rule with a periodic trigger that runs every hour which will detect any changes in the S3 bucket configuration. Associate a Lambda function in the rule that will revert any recent changes made in the bucket is incorrect because although this may work, this is not the fastest way of detecting a change in your resource configurations in AWS. Since the rule is using a periodic trigger, the rule will run every hour and not in near real-time unlike the Configuration changes trigger. So say a new configuration was applied at 12:01 PM, the change will only be detected at 1:00 PM after the rule has been run.

References:

<https://docs.aws.amazon.com/config/latest/developerguide/evaluate-config-rules.html>

https://docs.aws.amazon.com/config/latest/developerguide/evaluate-config_develop-rules_getting-started.html

Check out this AWS Config Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-config/>

- Using Amazon CloudWatch Events, integrate an Event Bus with AWS CloudTrail API in order to trigger an AWS Lambda function that will detect and revert any particular changes.

- Integrate CloudWatch Events and a Lambda function to create a scheduled job that runs every hour to scan the IAM policy attached to the federated access role. Configure the function to detect as well as revert any recent changes made in the current configuration.

Q22) A multinational investment bank has multiple AWS accounts that are consolidated using AWS organizations. Due to the strict financial regulations, all of the API calls made on each of their AWS resources across their accounts must be monitored and tracked for auditing purposes. AWS CloudTrail will be used to monitor all API activities and detect sensitive security issues in the company's AWS accounts. The DevOps Team was assigned to come up with a solution to automatically remediate CloudTrail being disabled on some AWS accounts.

As a DevOps Engineer, what solution should you apply that provides the LEAST amount of downtime for the CloudTrail log deliveries?

- Integrate CloudWatch Events and AWS Lambda to have an automated process that runs every minute to query the CloudTrail in the current account. Ensure that the Lambda function uses the AWS SDK. Once a DeleteTrail event is detected, the Lambda function will re-enable the logging for that trail by calling the CreateTrail API on the resource ARN.

- Use the cloudtrail-enabled managed rule in the AWS Systems Manager Configuration Compliance to evaluate whether the AWS account enabled AWS CloudTrail. This managed rule will automatically remediate the accounts that disabled its CloudTrail.

- Use the cloudtrail-enabled AWS Config managed rule to evaluate whether the AWS account enabled AWS CloudTrail with a trigger type of Configuration changes. This managed rule will automatically remediate the accounts that disabled its CloudTrail.

- ✔ Use the cloudtrail-enabled AWS Config managed rule with a periodic interval of 1 hour to evaluate whether your AWS account enabled the AWS CloudTrail. Set up a CloudWatch Events rule for AWS Config rules compliance change. Launch a Lambda function that uses the AWS SDK and add the Amazon Resource Name (ARN) of the Lambda function as the target in the CloudWatch Events rule. Once a StopLogging event is detected, the Lambda function will re-enable the logging for that trail by calling the StartLogging API on the resource ARN.

Explanation:- AWS Config provides AWS managed rules, which are predefined, customizable rules that AWS Config uses to evaluate whether your AWS resources comply with common best practices. For example, you could use a managed rule to quickly start assessing whether your Amazon Elastic Block Store (Amazon EBS) volumes are encrypted or whether specific tags are applied to your resources. You can set up and activate these rules without writing the code to create an AWS Lambda function, which is required if you want to create custom rules. The AWS Config console guides you through the process of configuring and activating a managed rule. You can also use the AWS Command Line Interface or AWS Config API to pass the JSON code that defines your configuration of a managed rule.

You can customize the behavior of a managed rule to suit your needs. For example, you can define the rule's scope to constrain which resources trigger an evaluation for the rule, such as EC2 instances or volumes. You can customize the rule's parameters to define attributes that your resources must have to comply with the rule. For example, you can customize a parameter to specify that your security group should block incoming traffic to a specific port number.

After you activate a rule, AWS Config compares your resources to the conditions of the rule. After this initial evaluation, AWS Config continues to run evaluations each time one is triggered. The evaluation triggers are defined as part of the rule, and they can include the following types:

Configuration changes – AWS Config triggers the evaluation when any resource that matches the rule's scope changes in configuration. The evaluation runs after AWS Config sends a configuration item change notification.

Periodic – AWS Config runs evaluations for the rule at a frequency that you choose (for example, every 24 hours).

The cloudtrail-enabled Checks whether AWS CloudTrail is enabled in your AWS account. Optionally, you can specify which S3 bucket, SNS topic, and Amazon CloudWatch Logs ARN to use.

Hence, the correct answer is: Use the cloudtrail-enabled AWS Config managed rule with a periodic interval of 1 hour to evaluate whether your AWS account enabled the AWS CloudTrail. Set up a CloudWatch Events rule for AWS Config rules compliance change. Launch a Lambda function that uses the AWS SDK and add the Amazon Resource Name (ARN) of the Lambda function as the target in the CloudWatch Events rule. Once a StopLogging event is detected, the Lambda function will re-enable the logging for that trail by calling the StartLogging API on the resource ARN.

The option that says: Use the cloudtrail-enabled AWS Config managed rule to evaluate whether the AWS account enabled AWS CloudTrail with a trigger type of Configuration changes. This managed rule will automatically remediate the accounts that disabled its CloudTrail is incorrect because AWS Config will not automatically remediate the accounts that disabled its CloudTrail. You have to manually set this up using a CloudWatch Events rule and a custom Lambda function that calls the StartLogging API to enable CloudTrail back again.

The option that says: Use the cloudtrail-enabled managed rule in the AWS Systems Manager Configuration Compliance to evaluate whether the AWS account enabled AWS CloudTrail. This managed rule will automatically remediate the accounts that disabled its CloudTrail is incorrect because the AWS Systems Manager Configuration Compliance simply scans your fleet of managed instances for patch compliance and configuration inconsistencies. This is not used in conjunction with CloudTrail. You have to use the cloudtrail-enabled AWS Config managed rule instead.

The option that says: Integrate CloudWatch Events and AWS Lambda to have an automated process that runs every minute to query the CloudTrail in the current account. Ensure that the Lambda function uses the AWS SDK. Once a DeleteTrail event is detected, the Lambda function will re-enable the logging for that trail by calling the CreateTrail API on the resource ARN is incorrect because you should detect the StopLogging event and call the StartLogging API instead to enable the CloudTrail again. The DeleteTrail and CreateTrail events, as their name implies, are simply for deleting and creating the trails respectively.

References:

<https://aws.amazon.com/blogs/mt/monitor-changes-and-auto-enable-logging-in-aws-cloudtrail/>

<https://docs.aws.amazon.com/config/latest/developerguide/cloudtrail-enabled.html>

Q23) An enterprise resource planning application is hosted in an Auto Scaling group of EC2 instances behind an Application Load Balancer which uses an Amazon DynamoDB table as its data store. For the application's reporting module, there are five Lambda functions which are reading from the DynamoDB Streams of the table to count the number of products, monitor the inventory, generate reports, move new items to a Kinesis data Firehose for analytics, and many others. The operations team discovered that in peak times, the Lambda functions are getting a stream throttling error and some of the requests fail which affects the application performance. In this scenario, which of the following is the MOST scalable and cost-effective solution that requires the LEAST amount of operational overhead?

- Create a new global secondary index (GSI) in the DynamoDB table to improve performance of the queries. Increase the allocated RCU of the table. Disable the Amazon DynamoDB streams and refactor the Lambda functions to directly query from the table.
- Delete all of the Lambda functions and move all of the processing on an ECS Cluster with Auto Scaling enabled. Set up AWS Glue to consume the DynamoDB stream which will be processed by ECS.
- ✔ Refactor your architecture to use Amazon Kinesis Adapter for real-time processing of streaming data at massive scale instead of directly consuming the stream using Lambda. Invoke the Lambda functions using Amazon Kinesis Data Analytics.

Explanation:-Using the Amazon Kinesis Adapter is the recommended way to consume streams from Amazon DynamoDB. The DynamoDB Streams API is intentionally similar to that of Kinesis Data Streams, a service for real-time processing of streaming data at massive scale. In both services, data streams are composed of shards, which are containers for stream records. Both services' APIs contain ListStreams, DescribeStream, GetShards, and GetShardIterator operations. (Although these DynamoDB Streams actions are similar to their counterparts in Kinesis Data Streams, they are not 100 percent identical.)

You can write applications for Kinesis Data Streams using the Kinesis Client Library (KCL). The KCL simplifies coding by providing useful abstractions above the low-level Kinesis Data Streams API.

As a DynamoDB Streams user, you can use the design patterns found within the KCL to process DynamoDB Streams shards and stream records. To do this, you use the DynamoDB Streams Kinesis Adapter. The Kinesis Adapter implements the Kinesis Data Streams interface so that the KCL can be used for consuming and processing records from DynamoDB Streams.

Hence, the correct answer is: Refactor your architecture to use Amazon Kinesis Adapter for real-time processing of streaming data at massive scale instead of directly consuming the stream using Lambda. Invoke the Lambda functions using Kinesis Analytics.

The option that says: Delete all of the Lambda functions and move all of the processing on an ECS Cluster with Auto Scaling enabled. Set up an AWS Glue to consume the DynamoDB stream which will be processed by ECS is incorrect because using an ECS Cluster is more expensive than using Lambda functions. The use of AWS Glue is not warranted and irrelevant here since this is just a fully-managed extract, transform, and load (ETL) service that makes it easy for customers to prepare and load their data for analytics, and not for consuming DynamoDB streams.

The option that says: Create a new local secondary index (LSI) in the DynamoDB table to improve performance of the queries. Double the allocated RCU of the table. Refactor the Lambda functions to directly query from the table and disable the DynamoDB streams. Increase the concurrency limits of each Lambda function to avoid throttling errors is incorrect because doubling the allocated RCU of the table will significantly increase the cost of your architecture and technically, this will not solve the throttling issue of the Lambda functions. Same goes with the creation of an LSI since the issue here is the consumption of the streams and not the actual DynamoDB query performance. Although increasing the concurrency limits of the Lambda function may help, this is not the most scalable solution to consume the high amount of DynamoDB Streams. The recommended way is to use an Amazon Kinesis Adapter instead in order to consume the streams at a massive scale.

The option that says: Create a new global secondary index (GSI) in the DynamoDB table to improve performance of the queries. Increase the allocated RCU of the table. Disable the Amazon DynamoDB streams and refactor the Lambda functions to directly query from the table is incorrect because just as mentioned above, adding a GSI and increasing the allocated RCU are actually not related in this issue. In fact, querying directly from the table entails a lot of operation overhead since you have to develop the queries and maintain the indices as well as the capacity units (e.g. RCU, WCU) of your table.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.KCLAdapter.html>

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html>

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-amazon-dynamodb/>

- Create a new local secondary index (LSI) in the DynamoDB table to improve performance of the queries. Double the allocated RCU of the table. Refactor the Lambda functions to directly query from the table and disable the DynamoDB streams. Increase the concurrency limits of each Lambda function to avoid throttling errors.

Q24) A multinational investment bank is using AWS Organizations to handle their multiple AWS accounts across various AWS regions around the world. To comply with the strict financial IT regulations, they have to ensure that all of their EBS volumes in all of their AWS accounts are encrypted. A DevOps engineer has been requested to set up an automated solution that will provide a detailed report of all unencrypted EBS volumes of the company as well as to notify them if there is a newly launched EC2 instance which uses an unencrypted volume. Which of the following should the DevOps engineer implement to meet this requirement with the LEAST amount of operational overhead?

- Use the AWS Systems Manager Configuration Compliance to monitor all EBS volumes across all the accounts and AWS Regions of the company. Export and store the detailed compliance report to an S3 bucket and then deliver the notifications using Amazon SNS.
- Prepare a CloudFormation template which contains an AWS Config managed rule for EBS encryption of your EBS volumes. Deploy the template across all accounts and regions of the company using the CloudFormation stack set. Store consolidated results of the AWS config rules evaluation in an Amazon S3 bucket. When non-compliant EBS resources are detected, send a notification to the Operations team using Amazon SNS.
- Configure AWS CloudTrail to deliver all events to an Amazon S3 bucket in a centralized AWS account. Run an AWS Lambda function to parse AWS CloudTrail logs whenever logs are delivered to the S3 bucket using the S3 event notification. Use the same Lambda function to publish the results to Amazon SNS.
- ✔ Set up an AWS Config rule with a corresponding Lambda function on all the target accounts of the company. Collect data from multiple accounts and AWS Regions using AWS Config aggregators. Export the aggregated report to an S3 bucket then deliver the notifications using Amazon SNS.

Explanation:-AWS Config provides a detailed view of the configuration of AWS resources in your AWS account. This includes how the resources are related to one another and how they were configured in the past so that you can see how the configurations and relationships change over time. An AWS resource is an entity you can work with in AWS, such as an Amazon Elastic Compute Cloud (EC2) instance, an Amazon Elastic Block Store (EBS) volume, a security group, or an Amazon Virtual Private Cloud (VPC).

With AWS Config, you can do the following:

- Evaluate your AWS resource configurations for desired settings.

- Get a snapshot of the current configurations of the supported resources that are associated with your AWS account.
- Retrieve configurations of one or more resources that exist in your account.
- Retrieve historical configurations of one or more resources.
- Receive a notification whenever a resource is created, modified, or deleted.
- View relationships between resources. For example, you might want to find all resources that use a particular security group.

An aggregator is an AWS Config resource type that collects AWS Config configuration and compliance data from the following:

- Multiple accounts and multiple regions.
- Single account and multiple regions.
- An organization in AWS Organizations and all the accounts in that organization.

You can use an aggregator to view the resource configuration and compliance data recorded in AWS Config.

Hence, the correct answer is: Set up an AWS Config rule with a corresponding Lambda function on all the target accounts of the company. Collect data from multiple accounts and AWS Regions using AWS Config aggregators. Export the aggregated report to an S3 bucket then deliver the notifications using Amazon SNS.

The option that says: Configure AWS CloudTrail to deliver all events to an Amazon S3 bucket in a centralized AWS account. Run an AWS Lambda function to parse AWS CloudTrail logs whenever logs are delivered to the S3 bucket using the S3 event notification. Use the same Lambda function to publish the results to Amazon SNS is incorrect because although this solution may work, it certainly entails a lot of operational overhead to execute and implement. Parsing thousands of API actions from all of your accounts in CloudTrail just to ensure that the EBS encryption was enabled on all volumes could take a significant amount of time compared with just using AWS Config.

The option that says: Prepare a CloudFormation template which contains an AWS Config managed rule for EBS encryption of your EBS volumes. Deploy the template across all accounts and regions of the company using the CloudFormation stack set. Store consolidated results of the AWS config rules evaluation in an Amazon S3 bucket. When non-compliant EBS resources are detected, send a notification to the Operations team using Amazon SNS is incorrect because although it is right to use AWS Config here, this solution still entails a lot of management overhead to maintain all of the CloudFormation templates. A better solution is to use AWS Config aggregators instead.

The option that says: Use the AWS Systems Manager Configuration Compliance to monitor all EBS volumes across all the accounts and AWS Regions of the company. Export and store the detailed compliance report to an S3 bucket and then deliver the notifications using Amazon SNS is incorrect because although you can collect and aggregate data from multiple AWS accounts and Regions using the AWS Systems Manager Configuration Compliance service, this solution has a lot of prerequisites and configuration needed. You have to install SSM agent to all of your EC2 instances, create Resource Data Syncs, set up a custom compliance type to check the EBS encryption and many others. Moreover, the AWS Systems Manager Configuration Compliance service is more suitable for verifying the patch compliance of all your resources.

References:

<https://docs.aws.amazon.com/config/latest/developerguide/aggregate-data.html>

<https://docs.aws.amazon.com/config/latest/developerguide/WhatIsConfig.html>

Check out this AWS Config Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-config/>

Q25) A multinational investment bank has several AWS accounts across the globe that host its cloud-based applications. A DevOps Engineer was tasked to secure all of the company's AWS resources as well as their web applications from common web vulnerabilities and cyber attacks. He must also safeguard their cloud infrastructure from Distributed Denial of Service attack (DDoS) in which there are numerous incoming traffic coming from many different locations that simultaneously go into the company's web application and flood their network.

Which among the options below can the Engineer implement as part of the company's DDoS attack surface reduction strategy to minimize the blast radius in their cloud infrastructure? (Select TWO)

☒ Set up AWS WAF rules that identify and block common DDoS request patterns to effectively mitigate a DDoS attack on the company's cloud infrastructure. Ensure that the Network Access Control Lists (ACLs) only allow the required ports and network addresses in the VPC.

Explanation:-A Distributed Denial of Service (DDoS) attack is a malicious attempt to make a targeted system, such as a website or application, unavailable to end-users. To achieve this, attackers use a variety of techniques that consume network or other resources, interrupting access for legitimate end-users.

Another important consideration when architecting on AWS is to limit the opportunities that an attacker may have to target your application. For example, if you do not expect an end-user to directly interact with certain resources, you will want to make sure that those resources are not accessible from the Internet. Similarly, if you do not expect end-users or external applications to communicate with your application on certain ports or protocols, you will want to make sure that traffic is not accepted. This concept is known as attack surface reduction. Resources that are not exposed to the Internet are more difficult to attack, which limits the options an attacker might have to target the availability of your application.

AWS Shield is a managed DDoS protection service that is available in two tiers: Standard and Advanced. AWS Shield Standard applies always-on detection and inline mitigation techniques, such as deterministic packet filtering and priority-based traffic shaping, to minimize application downtime and latency. AWS Shield Standard is included automatically and transparently to your Elastic Load Balancing load balancers, Amazon CloudFront distributions, and Amazon Route 53 resources at no additional cost. When you use these services that include AWS Shield Standard, you receive comprehensive availability protection against all known infrastructure layer attacks.

Customers who have the technical expertise to manage their own monitoring and mitigation of application-layer attacks can use AWS Shield together with AWS WAF rules to create a comprehensive DDoS attack mitigation strategy.

Hence, the correct answers are:

- Use AWS Shield Advanced to enable enhanced DDoS attack detection and monitoring for application-layer traffic of the company's AWS resources. Ensure that every security group in the VPC only allows certain ports and traffic from authorized servers or services. Protect your origin servers by putting it behind a CloudFront distribution.

- Set up AWS WAF rules that identify and block common DDoS request patterns to effectively mitigate a DDoS attack on the company's cloud infrastructure. Ensure that the Network Access Control Lists (ACLs) only allow the required ports and network addresses in the VPC.

The option that says: Set up AWS Systems Manager Session Manager to filter all client-side web sessions of their Amazon EC2 instances. Use extra-large EC2 instances to accommodate a surge of incoming traffic caused by a DDoS attack. Configure Elastic Load Balancing and Auto Scaling to your EC2 instances across multiple Availability Zones to improve availability and scalability to your compute capacity is incorrect because although it improves the scalability of your network in case of an ongoing DDoS attack, it simply absorbs the heavy application layer traffic and doesn't minimize the attack surface in your cloud architecture. In addition, the AWS Systems Manager Session Manager is primarily used to provide secure and auditable instance management without the need to open inbound ports, maintain bastion hosts, or manage SSH keys, but not to filter client-side web sessions.

The option that says: Set up a Multi-Account Multi-Region Data Aggregation using AWS Config to monitor all of the company's AWS accounts.

Enable the Versioning feature on all of the Amazon S3 buckets. Automate the OS patching of all of the company's Amazon EC2 instances using Systems Manager Patch Manager is incorrect because although it is recommended that all of your instances are properly patched using the Systems Manager Patch Manager, it is still not enough to protect your cloud infrastructure against DDoS attacks. The S3 Versioning feature is primarily used to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket, but not as DDoS attack mitigation.

The option that says: Enable Multi-Factor Authentication (MFA) to all of the IAM users as well as Amazon S3 buckets. Improve the security of your AWS resources using Systems Manager State Manager, AWS Config, and Trusted Advisor is incorrect because MFA doesn't minimize the DDoS

attack surface area. The AWS Systems Manager State Manager is just a configuration management service that automates the process of keeping your Amazon EC2 and hybrid infrastructure in a state that you define. This service will not help you minimize the blast radius of a security attack.

References:

- <https://aws.amazon.com/answers/networking/aws-ddos-attack-mitigation/>
- https://d1.awsstatic.com/whitepapers/Security/DDoS_White_Paper.pdf
- https://d0.awsstatic.com/whitepapers/DDoS_White_Paper_June2015.pdf

- Set up AWS Systems Manager Session Manager to filter all client-side web sessions of their Amazon EC2 instances. Use extra-large EC2 instances to accommodate a surge of incoming traffic caused by a DDoS attack. Configure Elastic Load Balancing and Auto Scaling to your EC2 instances across multiple Availability Zones to improve availability and scalability to your compute capacity.
- Enable Multi-Factor Authentication (MFA) to all of the IAM users as well as Amazon S3 buckets. Improve the security of your AWS resources using Systems Manager State Manager, AWS Config, and Trusted Advisor
- Set up a Multi-Account Multi-Region Data Aggregation using AWS Config to monitor all of the company's AWS accounts. Enable the Versioning feature on all of the Amazon S3 buckets. Automate the OS patching of all of the company's Amazon EC2 instances using Systems Manager Patch Manager.
- ✔ Use AWS Shield Advanced to enable enhanced DDoS attack detection and monitoring for application-layer traffic of the company's AWS resources. Ensure that every security group in the VPC only allows certain ports and traffic from authorized servers or services. Protect your origin servers by putting it behind a CloudFront distribution.

Explanation:-A Distributed Denial of Service (DDoS) attack is a malicious attempt to make a targeted system, such as a website or application, unavailable to end-users. To achieve this, attackers use a variety of techniques that consume network or other resources, interrupting access for legitimate end-users.

Another important consideration when architecting on AWS is to limit the opportunities that an attacker may have to target your application. For example, if you do not expect an end-user to directly interact with certain resources, you will want to make sure that those resources are not accessible from the Internet. Similarly, if you do not expect end-users or external applications to communicate with your application on certain ports or protocols, you will want to make sure that traffic is not accepted. This concept is known as attack surface reduction. Resources that are not exposed to the Internet are more difficult to attack, which limits the options an attacker might have to target the availability of your application.

AWS Shield is a managed DDoS protection service that is available in two tiers: Standard and Advanced. AWS Shield Standard applies always-on detection and inline mitigation techniques, such as deterministic packet filtering and priority-based traffic shaping, to minimize application downtime and latency. AWS Shield Standard is included automatically and transparently to your Elastic Load Balancing load balancers, Amazon CloudFront distributions, and Amazon Route 53 resources at no additional cost. When you use these services that include AWS Shield Standard, you receive comprehensive availability protection against all known infrastructure layer attacks.

Customers who have the technical expertise to manage their own monitoring and mitigation of application-layer attacks can use AWS Shield together with AWS WAF rules to create a comprehensive DDoS attack mitigation strategy.

Hence, the correct answers are:

- Use AWS Shield Advanced to enable enhanced DDoS attack detection and monitoring for application-layer traffic of the company's AWS resources. Ensure that every security group in the VPC only allows certain ports and traffic from authorized servers or services. Protect your origin servers by putting it behind a CloudFront distribution.

- Set up AWS WAF rules that identify and block common DDoS request patterns to effectively mitigate a DDoS attack on the company's cloud infrastructure. Ensure that the Network Access Control Lists (ACLs) only allow the required ports and network addresses in the VPC.

The option that says: Set up AWS Systems Manager Session Manager to filter all client-side web sessions of their Amazon EC2 instances. Use extra-large EC2 instances to accommodate a surge of incoming traffic caused by a DDoS attack. Configure Elastic Load Balancing and Auto Scaling to your EC2 instances across multiple Availability Zones to improve availability and scalability to your compute capacity is incorrect because although it improves the scalability of your network in case of an ongoing DDoS attack, it simply absorbs the heavy application layer traffic and doesn't minimize the attack surface in your cloud architecture. In addition, the AWS Systems Manager Session Manager is primarily used to provide secure and auditable instance management without the need to open inbound ports, maintain bastion hosts, or manage SSH keys, but not to filter client-side web sessions.

The option that says: Set up a Multi-Account Multi-Region Data Aggregation using AWS Config to monitor all of the company's AWS accounts. Enable the Versioning feature on all of the Amazon S3 buckets. Automate the OS patching of all of the company's Amazon EC2 instances using Systems Manager Patch Manager is incorrect because although it is recommended that all of your instances are properly patched using the Systems Manager Patch Manager, it is still not enough to protect your cloud infrastructure against DDoS attacks. The S3 Versioning feature is primarily used to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket, but not as DDoS attack mitigation.

The option that says: Enable Multi-Factor Authentication (MFA) to all of the IAM users as well as Amazon S3 buckets. Improve the security of your AWS resources using Systems Manager State Manager, AWS Config, and Trusted Advisor is incorrect because MFA doesn't minimize the DDoS attack surface area. The AWS Systems Manager State Manager is just a configuration management service that automates the process of keeping your Amazon EC2 and hybrid infrastructure in a state that you define. This service will not help you minimize the blast radius of a security attack.

References:

- <https://aws.amazon.com/answers/networking/aws-ddos-attack-mitigation/>
- https://d1.awsstatic.com/whitepapers/Security/DDoS_White_Paper.pdf
- https://d0.awsstatic.com/whitepapers/DDoS_White_Paper_June2015.pdf

Q26) You have a separate AWS account on which developers can freely spawn their own AWS resources and test their new builds. Given the lax restriction in this environment, you checked AWS Trusted Advisor and it shows that several instances use the default security group rule that opens inbound port 22 to all IP addresses. Even for a test environment, you still want to restrict the port 22 access from the Public IP of your on-premises data center only. With this, you want to be notified of any security check recommendations from Trusted Advisor and automatically solve the non-compliance based on the results. What are the steps that you should take to set up the required solution? (Select THREE)

- ✔ Create a Lambda function and integrate CloudWatch Events and AWS Lambda to execute the function on a regular schedule to check AWS Trusted Advisor via API. Based on the results, publish a message to an SNS Topic to notify the subscribers.

Explanation:-AWS Config enables you to assess, audit, and evaluate the configurations of your AWS resources. Config continuously monitors and records your AWS resource configurations and allows you to automate the evaluation of recorded configurations against desired configurations. You can configure AWS Config to stream configuration changes and notifications to an Amazon SNS topic. This way, you can be notified when AWS Config evaluates your custom or managed rules against your resources.

AWS Config now includes remediation capability with AWS Config rules. This feature gives you the ability to associate and execute remediation actions with AWS Config rules to address noncompliant resources. You can choose from a list of available remediation actions. For example, you can create an AWS Config rule to check that your Amazon S3 buckets do not allow public read access. You can then associate a remediation action to disable public access for noncompliant S3 buckets.

It's easy to set up remediation actions through the AWS Config console or API. Just choose the remediation action you want to associate from a pre-populated list, or create your own custom remediation actions using AWS Systems Manager Automation documents.

You can also write a scheduled Lambda function to check Trusted Advisor regularly. You can specify a fixed rate (for example, execute a Lambda function every hour or 15 minutes), or you can specify a Cron expression using CloudWatch Events. You can retrieve and refresh Trusted Advisor

results programmatically. The AWS Support service enables you to write applications that interact with AWS Trusted Advisor.

Hence, the correct answers are:

- Create a Lambda function and integrate CloudWatch Events and AWS Lambda to execute the function on a regular schedule to check AWS Trusted Advisor via API. Based on the results, publish a message to an SNS Topic to notify the subscribers.
- Set up custom AWS Config rule that checks security groups to make sure that port 22 is not open to public. Send a notification to an SNS topic for non-compliance.
- Set up custom AWS Config rule to execute a remediation action using AWS Systems Manager Automation to update the publicly open port 22 on the instances and restrict to only your office's public IP.

The option that says: Create an AWS Config Cron job to schedule your checks on all AWS security groups and send results to SNS for the non-compliance notification is incorrect because you do not schedule AWS Config with Cron jobs since this service can be configured to run periodically. The option that says: Setup custom AWS Config rule to execute a remediation action that triggers a Lambda Function to update the publicly open port 22 in the security group and restrict to only your office's public IP is incorrect because the custom remediation actions in AWS Config should be used in conjunction with AWS Systems Manager Automation documents.

The option that says: Create a Lambda function that executes every hour to refresh AWS Trusted Advisor scan results via API. The automated notification on AWS trusted Advisor will notify you of any changes is incorrect because the notification is only sent on a weekly basis, which can be quite long if you are concerned about security issues.

References:

<https://aws.amazon.com/about-aws/whats-new/2019/03/use-aws-config-to-remediate-noncompliant-resources/>

<https://docs.aws.amazon.com/config/latest/developerguide/notifications-for-AWS-Config.html>

<https://aws.amazon.com/blogs/aws/aws-config-rules-dynamic-compliance-checking-for-cloud-resources/>

Check out this AWS Config Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-config/>

- Create an AWS Config Cron job to schedule your checks on all AWS security groups and send results to SNS for the non-compliance notification.
- Set up custom AWS Config rule to execute a remediation action that triggers a Lambda Function to update the publicly open port 22 in the security group and restrict to only your office's public IP.
- ✔ Set up custom AWS Config rule that checks security groups to make sure that port 22 is not open to public. Send a notification to an SNS topic for non-compliance.

Explanation:-AWS Config enables you to assess, audit, and evaluate the configurations of your AWS resources. Config continuously monitors and records your AWS resource configurations and allows you to automate the evaluation of recorded configurations against desired configurations. You can configure AWS Config to stream configuration changes and notifications to an Amazon SNS topic. This way, you can be notified when AWS Config evaluates your custom or managed rules against your resources.

AWS Config now includes remediation capability with AWS Config rules. This feature gives you the ability to associate and execute remediation actions with AWS Config rules to address noncompliant resources. You can choose from a list of available remediation actions. For example, you can create an AWS Config rule to check that your Amazon S3 buckets do not allow public read access. You can then associate a remediation action to disable public access for noncompliant S3 buckets.

It's easy to set up remediation actions through the AWS Config console or API. Just choose the remediation action you want to associate from a pre-populated list, or create your own custom remediation actions using AWS Systems Manager Automation documents.

You can also write a scheduled Lambda function to check Trusted Advisor regularly. You can specify a fixed rate (for example, execute a Lambda function every hour or 15 minutes), or you can specify a Cron expression using CloudWatch Events. You can retrieve and refresh Trusted Advisor results programmatically. The AWS Support service enables you to write applications that interact with AWS Trusted Advisor.

Hence, the correct answers are:

- Create a Lambda function and integrate CloudWatch Events and AWS Lambda to execute the function on a regular schedule to check AWS Trusted Advisor via API. Based on the results, publish a message to an SNS Topic to notify the subscribers.
- Set up custom AWS Config rule that checks security groups to make sure that port 22 is not open to public. Send a notification to an SNS topic for non-compliance.
- Set up custom AWS Config rule to execute a remediation action using AWS Systems Manager Automation to update the publicly open port 22 on the instances and restrict to only your office's public IP.

The option that says: Create an AWS Config Cron job to schedule your checks on all AWS security groups and send results to SNS for the non-compliance notification is incorrect because you do not schedule AWS Config with Cron jobs since this service can be configured to run periodically. The option that says: Setup custom AWS Config rule to execute a remediation action that triggers a Lambda Function to update the publicly open port 22 in the security group and restrict to only your office's public IP is incorrect because the custom remediation actions in AWS Config should be used in conjunction with AWS Systems Manager Automation documents.

The option that says: Create a Lambda function that executes every hour to refresh AWS Trusted Advisor scan results via API. The automated notification on AWS trusted Advisor will notify you of any changes is incorrect because the notification is only sent on a weekly basis, which can be quite long if you are concerned about security issues.

References:

<https://aws.amazon.com/about-aws/whats-new/2019/03/use-aws-config-to-remediate-noncompliant-resources/>

<https://docs.aws.amazon.com/config/latest/developerguide/notifications-for-AWS-Config.html>

<https://aws.amazon.com/blogs/aws/aws-config-rules-dynamic-compliance-checking-for-cloud-resources/>

Check out this AWS Config Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-config/>

- Create a Lambda function that executes every hour to refresh AWS Trusted Advisor scan results via API. The automated notification on AWS trusted Advisor will notify you of any changes.
- ✔ Set up custom AWS Config rule to execute a remediation action using AWS Systems Manager Automation to update the publicly open port 22 on the instances and restrict to only your office's public IP.

Explanation:-AWS Config enables you to assess, audit, and evaluate the configurations of your AWS resources. Config continuously monitors and records your AWS resource configurations and allows you to automate the evaluation of recorded configurations against desired configurations. You can configure AWS Config to stream configuration changes and notifications to an Amazon SNS topic. This way, you can be notified when AWS Config evaluates your custom or managed rules against your resources.

AWS Config now includes remediation capability with AWS Config rules. This feature gives you the ability to associate and execute remediation actions with AWS Config rules to address noncompliant resources. You can choose from a list of available remediation actions. For example, you can create an AWS Config rule to check that your Amazon S3 buckets do not allow public read access. You can then associate a remediation action to disable public access for noncompliant S3 buckets.

It's easy to set up remediation actions through the AWS Config console or API. Just choose the remediation action you want to associate from a pre-populated list, or create your own custom remediation actions using AWS Systems Manager Automation documents.

You can also write a scheduled Lambda function to check Trusted Advisor regularly. You can specify a fixed rate (for example, execute a Lambda function every hour or 15 minutes), or you can specify a Cron expression using CloudWatch Events. You can retrieve and refresh Trusted Advisor results programmatically. The AWS Support service enables you to write applications that interact with AWS Trusted Advisor.

Hence, the correct answers are:

- Create a Lambda function and integrate CloudWatch Events and AWS Lambda to execute the function on a regular schedule to check AWS Trusted Advisor via API. Based on the results, publish a message to an SNS Topic to notify the subscribers.

- Set up custom AWS Config rule that checks security groups to make sure that port 22 is not open to public. Send a notification to an SNS topic for non-compliance.

- Set up custom AWS Config rule to execute a remediation action using AWS Systems Manager Automation to update the publicly open port 22 on the instances and restrict to only your office's public IP.

The option that says: Create an AWS Config Cron job to schedule your checks on all AWS security groups and send results to SNS for the non-compliance notification is incorrect because you do not schedule AWS Config with Cron jobs since this service can be configured to run periodically.

The option that says: Setup custom AWS Config rule to execute a remediation action that triggers a Lambda Function to update the publicly open port 22 in the security group and restrict to only your office's public IP is incorrect because the custom remediation actions in AWS Config should be used in conjunction with AWS Systems Manager Automation documents.

The option that says: Create a Lambda function that executes every hour to refresh AWS Trusted Advisor scan results via API. The automated notification on AWS trusted Advisor will notify you of any changes is incorrect because the notification is only sent on a weekly basis, which can be quite long if you are concerned about security issues.

References:

<https://aws.amazon.com/about-aws/whats-new/2019/03/use-aws-config-to-remediate-noncompliant-resources/>

<https://docs.aws.amazon.com/config/latest/developerguide/notifications-for-AWS-Config.html>

<https://aws.amazon.com/blogs/aws/aws-config-rules-dynamic-compliance-checking-for-cloud-resources/>

Check out this AWS Config Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-config/>

Q27) There are confidential files containing patent information that are stored on your Amazon S3 bucket in the US East (Ohio) region. Among team members working on the project, you want to be able to monitor the actions made on your objects such as PUT, GET, and DELETE operations. You want to easily search and review these actions for auditing purposes. Which of the following options will you implement to achieve this in the MOST cost-effective manner?

- Enable logging on your Amazon S3 bucket to save the access logs on a separate S3 bucket. Import logs to AWS Elasticsearch service to easily search and query the logs.
- Enable logging on your S3 bucket. Create a CloudWatch Events rule that watches the object-level events of your S3 bucket. Create a target on the rule to send the event logs to a CloudWatch Log group. View and search the logs on CloudWatch Logs.
- ✔ Create an AWS CloudTrail trail to track and store your S3 API call logs to an Amazon S3 bucket. Create a Lambda function that logs data events of your S3 bucket. Trigger this Lambda function using CloudWatch Events rule for every action taken on your S3 objects. View the logs on the CloudWatch Logs group.

Explanation:-When an event occurs in your account, CloudTrail evaluates whether the event matches the settings for your trails. Only events that match your trail settings are delivered to your Amazon S3 bucket and Amazon CloudWatch Logs log group.

You can configure your trails to log the following:

Data events: These events provide insight into the resource operations performed on or within a resource. These are also known as data plane operations.

Management events: Management events provide insight into management operations that are performed on resources in your AWS account. These are also known as control plane operations. Management events can also include non-API events that occur in your account. For example, when a user logs in to your account, CloudTrail logs the ConsoleLogin event.

You can configure multiple trails differently so that the trails process and log only the events that you specify. For example, one trail can log read-only data and management events, so that all read-only events are delivered to one S3 bucket. Another trail can log only write-only data and management events, so that all write-only events are delivered to a separate S3 bucket.

You can also configure your trails to have one trail log and deliver all management events to one S3 bucket, and configure another trail to log and deliver all data events to another S3 bucket.

Data events provide insight into the resource operations performed on or within a resource. These are also known as data plane operations. Data events are often high-volume activities.

Example data events include:

Amazon S3 object-level API activity (for example, GetObject, DeleteObject, and PutObject API operations)

AWS Lambda function execution activity (the Invoke API)

Data events are disabled by default when you create a trail. To record CloudTrail data events, you must explicitly add the supported resources or resource types for which you want to collect activity to a trail.

You can log the object-level API operations on your S3 buckets. Before Amazon CloudWatch Events can match these events, you must use AWS CloudTrail to set up a trail configured to receive these events. To log data events for an S3 bucket to AWS CloudTrail and CloudWatch Events, create a trail. A trail captures API calls and related events in your account and delivers the log files to an S3 bucket that you specify. After you create a trail and configure it to capture the log files you want, you need to be able to find the log files and interpret the information they contain.

Typically, log files appear in your bucket within 15 minutes of the recorded AWS API call or other AWS event. Then you need to create a Lambda function to log data events for your S3 bucket. Finally, you need to create a trigger to run your Lambda function in response to an Amazon S3 data event. You can create this rule on CloudWatch Events, and setting Lambda function as the target. Your logs will show up on CloudWatch log group, which you can view and search as needed

Hence, the correct answer is: Create an AWS CloudTrail trail to track and store your S3 API call logs to an Amazon S3 bucket. Create a Lambda function that logs data events of your S3 bucket. Trigger this Lambda function using CloudWatch Events rule for every action taken on your S3 objects. View the logs on the CloudWatch Logs group.

The option that says: Enable logging on your Amazon S3 bucket to save the access logs on a separate S3 bucket. Import logs to AWS Elasticsearch service to easily search and query the logs is incorrect because this would incur more cost if you provision an Elasticsearch cluster. Take note that the scenario asks for the most cost-effective solution.

The option that says: Enable logging on your S3 bucket. Create a CloudWatch Events rule that watches the object-level events of your S3 bucket.

Create a target on the rule to send the event logs to a CloudWatch Log group. View and search the logs on CloudWatch Logs is incorrect because before Amazon CloudWatch Events can match API call events. You must use AWS CloudTrail first to set up a trail configured to receive these events.

The option that says: Create an AWS CloudWatch Log group and configure S3 logging to send object-level events to this log group. View and search the event logs on the created CloudWatch log group is incorrect because you can't configure an S3 bucket to directly send access logs to a CloudWatch log group.

References:

<https://aws.amazon.com/blogs/aws/cloudtrail-update-capture-and-process-amazon-s3-object-level-api-activity/>

<https://docs.aws.amazon.com/awscloudtrail/latest/userguide/logging-management-and-data-events-with-cloudtrail.html#example-logging-all-S3-objects>

<https://docs.aws.amazon.com/awscloudtrail/latest/userguide/logging-data-events-with-cloudtrail.html>

Check out this AWS CloudTrail Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-cloudtrail/>

● Create an AWS CloudWatch Log group and configure S3 logging to send object-level events to this log group. View and search the event logs on the created CloudWatch log group.

Q28) large hospital has an online medical record system that is hosted in a fleet of Windows EC2 instances with several EBS volumes attached to it. The IT Security team mandated that the latest security patches should be installed to all of their Amazon EC2 instances using an automated patching system. They should also have to implement a functionality that checks all of their EC2 instances if they are using an approved Amazon Machine Image (AMI) in their AWS Cloud environment. The patching system should not impede developers from launching instances using an unapproved AMI, but nonetheless, they still have to be notified if there are non-compliant EC2 instances in their VPC. As a DevOps Engineer, which of the following should you implement to protect and monitor all of your instances as required above?

● Automatically patch all of your Amazon EC2 instances and detect uncompliant EC2 instances which do not use approved AMIs using AWS Shield Advanced. Create CloudWatch Alarms to notify you if there are any non-compliant instances running in your VPC.

● Use Amazon GuardDuty to continuously monitor your Amazon EC2 instances if the latest security patches are installed and also to check if there are any unapproved AMIs being used. Use CloudWatch Alarms to notify you if there are any non-compliant instances running in your VPC.

● Set up an IAM policy that will restrict the developers from launching EC2 instances with an unapproved AMI. Use CloudWatch Alarms to notify you if there are any non-compliant instances running in your VPC.

✔ Create a patch baseline in AWS Systems Manager Patch Manager that defines which patches are approved for installation on your instances. Set up the AWS Config Managed Rule which automatically checks whether your running EC2 instances are using approved AMIs. Create CloudWatch Alarms to notify you if there are any non-compliant instances running in your VPC.

Explanation:-AWS Systems Manager Patch Manager automates the process of patching managed instances with security-related updates. For Linux-based instances, you can also install patches for non-security updates. You can patch fleets of Amazon EC2 instances or your on-premises servers and virtual machines (VMs) by operating system type. This includes supported versions of Windows Server, Ubuntu Server, Red Hat Enterprise Linux (RHEL), SUSE Linux Enterprise Server (SLES), CentOS, Amazon Linux, and Amazon Linux 2. You can scan instances to see only a report of missing patches, or you can scan and automatically install all missing patches.

Patch Manager uses patch baselines, which include rules for auto-approving patches within days of their release, as well as a list of approved and rejected patches. You can install patches on a regular basis by scheduling patching to run as a Systems Manager Maintenance Window task. You can also install patches individually or to large groups of instances by using Amazon EC2 tags. You can add tags to your patch baselines themselves when you create or update them.

AWS Config provides AWS managed rules, which are predefined, customizable rules that AWS Config uses to evaluate whether your AWS resources comply with common best practices. For example, you could use a managed rule to quickly start assessing whether your Amazon Elastic Block Store (Amazon EBS) volumes are encrypted or whether specific tags are applied to your resources. You can set up and activate these rules without writing the code to create an AWS Lambda function, which is required if you want to create custom rules. The AWS Config console guides you through the process of configuring and activating a managed rule. You can also use the AWS Command Line Interface or AWS Config API to pass the JSON code that defines your configuration of a managed rule.

In this scenario, you can use a combination of AWS Config Managed Rules and AWS Systems Manager Patch Manager to meet the requirements.

Hence, the correct option is: Create a patch baseline in AWS Systems Manager Patch Manager that defines which patches are approved for installation on your instances. Set up the AWS Config Managed Rule which automatically checks whether your running EC2 instances are using approved AMIs. Create CloudWatch Alarms to notify you if there are any non-compliant instances running in your VPC.

The option that says: Set up an IAM policy that will restrict the developers from launching EC2 instances with an unapproved AMI. Use CloudWatch Alarms to notify you if there are any non-compliant instances running in your VPC is incorrect because although you can use an IAM policy to prohibit your developers from launching unapproved AMIs, this will impede their work which violates what the scenario requires. Remember that the scenario says that the system that you will implement should not impede developers from launching instances using an unapproved AMI.

The option that says: Use Amazon GuardDuty to continuously monitor your Amazon EC2 instances if the latest security patches are installed and also to check if there are any unapproved AMIs being used. Use CloudWatch Alarms to notify you if there are any non-compliant instances running in your VPC is incorrect because Amazon GuardDuty is primarily used as a threat detection service that continuously monitors for malicious or unauthorized behavior to help you protect your AWS accounts and workloads. It monitors for activity such as unusual API calls or potentially unauthorized deployments that indicate a possible account compromise, however, it does not check if your EC2 instances are using an approved AMI or not.

The option that says: Automatically patch all of your Amazon EC2 instances and detect uncompliant EC2 instances which do not use approved AMIs using AWS Shield Advanced. Create CloudWatch Alarms to notify you if there are any non-compliant instances running in your VPC is incorrect because the AWS Shield Advanced service is more suitable to be used to prevent DDoS attacks in your AWS resources. It cannot check the specific AMIs that your EC2 instances are using.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-patch.html>

https://docs.aws.amazon.com/config/latest/developerguide/evaluate-config_use-managed-rules.html

<https://docs.aws.amazon.com/config/latest/developerguide/approved-amis-by-id.html>

Check out these cheat sheets on AWS Config and AWS Systems Manager:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-config/>

<https://tutorialsdojo.com/aws-cheat-sheet-aws-systems-manager/>

Q29) You are a project owner of a global online marketplace application that is composed of several components. The application code is hosted in an AWS CodeCommit repository. Due to the project complexity and the large pool of developers concurrently working on the project, the source code repository had accumulated over a hundred different branches for its development, testing, and production environments. This setup has caused you to lose track of all the relevant code changes. To have better visibility, you want to capture all events in the repository, such as cloning a repo or creating a branch on a single location for you to review.

Which of the following actions will you take to achieve this?

● Create a CloudWatch Log group. On CodeCommit, go to repository settings and select the Notifications tab. Select all the events you want to capture and choose the Log group as the target for you notifications.

● Create a new Amazon SNS topic. On CodeCommit, go to the repository settings and select the SNS topic as the destination on the Events Tab. Select the events you want to capture and these will be sent to the SNS topic and notify you when they occur.

✔ Create a Lambda function that sends event logs to AWS CloudWatch Logs and set a trigger by selecting the CodeCommit repository. On CodeCommit, go to the repository settings and select the Lambda function on the Triggers list when creating a new trigger.

Explanation:-You can set up notification rules for a repository so that repository users receive emails about the repository event types you specify. Notifications are sent when events match the notification rule settings. You can create an Amazon SNS topic to use for notifications or use an existing one in your AWS account. You use the CodeCommit console and the AWS CLI to configure notifications.

Although you can configure a trigger to use Amazon SNS to send emails about some repository events, those events are limited to operational

events, such as creating branches and pushing code to a branch. Triggers do not use CloudWatch Events rules to evaluate repository events. They are more limited in scope.

You can integrate Amazon SNS topics and Lambda functions with triggers in CodeCommit, but you must first create and then configure resources with a policy that grants CodeCommit the permissions to interact with those resources. You must create the resource in the same AWS Region as the CodeCommit repository. For example, if the repository is in US East (Ohio) (us-east-2), the Amazon SNS topic or Lambda function must be in US East (Ohio).

Hence, the correct answer is: Create a Lambda function that sends event logs to AWS CloudWatch Logs and set a trigger by selecting the CodeCommit repository. On CodeCommit, go to the repository settings and select the Lambda function on the Triggers list when creating a new trigger.

The option that says: Create a new Amazon SNS topic. On CodeCommit, go to the repository settings and select the SNS topic as the destination on the Events tab. Select the events you want to capture and these will be sent to the SNS topic and notify you when they occur is incorrect because there is no Events tab on the CodeCommit repository settings. The repository notification setting is the appropriate one that you should configure.

The option that says: Create an Amazon SES topic and subscribe to it. On CodeCommit, go to repository settings and select the Notifications tab. Select all the events you want to capture and choose the SES topic as the target for your notification is incorrect because you have to use Amazon SNS instead of Amazon SES.

The option that says: Create a CloudWatch Log group. On CodeCommit, go to repository settings and select the Notifications tab. Select all the events you want to capture and choose the CloudWatch Log group as the target for you notifications is incorrect because you can't send notifications directly to CloudWatch Logs. You have to use Amazon SNS or a Lambda trigger first.

References:

<https://docs.aws.amazon.com/codecommit/latest/userguide/how-to-notify-lambda.html>

<https://docs.aws.amazon.com/codecommit/latest/userguide/how-to-repository-email.html>

<https://docs.aws.amazon.com/codecommit/latest/userguide/how-to-notify.html>

- Create an Amazon SES topic and subscribe to it. On CodeCommit, go to repository settings and select the Notifications tab. Select all the events you want to capture and choose the SES topic as the target for your notification.

Q30) A multinational insurance firm has recently consolidated its multiple AWS accounts using AWS Organizations with several Organizational units (OUs) that group each department of the firm. Their IT division consists of two teams: the DevOps team and the Release & Deployment team. The first one is responsible for protecting its cloud infrastructure and ensuring that all of its AWS resources are compliant, while the latter is responsible for deploying new applications to AWS Cloud. The DevOps team has been instructed to set up a system that regularly checks if all of the running EC2 instances are using an approved AMI. However, the solution should not stop the Release & Deployment team from deploying an EC2 instance running on a non-approved AMI for their release process. The DevOps team must build a notification system that sends the compliance state of the AWS resources to improve and maintain the security of their cloud resources. Which of the following is the MOST suitable solution that the DevOps team should implement?

- Automatically check all of the AMIs that are being used by your EC2 instances using the Amazon Inspector service. Use an SNS topic that will send a notification to both the DevOps and Release & Deployment teams if there is a non-compliant EC2 instance running in their VPCs.
- Verify whether the running EC2 instances in your VPCs are using approved AMIs using Trusted Advisor checks. Set up a CloudWatch alarm and integrate it with the Trusted Advisor metrics that will check all of the AMIs being used by your Amazon EC2 instances and that will send a notification to both teams if there is a running instance which uses an unapproved AMI.
- Assign a Service Control Policy and an IAM policy that restricts the AWS accounts and the development team from launching an EC2 instance using an unapproved AMI. Set up a CloudWatch alarm that will automatically notify the DevOps team if there are non-compliant EC2 instances running in their VPCs.
- ✔ Set up an AWS Config Managed Rule and specify a list of approved AMI IDs. Modify the rule to check whether the running Amazon EC2 instances are using specified AMIs. Configure AWS Config to stream configuration changes and notifications to an Amazon SNS topic which will send a notification to both teams for non-compliant instances.

Explanation:-When you run your applications on AWS, you usually use AWS resources, which you must create and manage collectively. As the demand for your application keeps growing, so does your need to keep track of your AWS resources. AWS Config is designed to help you oversee your application resources.

AWS Config provides a detailed view of the configuration of AWS resources in your AWS account. This includes how the resources are related to one another and how they were configured in the past so that you can see how the configurations and relationships change over time. With AWS Config, you can do the following:

- Evaluate your AWS resource configurations for desired settings.
- Get a snapshot of the current configurations of the supported resources that are associated with your AWS account.
- Retrieve configurations of one or more resources that exist in your account.
- Retrieve historical configurations of one or more resources.
- Receive a notification whenever a resource is created, modified, or deleted.
- View relationships between resources. For example, you might want to find all resources that use a particular security group.

AWS Config provides AWS managed rules, which are predefined, customizable rules that AWS Config uses to evaluate whether your AWS resources comply with common best practices. In this scenario, you can use the approved-amis-by-id AWS managed rule which checks whether running instances are using specified AMIs.

Hence, the correct answer is: Set up an AWS Config Managed Rule and specify a list of approved AMI IDs. Modify the rule to check whether the running Amazon EC2 instances are using specified AMIs. Configure AWS Config to stream configuration changes and notifications to an Amazon SNS topic which will send a notification to both teams for non-compliant instances.

The option that says: Assign a Service Control Policy and an IAM policy that restricts the AWS accounts and the development team from launching an EC2 instance using an unapproved AMI. Set up a CloudWatch alarm that will automatically notify the DevOps team if there are non-compliant EC2 instances running in their VPCs is incorrect because setting up an SCP and IAM Policy will totally restrict the Release & Deployment team from launching EC2 instances with unapproved AMIs. The scenario clearly says that the solution should not have this kind of restriction.

The option that says: Automatically check all of the AMIs that are being used by your EC2 instances using the Amazon Inspector service. Use an SNS topic that will send a notification to both the DevOps and Release & Deployment teams if there is a non-compliant EC2 instance running in their VPCs is incorrect because the Amazon Inspector service is just an automated security assessment service that helps improve the security and compliance of applications deployed on AWS. It does not have the capability to detect EC2 instances that are using unapproved AMIs, unlike AWS Config.

The option that says: Verify whether the running EC2 instances in your VPCs are using approved AMIs using Trusted Advisor checks. Set up a CloudWatch alarm and integrate it with the Trusted Advisor metrics that will check all of the AMIs being used by your Amazon EC2 instances and that will send a notification to both teams if there is a running instance which uses an unapproved AMI is incorrect because AWS Trusted Advisor is primarily used to check if your cloud infrastructure is in compliance with the best practices and recommendations across five categories: cost optimization, security, fault tolerance, performance, and service limits. Their security checks for EC2 does not cover the checking of individual AMIs that are being used by your EC2 instances.

References:

Q31) An IT consulting firm has a Docker application hosted on an Amazon ECS cluster in their AWS VPC. It has been encountering intermittent unavailability issues and time outs lately, which affects their production environment. A DevOps engineer was instructed to instrument the application to detect where high latencies are occurring and to determine the specific services and paths impacting application performance. Which of the following steps should the Engineer do to accomplish this task properly?

- Add the xray-daemon.config configuration file in your Docker image. Set up the port mappings and network mode settings in your task definition file to allow traffic on UDP port 2000.
- In the Amazon ECS container agent configuration, pass a user data script in the /etc/ecs/ecs.config file that will install the X-Ray daemon. The script will automatically run when the Amazon ECS container instance is launched. Configure the network mode settings and port mappings in the container agent to allow traffic on TCP port 3000.
- Produce a Docker image that runs the X-Ray daemon. Upload the image to a Docker image repository and then deploy it to your Amazon ECS cluster. Configure the network mode settings and port mappings in the container agent to allow traffic on TCP port 2000.
- ✔ Produce a Docker image that runs the X-Ray daemon. Upload the image to a Docker image repository, and then deploy it to your Amazon ECS cluster. Configure the network mode settings and port mappings in your task definition file to allow traffic on UDP port 2000.

Explanation:-The AWS X-Ray SDK does not send trace data directly to AWS X-Ray. To avoid calling the service every time your application serves a request, the SDK sends the trace data to a daemon, which collects segments for multiple requests and uploads them in batches. Use a script to run the daemon alongside your application.

To properly instrument your applications in Amazon ECS, you have to create a Docker image that runs the X-Ray daemon, upload it to a Docker image repository, and then deploy it to your Amazon ECS cluster. You can use port mappings and network mode settings in your task definition file to allow your application to communicate with the daemon container.

The AWS X-Ray daemon is a software application that listens for traffic on UDP port 2000, gathers raw segment data, and relays it to the AWS X-Ray API. The daemon works in conjunction with the AWS X-Ray SDKs and must be running so that data sent by the SDKs can reach the X-Ray service.

Hence, the correct steps to properly instrument the application is to: Produce a Docker image that runs the X-Ray daemon. Upload the image to a Docker image repository, and then deploy it to your Amazon ECS cluster. Configure the network mode settings and port mappings in your task definition file to allow traffic on UDP port 2000.

The option that says: Produce a Docker image that runs the X-Ray daemon. Upload the image to a Docker image repository, and then deploy it to your Amazon ECS cluster. Configure the network mode settings and port mappings in the container agent to allow traffic on TCP port 2000 is incorrect because this should be done in the task definition and not in the container agent. Moreover, X-Ray is primarily using the UDP port 2000 so this should also be added alongside the TCP port mapping.

The option that says: In the Amazon ECS container agent configuration, pass a user data script in the /etc/ecs/ecs.config file that will install the X-Ray daemon. The script will automatically run when the Amazon ECS container instance is launched. Configure the network mode settings and port mappings in the container agent to allow traffic on TCP port 3000 is incorrect because X-Ray is using the UDP port 2000 and not TCP port 3000. In addition, it is better to configure your ECS task definitions instead of the Amazon ECS container agent to enable X-Ray.

The option that says: Add the xray-daemon.config configuration file in your Docker image. Set up the port mappings and network mode settings in your task definition file to allow traffic on UDP port 2000 is incorrect because this step is not suitable for ECS. The xray-daemon.config configuration file is primarily used in Elastic Beanstalk.

References:

<https://docs.aws.amazon.com/xray/latest/devguide/xray-daemon-ecs.html>

<https://docs.aws.amazon.com/xray/latest/devguide/xray-daemon.html>

<https://docs.aws.amazon.com/xray/latest/devguide/scorekeep-ecs.html>

Check out this AWS X-Ray Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-x-ray/>

Instrumenting your Application with AWS X-Ray:

<https://tutorialsdojo.com/aws-cheat-sheet-instrumenting-your-application-with-aws-x-ray/>

Q32) A leading technology company with a hybrid cloud architecture has a suite of web applications that is composed of 50 modules. Each of the module is a multi-tiered application hosted in an Auto Scaling group of On-Demand EC2 instances behind an ALB with an external Amazon RDS. The Application Security team is mandated to block access from external IP addresses and only allow access to the 50 applications from the corporate data center. A group of 10 proxy servers with an associated IP address each are used for the corporate network to connect to the Internet. The 10 proxy IP addresses are being refreshed twice a month. The Network team uploads a CSV file that contains the latest proxy IP addresses into a private S3 bucket. The DevOps Engineer must build a solution to ensure that the applications are accessible from the corporate network in the most cost-effective way and with minimal operational effort. As a DevOps Engineer, how can you meet the above requirement?

- Configure the ELB security groups to allow HTTPS inbound access from the Internet. Set up Amazon Cognito to integrate the company's Active Directory as the identity provider. Integrate all of the 50 modules with Amazon Cognito to ensure that only the company employees can log into the application. Store the user access logs to Amazon CloudWatch Logs to record user access activities. Use AWS Config for configuration management that runs twice a month to update the settings accordingly.
- Host all of the applications and modules in the same Virtual Private Cloud (VPC). Set up a Direct Connect connection with an active/standby configuration. Update the ELB security groups to allow only inbound HTTPS connections from the corporate network IP addresses.
- Develop a custom Python-based Bolo script using the AWS SDK for Python. Configure the script to download the CSV file that contains the proxy IP addresses and update the ELB security groups to allow only HTTPS inbound from the given IP addresses. Host the script in a Lambda function and run it every minute using CloudWatch Events.
- ✔ Launch a Lambda function to read the list of proxy IP addresses from the S3 bucket. Configure the function to update the ELB security groups to allow HTTPS requests only from the given IP addresses. Use the Amazon S3 Event Notification to automatically invoke the Lambda function when the CSV file is updated.

Explanation:-WS Lambda is a serverless compute service that runs your code in response to events and automatically manages the underlying compute resources for you. You can use AWS Lambda to extend other AWS services with custom logic, or create your own back-end services that operate at AWS scale, performance, and security. AWS Lambda can automatically run code in response to multiple events, such as HTTP requests via Amazon API Gateway, modifications to objects in Amazon S3 buckets, table updates in Amazon DynamoDB, and state transitions in AWS Step Functions.

Lambda runs your code on high-availability compute infrastructure and performs all the administration of the compute resources, including server

and operating system maintenance, capacity provisioning and automatic scaling, code and security patch deployment, and code monitoring and logging. All you need to do is supply the code.

Lambda does not enforce any restrictions on your function logic – if you can code for it, you can run it within a Lambda function. As part of your function, you may need to call other APIs, or access other AWS services like databases.

By default, your service or API must be accessible over the public internet for AWS Lambda to access it. However, you may have APIs or services that are not exposed this way. Typically, you create these resources inside Amazon Virtual Private Cloud (Amazon VPC) so that they cannot be accessed over the public Internet. These resources could be AWS service resources, such as Amazon Redshift data warehouses, Amazon ElastiCache clusters, or Amazon RDS instances. They could also be your own services running on your own EC2 instances. By default, resources within a VPC are not accessible from within a Lambda function.

AWS Lambda runs your function code securely within a VPC by default. However, to enable your Lambda function to access resources inside your private VPC, you must provide additional VPC-specific configuration information that includes VPC subnet IDs and security group IDs. AWS Lambda uses this information to set up elastic network interfaces (ENIs) that enable your function to connect securely to other resources within your private VPC.

Hence, the correct answer is: Launch a Lambda function to read the list of proxy IP addresses from the S3 bucket. Configure the function to update the ELB security groups to allow HTTPS requests only from the given IP addresses. Use the Amazon S3 Event Notification to automatically invoke the Lambda function when the CSV file is updated.

The option that says: Host all of the applications and modules in the same Virtual Private Cloud (VPC). Set up a Direct Connect connection with an active/standby configuration. Update the ELB security groups to allow only inbound HTTPS connections from the corporate network IP addresses is incorrect because setting up a Direct Connect connection costs a significant amount of money. Remember that the scenario says that you have to ensure that the applications are accessible from the corporate network in the most cost-effective way and with minimal operational effort.

The option that says: Develop a custom Python-based Bolo script using the AWS SDK for Python. Configure the script to download the CSV file that contains the proxy IP addresses and update the ELB security groups to allow only HTTPS inbound from the given IP addresses. Host the script in a Lambda function and run it every minute using CloudWatch Events is incorrect because running the Lambda function every minute will increase your compute costs. A better solution is to use Amazon S3 Event Notification to automatically invoke the Lambda function when the CSV file is updated.

The option that says: Configure the ELB security groups to allow HTTPS inbound access from the Internet. Set up Amazon Cognito to integrate the company's Active Directory as the identity provider. Integrate all of the 50 modules with Amazon Cognito to ensure that only the company employees can log into the application. Store the user access logs to Amazon CloudWatch Logs to record user access activities. Use AWS Config for configuration management that runs twice a month to update the settings accordingly is incorrect because using Amazon Cognito in this scenario is not warranted and unnecessary as well as the use of AWS Config. Using AWS Lambda can fulfill the requirements in this scenario.

References:

<https://aws.amazon.com/blogs/security/how-to-set-up-an-outbound-vpc-proxy-with-domain-whitelisting-and-content-filtering/>

<https://aws.amazon.com/blogs/security/how-to-add-dns-filtering-to-your-nat-instance-with-squid/>

<https://docs.aws.amazon.com/AmazonS3/latest/dev/NotificationHowTo.html>

Q33) You have migrated your application API server from a cluster of EC2 instances to a combination of API gateway and AWS Lambda. You are used to canary deployments on your EC2 cluster where you carefully check any errors on the application before doing the full deployment. However, you can't do this on your current AWS Lambda setup since the deployment switches quickly from one version to another.

How can you implement the same functionality on AWS Lambda?

- ☐ Deploy your app using Traffic shifting with Amazon Route 53.
- ☐ Use Route 53 weighted routing policy with API Gateway.
- ☐ Use CodeDeploy to perform rolling update of the latest Lambda function.
- ☒ Deploy your app using Traffic shifting with AWS Lambda aliases.

Explanation:-By default, an alias points to a single Lambda function version. When the alias is updated to point to a different function version, incoming request traffic in turn instantly points to the updated version. This exposes that alias to any potential instabilities introduced by the new version. To minimize this impact, you can implement the routing-config parameter of the Lambda alias that allows you to point to two different versions of the Lambda function and dictate what percentage of incoming traffic is sent to each version.

With the introduction of alias traffic shifting, it is now possible to trivially implement canary deployments of Lambda functions. By updating additional version weights on an alias, invocation traffic is routed to the new function versions based on the weight specified. Detailed CloudWatch metrics for the alias and version can be analyzed during the deployment, or other health checks performed, to ensure that the new version is healthy before proceeding.

Hence, the correct answer is: Deploy your app using Traffic shifting with AWS Lambda aliases.

The option that says: Use CodeDeploy to perform rolling update of the latest Lambda function is incorrect because AWS Lambda does not support native rolling update. Deployments of a Lambda function could only be performed in a single flip by updating the function code for version \$LATEST, or by updating an alias to target a different function version.

The option that says: Deploy your app using Traffic shifting with Amazon Route 53 is incorrect because Route 53 does not support traffic shifting for Lambda deployments.

The option that says: Use Route 53 weighted routing policy with API Gateway is incorrect because you can't use a weighted routing policy with Route 53 and API gateway.

References:

<https://aws.amazon.com/blogs/compute/implementing-canary-deployments-of-aws-lambda-functions-with-alias-traffic-shifting/>

<https://docs.aws.amazon.com/lambda/latest/dg/lambda-traffic-shifting-using-aliases.html>

<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/automating-updates-to-serverless-apps.html>

Check out this AWS Lambda Cheat Sheet:

<https://tutorialsdojo.com/aws-lambda/>

Q34) You are hosting your application code on AWS CodeCommit in US East (Ohio) region. You want to be notified when anyone pushes code to the master branch so you have created an AWS SNS topic on the US East (N. Virginia) region, on which your corporate Amazon SES email domain is also hosted. On CodeCommit, you have created a trigger that will publish a message to your SNS topic. However, your CodeCommit trigger is not working as expected.

Which of the following is the MOST likely cause of this issue?

- ☐ You need to input the full ARN of the SNS topic on CodeCommit trigger destination since your SNS topic is on a different region from your CodeCommit repository.
- ☐ AWS CodeCommit needs to have permission to interact with Amazon SNS. Configure the necessary policy for CodeCommit to publish to Amazon SNS topic.
- ☐ AWS CodeCommit needs to have permission to interact with both Amazon SNS and SES. Configure the necessary policy for CodeCommit to publish to Amazon SNS topic and to use SES to send emails.
- ☒ Your CodeCommit repository and AWS SNS topic must be on the same region for the triggers to work.

Explanation:-You can configure a CodeCommit repository so that code pushes or other events trigger actions, such as sending a notification from Amazon Simple Notification Service (Amazon SNS) or invoking a function in AWS Lambda. You can create up to 10 triggers for each CodeCommit repository.

Triggers are commonly configured to:

- Send emails to subscribed users every time someone pushes to the repository.
- Notify an external build system to start a build after someone pushes to the main branch of the repository.

Scenarios like notifying an external build system require writing a Lambda function to interact with other applications. The email scenario simply requires creating an Amazon SNS topic.

You can integrate Amazon SNS topics and Lambda functions with triggers in CodeCommit, but you must first create and then configure resources with a policy that grants CodeCommit the permissions to interact with those resources. You must create the resource in the same AWS Region as the CodeCommit repository. For example, if the repository is in US East (Ohio) (us-east-2), the Amazon SNS topic or Lambda function must be in US East (Ohio).

Hence, the correct answer is: Your CodeCommit repository and Amazon SNS topic must be on the same region for the triggers to work.

The option that says: AWS CodeCommit needs to have permission to interact with Amazon SNS. Configure the necessary policy for CodeCommit to publish to Amazon SNS topic is incorrect because, for Amazon SNS topics, you do not need to configure additional IAM policies or permissions if the Amazon SNS topic is created using the same account as the CodeCommit repository. However this will still not work if the SNS topic and CodeCommit repository are on a different region.

The option that says: You need to input the full ARN of the SNS topic on CodeCommit trigger destination since your SNS topic is on a different region from your CodeCommit repository is incorrect because even if you do this, the problem will still persist. Your CodeCommit repository and the SNS topic must be created in the same AWS region in order to resolve this issue.

The option that says: AWS CodeCommit needs to have permission to interact with both AWS SNS and SES. Configure the necessary policy for CodeCommit to publish to AWS SNS topic and to use SES to send emails is incorrect. CodeCommit does not need permission to SES as SNS will handle that. And you don't have to configure additional IAM policies for SNS if your repository is on the same account.

References:

<https://docs.aws.amazon.com/codecommit/latest/userguide/how-to-notify.html>

<https://docs.aws.amazon.com/codecommit/latest/userguide/how-to-notify-sns.html>

<https://docs.aws.amazon.com/codecommit/latest/userguide/how-to-notify-edit.html>

Check out this AWS CodeCommit Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-codecommit/>

Q35) Your Data Analytics team uses a large Hadoop cluster with 100 EC2 instance nodes for gathering trends about user behavior on your mobile app. Every time you receive an email notification about an instance that will need maintenance or will be retired by Amazon, you manually forward it to the Data Analytics team. They need to determine how the maintenance will affect their cluster and monitor how the Hadoop Distributed File System (HDFS) recovers. Which of the following is the FASTEST way to automate this notification process?

- Add your Data Analytics team email as an alternate contact for your AWS account. They will receive these EC2 maintenance emails directly so you won't have to forward it to them.
- Create a CloudWatch Metric Filter for AWS Health Events of your related EC2 instances. Create an Alarm for this metric to notify you whenever the threshold is reached.
- ✔ Create an AWS CloudWatch Events rule for AWS Health. Select EC2 Service and select the Events you want to get notified. Set the target to an Amazon SNS topic that the Data Analytics team is subscribed to.

Explanation:-You can use Amazon CloudWatch Events to detect and react to changes in the status of AWS Personal Health Dashboard (AWS Health) events. Then, based on the rules that you create, CloudWatch Events invokes one or more target actions when an event matches the values that you specify in a rule.

Depending on the type of event, you can send notifications, capture event information, take corrective action, initiate events, or take other actions. In this scenario, you first need to create an Events rule for the AWS Health, and choose the "EC2 service" as well as specific categories of events or scheduled changes that are planned to your account.

Then, you can then set the Target of this rule to an SNS topic on which the Data Analytics team will subscribe. For every event that matches this event rule, a notification will be sent to the subscribers of your SNS topic.

Hence, the correct answer is: Create an AWS CloudWatch Events rule for AWS Health. Select EC2 Service and select the Events you want to get notified. Set the target to an Amazon SNS topic that the Data Analytics team is subscribed to.

The option that says: Create a CloudWatch Metric Filter for AWS Health Events of your related EC2 instances. Create an Alarm for this metric to notify you whenever the threshold is reached is incorrect because you can only create Metric filter for CloudWatch log groups. AWS Health events are not automatically sent to CloudWatch Log groups.

The option that says: Create a CloudWatch Metric Filter for AWS Health Events of your related EC2 instances. Set the Metric Target to an SNS topic that will send messages to the Data Analytics team whenever an event is matched is incorrect because this set up is not possible as there is no "Metric Target" field on the AWS CloudWatch console.

The option that says: Add your Data Analytics team email as an alternate contact for your AWS account. They will receive these EC2 maintenance emails directly, so you won't have to forward it to them is incorrect because the Data Analytics team will also receive other financial or account/billing related emails using this setup. This is a security risk since the Data Analytics team email could potentially be compromised and be used to access your AWS account.

References:

<https://docs.aws.amazon.com/health/latest/ug/cloudwatch-events-health.html>

<https://aws.amazon.com/blogs/aws/new-cloudwatch-events-track-and-respond-to-changes-to-your-aws-resources/>

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/automating_with_cloudwatch_events.html

- Create a CloudWatch Metric Filter for AWS Health Events of your related EC2 instances. Set the Metric Target to an SNS topic that will send messages to the Data Analytics team whenever an event is matched.

Q36) A cloud-based payments company is heavily using Amazon EC2 instances to host their applications in AWS Cloud. They would like to improve the security of their cloud resources by ensuring that all of their EC2 instances were launched from pre-approved AMIs only. The list of AMIs is set and managed by their IT Security team. Their Software Development team has an automated CI/CD process that launches several EC2 instances with new and untested AMIs for testing. The development process must not be affected by the new solution, which will be implemented by their Lead DevOps Engineer. Which of the following can the Engineer implement to satisfy the requirement with the LEAST impact on the development process? (Select TWO)

- Do regular scans using Amazon Inspector via a custom assessment template that determines if the Amazon EC2 instance is based upon a pre-approved AMI or not. Terminate the instances and inform the IT Security team by email about the security breach.
- Integrate AWS Lambda and CloudWatch Events to schedule a daily process that will search through the list of running Amazon EC2 instances

within your VPC. Configure the function to determine if any of these are based on unauthorized AMIs. Publish a new message to an Amazon SNS topic to inform the Security and Development teams that the issue occurred and then automatically terminate the EC2 instance.

Explanation:-WS Config is a service that enables you to assess, audit, and evaluate the configurations of your AWS resources. Config continuously monitors and records your AWS resource configurations and allows you to automate the evaluation of recorded configurations against desired configurations. With Config, you can review changes in configurations and relationships between AWS resources, dive into detailed resource configuration histories, and determine your overall compliance against the configurations specified in your internal guidelines. This enables you to simplify compliance auditing, security analysis, change management, and operational troubleshooting.

When you run your applications on AWS, you usually use AWS resources, which you must create and manage collectively. As the demand for your application keeps growing, so does your need to keep track of your AWS resources. AWS Config is designed to help you oversee your application resources.

AWS Config provides a detailed view of the configuration of AWS resources in your AWS account. This includes how the resources are related to one another and how they were configured in the past so that you can see how the configurations and relationships change over time. With AWS Config, you can do the following:

- Evaluate your AWS resource configurations for desired settings.
 - Get a snapshot of the current configurations of the supported resources that are associated with your AWS account.
 - Retrieve configurations of one or more resources that exist in your account.
 - Retrieve historical configurations of one or more resources.
 - Receive a notification whenever a resource is created, modified, or deleted.
 - View relationships between resources. For example, you might want to find all resources that use a particular security group.
- AWS Config provides AWS managed rules which are predefined, customizable rules that AWS Config uses to evaluate whether your AWS resources comply with common best practices. In this scenario, you can use the approved-amis-by-id AWS managed rule which checks whether running instances are using specified AMIs. You can also use a Lambda function which is scheduled to run regularly to scan all of the running EC2 instances in your VPC and check if there is an instance that was launched using an unauthorized AMI.

Hence, the correct answers are:

- Use AWS Config with a Lambda function that periodically evaluates whether there are EC2 instances launched based on non-approved AMIs. Set up a remediation action using AWS Systems Manager Automation that will automatically terminate the EC2 instance. Publish a message to an Amazon SNS topic to inform the IT Security and Development teams about the occurrence.
 - Integrate AWS Lambda and CloudWatch Events to schedule a daily process that will search through the list of running Amazon EC2 instances within your VPC. Configure the function to determine if any of these are based on unauthorized AMIs. Publish a new message to an Amazon SNS topic to inform the Security and Development teams that the issue occurred and then automatically terminate the EC2 instance.
- The option that says: Set up a centralized IT Systems Operations team that has the required policies, roles, and permissions. The team will manually process the security approval steps to ensure that Amazon EC2 instances are launched from pre-approved AMIs only is incorrect because having manual information security approval will impact the development process. A better solution is to implement an automated process using AWS Config or a scheduled job using AWS Lambda and CloudWatch Events.

The option that says: Set up IAM policies to restrict the ability of users to launch Amazon EC2 instances based on a specific set of pre-approved AMIs which were tagged by the IT Security team is incorrect because setting up an IAM Policy will totally restrict the development team from launching EC2 instances with unapproved AMIs which could impact their CI/CD process. The scenario clearly says that the solution should not have any interruption in the company's development process.

The option that says: Do regular scans using Amazon Inspector via a custom assessment template that determines if the Amazon EC2 instance is based upon a pre-approved AMI or not. Terminate the instances and inform the IT Security team by email about the security breach is incorrect because Amazon Inspector is just an automated security assessment service that helps improve the security and compliance of applications deployed in AWS. It does not have the capability to detect EC2 instances that are using unapproved AMIs, unlike AWS Config.

References:

<https://docs.aws.amazon.com/config/latest/developerguide/approved-amis-by-id.html>

https://docs.aws.amazon.com/config/latest/developerguide/evaluate-config_use-managed-rules.html

Check out this AWS Config Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-config/>

- Set up IAM policies to restrict the ability of users to launch Amazon EC2 instances based on a specific set of pre-approved AMIs which were tagged by the IT Security team.
- Set up a centralized IT Systems Operations team that has the required policies, roles, and permissions. The team will manually process the security approval steps to ensure that Amazon EC2 instances are launched from pre-approved AMIs only.

Q37) A company is planning to host their enterprise application in an ECS Cluster which uses the Fargate launch type. The database credentials should be provided to the AMI by using environment variables for security purposes. A DevOps engineer was instructed to ensure that the credentials are secure when passed to the image and that the sensitive passwords cannot be viewed on the cluster itself. In addition, the credentials must be kept in a dedicated storage with lifecycle management and key rotation.

Which of the following is the MOST suitable solution that the engineer should implement with the LEAST amount of effort?

- Upload and manage the database credentials using AWS Systems Manager Parameter Store then encrypt them using AWS KMS. Create an IAM Role for your Amazon ECS task execution role and reference it with your task definition, which allows access to both KMS and the Parameter Store. Within your container definition, specify secrets with the name of the environment variable to set in the container and the full ARN of the Systems Manager Parameter Store parameter containing the sensitive data to present to the container.
- Store the database credentials in the ECS task definition file of the ECS Cluster and encrypt them with KMS. Store the task definition JSON file in a private Amazon S3 bucket. Ensure that HTTPS is enabled on the bucket to encrypt the data in-flight. Set up an IAM role to the ECS task definition script that allows access to the specific S3 bucket and then pass the --cli-input-json parameter when calling the ECS register-task-definition. Reference the task definition JSON file in the S3 bucket which contains the database credentials.
- Store the database credentials using Docker Secrets in the ECS task definition file of the ECS Cluster where you can centrally manage sensitive data and securely transmit it to only those containers that need access to it. Ensure that the secrets are encrypted during transit and at rest.
- ✔ Store the database credentials using the AWS Secrets Manager. Encrypt the credentials using AWS KMS. Create an IAM Role for your Amazon ECS task execution role and reference it with your task definition which allows access to both KMS and AWS Secrets Manager. Within your container definition, specify secrets with the name of the environment variable to set in the container and the full ARN of the Secrets Manager secret, which contains the sensitive data, to present to the container.

Explanation:-Amazon ECS enables you to inject sensitive data into your containers by storing your sensitive data in either AWS Secrets Manager secrets or AWS Systems Manager Parameter Store parameters and then referencing them in your container definition. This feature is supported by tasks using both the EC2 and Fargate launch types.

Within your container definition, specify secrets with the name of the environment variable to set in the container and the full ARN of either the Secrets Manager secret or Systems Manager Parameter Store parameter containing the sensitive data to present to the container. The parameter that you reference can be from a different Region than the container using it, but must be from within the same account.

AWS Secrets Manager is a secrets management service that helps you protect access to your applications, services, and IT resources. This service

enables you to easily rotate, manage, and retrieve database credentials, API keys, and other secrets throughout their lifecycle. Using Secrets Manager, you can secure and manage secrets used to access resources in the AWS Cloud, on third-party services, and on-premises. If you want a single store for configuration and secrets, you can use Parameter Store. If you want a dedicated secrets store with lifecycle management, use Secrets Manager.

Hence, the correct answer is the option that says: Store the database credentials using the AWS Secrets Manager. Encrypt the credentials using AWS KMS. Create an IAM Role for your Amazon ECS task execution role and reference it with your task definition which allows access to both KMS and AWS Secrets Manager. Within your container definition, specify secrets with the name of the environment variable to set in the container and the full ARN of the Secrets Manager secret, which contains the sensitive data, to present to the container.

The option that says: Upload and manage the database credentials using AWS Systems Manager Parameter Store then encrypt them using AWS KMS. Create an IAM Role for your Amazon ECS task execution role and reference it with your task definition, which allows access to both KMS and the Parameter Store. Within your container definition, specify secrets with the name of the environment variable to set in the container and the full ARN of the Systems Manager Parameter Store parameter containing the sensitive data to present to the container is incorrect because although the use of Systems Manager Parameter Store in securing sensitive data in ECS is valid, this service doesn't provide dedicated storage with lifecycle management and key rotation, unlike Secrets Manager.

The option that says: Store the database credentials in the ECS task definition file of the ECS Cluster and encrypt them with KMS. Store the task definition JSON file in a private Amazon S3 bucket. Ensure that HTTPS is enabled on the bucket to encrypt the data in-flight. Set up an IAM role to the ECS task definition script that allows access to the specific S3 bucket and then pass the `--cli-input-json` parameter when calling the ECS `register-task-definition`. Reference the task definition JSON file in the S3 bucket which contains the database credentials is incorrect because although the solution may work, it is not recommended to store sensitive credentials in S3. This entails a lot of overhead and manual configuration steps which can be simplified by simply using the Secrets Manager or Systems Manager Parameter Store.

The option that says: Store the database credentials using Docker Secrets in the ECS task definition file of the ECS Cluster where you can centrally manage sensitive data and securely transmit it to only those containers that need access to it. Ensure that the secrets are encrypted during transit and at rest is incorrect because although you can use Docker Secrets to secure the sensitive database credentials, this feature is only applicable in Docker Swarm. In AWS, the recommended way to secure sensitive data is either through the use of Secrets Manager or Systems Manager Parameter Store.

References:

<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/specifying-sensitive-data.html>

<https://aws.amazon.com/blogs/mt/the-right-way-to-store-secrets-using-parameter-store/>

Check out this Amazon ECS Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-amazon-elastic-container-service-amazon-ecs/>

Check out this AWS Secrets Manager Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-secrets-manager/>

Q38) You have developed a custom web dashboard for your company that shows all instances running on AWS including the instance details. The web app relies on a DynamoDB table that will be updated whenever a new instance is created or terminated. You have several auto scaling groups of EC2 instances and you want to have an effective way of updating the DynamoDB table whenever a new instance is created or terminated. Which of the following steps will you implement?

- ☐ Configure a CloudWatch Event to your auto scaling group that will send a notification via SNS, which will trigger a Lambda function to update the DynamoDB table with the necessary instance details.
- ☐ Configure a CloudWatch alarm that will monitor the number of instances on your auto scaling group and trigger a Lambda function to update the DynamoDB table with the necessary details.
- ☐ Create a custom script that runs on the instances that will run on scale-in and scale-out events to update the DynamoDB table with the necessary details.
- ☒ Add lifecycle hooks to your auto scaling group so that a scale-out or scale-in will trigger a Lambda function to update the DynamoDB table with the necessary details.

Explanation:-Adding lifecycle hooks to your Auto Scaling group gives you greater control over how instances launch and terminate. Lifecycle hooks enable you to perform custom actions by pausing instances as an Auto Scaling group launches or terminates them.

For example, your newly launched instance completes its startup sequence and a lifecycle hook pauses the instance. While the instance is in a wait state, you can install or configure software on it, making sure that your instance is fully ready before it starts receiving traffic. For another example of the use of lifecycle hooks, when a scale-in event occurs, the terminating instance is first deregistered from the load balancer (if the Auto Scaling group is being used with Elastic Load Balancing). Then, a lifecycle hook pauses the instance before it is terminated. While the instance is in the wait state, you can, for example, connect to the instance and download logs or other data before the instance is fully terminated.

Hence, the correct answer is: Add lifecycle hooks to your auto scaling group so that a scale-out or scale-in will trigger a Lambda function to update the DynamoDB table with the necessary details.

The option that says: Create a custom script that runs on the instances that will run on scale-in and scale-out events to update the DynamoDB table with the necessary details is incorrect because although this may be possible, it will be harder to execute since a custom script needs to be developed first and will be run only at instance creation and termination.

The option that says: Configure a CloudWatch Event to your auto scaling group that will send a notification via SNS that will trigger a Lambda function to update the DynamoDB table with the necessary instance details is incorrect because it is unnecessary to send a notification to SNS. CloudWatch event can directly trigger a Lambda function.

The option that says: Configure a CloudWatch alarm that will monitor the number of instances on your auto scaling group and trigger a Lambda function to update the DynamoDB table with the necessary details is incorrect because although this may be possible, it is still not efficient since a CloudWatch alarm can trigger a Lambda function but this can be delayed. Therefore, it's better to just create a lifecycle hook.

References:

<https://docs.aws.amazon.com/autoscaling/ec2/userguide/lifecycle-hooks.html#adding-lifecycle-hooks>

<https://docs.aws.amazon.com/autoscaling/ec2/userguide/lifecycle-hooks.html#lifecycle-hooks-overview>

<https://docs.aws.amazon.com/autoscaling/ec2/userguide/AutoScalingGroupLifecycle.html>

Check out this AWS Auto Scaling Cheat Sheet:

<https://tutorialsdojo.com/aws-auto-scaling/>

Tutorials Dojo's AWS Certified DevOps Engineer Professional Exam Study Guide:

<https://tutorialsdojo.com/aws-certified-devops-engineer-professional/>

Q39) A company runs a popular e-commerce website that provides discounts and weekly sales promotions on various products. It was recently moved from its previous cloud hosting provider to AWS. For its architecture, it uses an Auto Scaling group of Reserved EC2 instances with an Application Load Balancer in front. Their DevOps team needs to set up a CloudFront web distribution that uses a custom domain name where the origin is set to point to the ALB. How can the DevOps Engineers properly implement an end-to-end HTTPS connection from the origin to the CloudFront viewers?

- Use a certificate that is signed by a trusted 3rd party certificate authority in the Application Load Balancer, which is then imported into AWS Certificate Manager. Choose the Match Viewer setting for the Viewer Protocol Policy to support both HTTP or HTTPS in CloudFront. Use an SSL/TLS certificate from a 3rd party certificate authority which was imported to either AWS Certificate Manager or the IAM certificate store.
- Generate an SSL self-signed certificate for the Application Load Balancer. Configure the Viewer Protocol Policy setting to HTTPS Only in CloudFront. Use an SSL/TLS certificate from a 3rd party certificate authority that was imported to either AWS Certificate Manager or the IAM certificate store.
- Utilize an SSL certificate that is signed by a trusted 3rd party certificate authority in the ALB, which is then imported into AWS Certificate Manager. Set the Viewer Protocol Policy to HTTPS Only in CloudFront. Use an SSL/TLS certificate from 3rd party certificate authority which was imported to an Amazon S3 bucket.
- ✔ Generate an SSL certificate that is signed by a trusted third-party certificate authority in the ALB. Import the certificate into AWS Certificate Manager. Set the Viewer Protocol Policy to HTTPS Only in CloudFront and then use an SSL/TLS certificate from a 3rd party certificate authority which was imported to either AWS Certificate Manager or the IAM certificate store.

Explanation:-For web distributions, you can configure CloudFront to require that viewers use HTTPS to request your objects, so that connections are encrypted when CloudFront communicates with viewers. You also can configure CloudFront to use HTTPS to get objects from your origin, so that connections are encrypted when CloudFront communicates with your origin.

Remember that there are rules on which type of SSL Certificate to use if you are using an EC2 or an ELB as your origin. This question is about setting up an end-to-end HTTPS connection between the Viewers, CloudFront, and your custom origin, which is an ALB instance.

The certificate issuer you must use depends on whether you want to require HTTPS between viewers and CloudFront or between CloudFront and your origin:

HTTPS between viewers and CloudFront

- You can use a certificate that was issued by a trusted certificate authority (CA) such as Comodo, DigiCert, Symantec or other third-party providers.
- You can use a certificate provided by AWS Certificate Manager (ACM)

HTTPS between CloudFront and a custom origin

- If the origin is not an ELB load balancer, such as Amazon EC2, the certificate must be issued by a trusted CA such as Comodo, DigiCert, Symantec or other third-party providers.
- If your origin is an ELB load balancer, you can also use a certificate provided by ACM.

If you're using your own domain name, such as tutorialsdojo.com, you need to change several CloudFront settings. You also need to use an SSL/TLS certificate provided by AWS Certificate Manager (ACM), or import a certificate from a third-party certificate authority into ACM or the IAM certificate store. Lastly, you should set the Viewer Protocol Policy to HTTPS Only in CloudFront.

Hence, the correct answer is: Generate an SSL certificate that is signed by a trusted third-party certificate authority in the ALB. Import the certificate into AWS Certificate Manager. Set the Viewer Protocol Policy to HTTPS Only in CloudFront and then use an SSL/TLS certificate from a 3rd party certificate authority which was imported to either AWS Certificate Manager or the IAM certificate store.

The option that says: Generate an SSL self-signed certificate for the Application Load Balancer. Configure the Viewer Protocol Policy setting to HTTPS Only in CloudFront. Use an SSL/TLS certificate from a 3rd party certificate authority that was imported to either AWS Certificate Manager or the IAM certificate store is incorrect because you cannot directly upload a self-signed certificate in your ALB.

The option that says: Use a certificate that is signed by a trusted 3rd party certificate authority in the Application Load Balancer, which is then imported into AWS Certificate Manager. Choose the Match Viewer setting for the Viewer Protocol Policy to support both HTTP or HTTPS in CloudFront. Use an SSL/TLS certificate from a 3rd party certificate authority which was imported to either AWS Certificate Manager or the IAM certificate store is incorrect because you have to set the Viewer Protocol Policy to HTTPS Only. With Match Viewer, CloudFront communicates with your custom origin using HTTP or HTTPS, depending on the protocol of the viewer request. For example, if you choose Match Viewer for Origin Protocol Policy and the viewer uses HTTP to request an object from CloudFront, CloudFront also uses the same protocol (HTTP) to forward the request to your origin.

The option that says: Utilize an SSL certificate that is signed by a trusted 3rd party certificate authority in the ALB, which is then imported into AWS Certificate Manager. Set the Viewer Protocol Policy to HTTPS Only in CloudFront. Use an SSL/TLS certificate from 3rd party certificate authority which was imported to an Amazon S3 bucket is incorrect because you cannot use an SSL/TLS certificate from a third-party certificate authority which was imported to S3.

References:

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/cnames-and-https-requirements.html>

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-https-viewers-to-cloudfront.html>

Check out this Amazon CloudFront Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-amazon-cloudfront/>

Q40) To further improve the company's CI/CD process, the company instructed their DevOps team to implement a notification system which will alert them for any failures in their pipelines in AWS. You were assigned to review the custom event pattern in Amazon CloudWatch Events that was prepared by one of your junior DevOps engineers as shown below: Which of the following will match this event pattern?

- There will be no match since the custom event pattern is invalid.
- ✔ All failed deploy and build actions across all the pipelines

Explanation:-Monitoring is an important part of maintaining the reliability, availability, and performance of AWS CodePipeline. You should collect monitoring data from all parts of your AWS solution so that you can more easily debug a multi-point failure if one occurs.

You can use the following tools to monitor your CodePipeline pipelines and their resources:

Amazon CloudWatch Events — Use Amazon CloudWatch Events to detect and react to pipeline execution state changes (for example, send an Amazon SNS notification or invoke a Lambda function).

AWS CloudTrail — Use CloudTrail to capture API calls made by or on behalf of CodePipeline in your AWS account and deliver the log files to an Amazon S3 bucket. You can choose to have CloudWatch publish Amazon SNS notifications when new log files are delivered so you can take quick action.

Console and CLI — You can use the CodePipeline console and CLI to view details about the status of a pipeline or a particular pipeline execution.

Amazon CloudWatch Events is a web service that monitors your AWS resources and the applications you run on AWS. You can use Amazon CloudWatch Events to detect and react to changes in the state of a pipeline, stage, or action. Then, based on rules you create, CloudWatch Events invokes one or more target actions when a pipeline, stage, or action enters the state you specify in a rule. Depending on the type of state change, you might want to send notifications, capture state information, take corrective action, initiate events, or take other actions.

You can configure notifications to be sent when the state changes for:

- Specified pipelines or all your pipelines. You control this by using "detail-type": "CodePipeline Pipeline Execution State Change".
- Specified stages or all your stages, within a specified pipeline or all your pipelines. You control this by using "detail-type": "CodePipeline Stage Execution State Change".
- Specified actions or all actions, within a specified stage or all stages, within a specified pipeline or all your pipelines. You control this by using "detail-type": "CodePipeline Action Execution State Change".

Hence, the correct answer is: All failed deploy and build actions across all the pipelines.

The option that says: All failed deploy and build pipelines is incorrect because the CodePipeline Action Execution State Change type tracks action-level events in CodePipeline and not pipeline-level events.

The option that says: All failed deploy and build stage across all the pipelines is incorrect because just as mentioned above, the configuration tracks action-level events not stage-level events. This could have been the right answer if the event type is CodePipeline Stage Execution State Change.

The option that says: There will be no match since the custom event pattern is invalid is incorrect because the provided pattern for the CloudWatch Events rule is a valid one.

References:

<https://docs.aws.amazon.com/codepipeline/latest/userguide/detect-state-changes-cloudwatch-events.html>

<https://docs.aws.amazon.com/codepipeline/latest/userguide/monitoring.html>

- All failed deploy and build pipelines
- All failed deploy and build stage across all the pipelines

Q41) A leading digital consultancy company has two teams in its IT department: the DevOps team and the Security team, that are working together on different components of its cloud architecture. AWS CloudFormation is used to manage its resources across all of its AWS accounts, including AWS Config for configuration management. The Security team applies the operating system-level updates and patches while the DevOps team manages application-level dependencies and updates. The DevOps team must use the latest AMI when launching new EC2 instances and deploying its flagship application. Which of the following options is the MOST scalable method for integrating the two processes and teams?

● Instruct the Security team to set up an AWS CloudFormation template that creates new versions of their AMIs and lists the Amazon Resource names (ARNs) of the AMIs in an encrypted S3 object as part of the stack output section. Direct the DevOps team to use the cross-stack reference to load the encrypted S3 object and obtain the most recent AMI ARNs.

● Instruct the Security team to maintain a nested stack in AWS CloudFormation that includes both the OS and the templates from the DevOps team. Order the Security team to use the stack update action to deploy updates to the application stack whenever the DevOps team changes the application code.

✔ Instruct the Security team to set up an AWS CloudFormation stack that creates an AWS CodePipeline pipeline that builds new Amazon Machine Images. Then, store the AMI ARNs as parameters in AWS Systems Manager Parameter Store as part of the pipeline output. Order the DevOps team to use the AWS::SSM::Parameter section in their CloudFormation stack to obtain the most recent AMI ARN from the Parameter Store.

Explanation:-Dynamic references provide a compact, powerful way for you to specify external values that are stored and managed in other services, such as the Systems Manager Parameter Store, in your stack templates. When you use a dynamic reference, CloudFormation retrieves the value of the specified reference when necessary during stack and change set operations.

CloudFormation currently supports the following dynamic reference patterns:

- ssm, for plaintext values stored in AWS Systems Manager Parameter Store
- ssm-secure, for secure strings stored in AWS Systems Manager Parameter Store
- secretsmanager, for entire secrets or specific secret values that are stored in AWS Secrets Manager

Some considerations when using dynamic references:

- You can include up to 60 dynamic references in a stack template.
- For transforms, such as AWS::Include and AWS::Serverless, AWS CloudFormation does not resolve dynamic references prior to invoking any transforms. Rather, AWS CloudFormation passes the literal string of the dynamic reference to the transform. Dynamic references (including those inserted into the processed template as the result of a transform) are resolved when you execute the change set using the template.
- Dynamic references for secure values, such as ssm-secure and secretsmanager, are not currently supported in custom resources.

Hence, the correct answer is: Instruct the Security team to set up an AWS CloudFormation stack that creates an AWS CodePipeline pipeline that builds new Amazon Machine Images. Then, store the AMI ARNs as parameters in AWS Systems Manager Parameter Store as part of the pipeline output. Order the DevOps team to use the AWS::SSM::Parameter section in their CloudFormation stack to obtain the most recent AMI ARN from the Parameter Store.

The option that says: Instruct the Security team to set up an AWS CloudFormation template that creates new versions of their AMIs and lists the Amazon Resource names (ARNs) of the AMIs in an encrypted S3 object as part of the stack output section. Direct the DevOps team to use the cross-stack reference to load the encrypted S3 object and obtain the most recent AMI ARNs is incorrect because it is better to store the parameters in AWS Systems Manager Parameter Store.

The option that says: Instruct the Security team to maintain a nested stack in AWS CloudFormation that includes both the OS and the templates from the DevOps team. Order the Security team to use the stack update action to deploy updates to the application stack whenever the DevOps team changes the application code is incorrect because using a nested stack will not decouple the responsibility of the two teams. Integrating AWS Systems Manager Parameter Store to store the ARN of the AMIs is a better solution.

The option that says: Instruct the Security team to use a CloudFormation stack that launches an AWS CodePipeline pipeline that builds new AMIs, then store the latest AMI ARNs in an encrypted S3 object as part of the pipeline output. Order the DevOps team to use a cross-stack reference within their own CloudFormation template to get that S3 object location and obtain the most recent AMI ARNs to use when deploying their application is incorrect because although this is a valid solution, it entails a lot of effort to set up a cross-stack reference within the DevOps team's own CloudFormation template to get that S3 object location and obtain the most recent AMI ARNs. You also have to ensure that the AMI ARN on Amazon S3 is the latest one.

References:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/dynamic-references.html>

<https://aws.amazon.com/blogs/mt/integrating-aws-cloudformation-with-aws-systems-manager-parameter-store/>

Check out this AWS CloudFormation Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-cloudformation/>

● Instruct the Security team to use a CloudFormation stack that launches an AWS CodePipeline pipeline that builds new AMIs then store the latest AMI ARNs in an encrypted S3 object as part of the pipeline output. Order the DevOps team to use a cross-stack reference within their own CloudFormation template to get that S3 object location and obtain the most recent AMI ARNs to use when deploying their application.

Q42) You have a fleet of 400 Amazon EC2 instances for your High Performance Computing (HPC) application. The fleet is configured to use target tracking scaling and is behind an ALB. You want to create a simple web page hosted on an S3 bucket that displays the status of the EC2 instances on the fleet. This web page will be updated whenever a new instance is launched or terminated. You are also required to keep a searchable log of these events so they can be reviewed later. Which of the following options will you implement?

- No need to do anything. CloudTrail records these events and you can view and search the logs on the console.
- Create a CloudWatch Events rule for the scale-in/scale-out event and deliver these logs to CloudWatch. Manually export the CloudWatch Log group to the S3 bucket to view them.
- Create a CloudWatch Events rule for the scale-in/scale-out event and create two targets for the event - one target to the webpage S3 bucket and

another to deliver event logs to CloudWatch.

✔ Write your own Lambda function to update the simple webpage on S3 and send event logs to CloudWatch. Create a CloudWatch Events rule to invoke the Lambda for scale-in/scale-out events.

Explanation:-You can create an AWS Lambda function that logs the changes in state for an Amazon EC2 instance. You can create a rule that runs the function whenever there is a state transition or a transition to one or more states that are of interest. Be sure to assign proper permissions to your Lambda function to write to S3 and to send logs to CloudWatch Logs. After creating the Lambda function, create a rule on CloudWatch Events that will watch for the scale-in/scale-out events. Set a trigger to run your Lambda function which will then update the S3 webpage and the send logs to CloudWatch.

Hence, the correct answer is: Write your own Lambda function to update the simple webpage on S3 and send event logs to CloudWatch. Create a CloudWatch Events rule to invoke the Lambda for scale-in/scale-out events.

The option that says: Create a CloudWatch Events rule for the scale-in/scale-out event and deliver these logs to CloudWatch. Manually export the CloudWatch Log group to the S3 bucket to view them is incorrect because you have to export the logs manually on an S3 bucket. This action is not recommended since you can simply use a Lambda function to log the changes of the EC2 instances to a file in the S3 bucket.

The option that says: Create a CloudWatch Events rule for the scale-in/scale-out event and create two targets for the event - one target to the webpage S3 bucket and another to deliver event logs to CloudWatch is incorrect because you cannot set an S3 bucket or object as a target for a CloudWatch Events rule.

The option that says: No need to do anything. CloudTrail records these events and you can view and search the logs on the console is incorrect because although this is true, it would be hard to search all the relevant logs from the trail. Moreover, it doesn't provide a solution for the required S3 web page.

References:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/events/LogEC2InstanceState.html>

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/events/CloudWatch-Events-Tutorials.html>

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/LaunchingAndUsingInstances.html>

Check out this Amazon CloudWatch Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-amazon-cloudwatch/>

Tutorials Dojo's AWS Certified DevOps Engineer Professional Exam Study Guide:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-certified-devops-engineer-professional/>

Q43) A company has an innovative mobile payment app that enables the users to easily pay their bills and transfer money without the hassle of logging in to their online banking, entering the account details of the other party and spending time going through other security verification processes. Anyone can easily pay another person, split the bill with their friends or pay for their coffee in an instant with just a few taps in their revolutionary mobile app. It is available on both Android and iOS devices, including a web portal that is deployed in AWS using OpsWorks Stacks and EC2 instances. The company has over 10 million users nationwide and close to 1,000 transactions every hour. A new feature is ready to be released which will enable the users to store their credit card information in their systems. However, the new version of the APIs and web portal cannot be deployed to the existing application stack due to the PCI-DSS compliance rule.

How should the DevOps Engineer deploy the new web portal for the mobile app without having any impact on the company's 10 million users?

● Upgrade the existing application stack in the Production environment in order to make it PCI-DSS compliant. Deploy the new version of the web portal on the existing application stack.

● Launch a brand new stack that contains the latest PCI-DSS compliant version of the web portal. Direct all the incoming traffic to the new application stack at once using the Amazon Route 53 service so that all the viewers can get to access new features.

● Use a Blue/Green deployment strategy to deploy the new PCI-DSS compliant web portal using AWS CodeDeploy, AWS SAM, and Lambda. The green environment represents the current web portal version serving production traffic while the blue environment is running the new version of the web portal.

✔ Launch a brand new OpsWorks stack that contains a new layer with the latest PCI-DSS compliant web portal version. Use a Blue/Green deployment strategy with Amazon Route 53 that shifts traffic between the existing stack and new stack. Route only a small portion of incoming production traffic to use the new application stack while maintaining the old application stack. Check the features of the new portal and once it's 100% validated, slowly increase incoming production traffic to the new stack. Change the Route 53 to revert to old stack if there are issues on the new stack.

Explanation:-Blue/green deployments provide near zero-downtime release and rollback capabilities. The fundamental idea behind blue/green deployment is to shift traffic between two identical environments that are running different versions of your application. The blue environment represents the current application version serving production traffic. In parallel, the green environment is staged running a different version of your application. After the green environment is ready and tested, production traffic is redirected from blue to green. If any problems are identified, you can roll back by reverting traffic back to the blue environment.

Hence, the correct answer in this scenario is the option that says: Launch a brand new OpsWorks stack that contains a new layer with the latest PCI-DSS compliant web portal version. Use a Blue/Green deployment strategy with Amazon Route 53 that shifts traffic between the existing stack and new stack. Route only a small portion of incoming production traffic to use the new application stack while maintaining the old application stack. Check the features of the new portal and once it's 100% validated, slowly increase incoming production traffic to the new stack. Change the Route 53 to revert to old stack if there are issues on the new stack.

The option that says: Upgrade the existing application stack in the Production environment in order to make it PCI-DSS compliant. Deploy the new version of the web portal on the existing application stack is incorrect because if you forcibly deploy the new web portal to the existing application stack and there is an issue on the deployment, there would be some inevitable downtime in order for you to fix the system and revert back to the original version. It is better to use blue/green deployments instead.

The option that says: Launch a brand new stack that contains the latest PCI-DSS compliant version of the web portal. Direct all the incoming traffic to the new application stack at once using the Amazon Route 53 service so that all the viewers can get to access new features is incorrect because although the current application stack can still be used if something goes wrong, the risk of service disruption is quite high when you direct all incoming traffic to the new portal. If something did go wrong, then there would be downtime in order for you to switch back to the old stack.

The option that says: Use a Blue/Green deployment strategy to deploy the new PCI-DSS compliant web portal using AWS CodeDeploy, AWS SAM and Lambda. The green environment represents the current web portal version serving production traffic while the blue environment is running the new version of the web portal is incorrect because although it mentioned Blue/Green deployment, the application stack doesn't mention about serverless computing at all. Hence, Lambda and AWS SAM are irrelevant in this situation. Moreover, the description of the blue and green environments are actually switched. The blue environment represents the current application while the green represents the new one.

References:

https://d0.awsstatic.com/whitepapers/AWS_Blue_Green_Deployments.pdf

https://docs.aws.amazon.com/opsworks/latest/userguide/welcome_classic.html

<https://docs.aws.amazon.com/opsworks/latest/userguide/best-deploy.html>

Q44) A rental payment startup has developed a web portal that enables users to pay for their rent using both their credit and debit cards online. They are using a third-party payment service to handle and process credit card payments on their platform since the portal is not fully compliant with the Payment Card Industry Data Security Standard (PCI DSS). The application is hosted in an Auto Scaling group of Amazon EC2 instances, which are launched in private subnets behind an internal-facing Application Load Balancer. The system must connect to an external payment service over the Internet to complete the transaction for every user payment.

As a DevOps Engineer, what would be the MOST suitable option to implement to satisfy the above requirement?

☐ Develop a shell script to automatically assign Elastic IP addresses to the Amazon EC2 instances. Add the script in the User Data of the EC2 instances, which automatically adds the Elastic IP address to the Network Access List upon launch. Update the route table associated with one or more of your private subnets to point Internet-bound traffic to a VPC Endpoint.

☒ Using a NAT Gateway, route credit card payment requests from the EC2 instances to the external payment service. Associate an Elastic IP address to the NAT Gateway. Update the route table associated with one or more of your private subnets to point Internet-bound traffic to the NAT gateway.

Explanation:-You can use a network address translation (NAT) gateway to enable instances in a private subnet to connect to the Internet or other AWS services, but prevent the Internet from initiating a connection with those instances.

To create a NAT gateway, you must specify the public subnet in which the NAT gateway should reside. You must also specify an Elastic IP address to associate with the NAT gateway when you create it. The Elastic IP address cannot be changed once you associate it with the NAT Gateway. After you've created a NAT gateway, you must update the route table associated with one or more of your private subnets to point Internet-bound traffic to the NAT gateway. This enables instances in your private subnets to communicate with the internet.

Each NAT gateway is created in a specific Availability Zone and implemented with redundancy in that zone. You have a limit on the number of NAT gateways you can create in an Availability Zone.

Remember the difference between NAT Instance and NAT Gateways. A NAT Instance needs to use a script to manage failover between instances while this is done automatically in NAT gateways.

Hence, the correct answer is: Using a NAT Gateway, route credit card payment requests from the EC2 instances to the external payment service. Associate an Elastic IP address to the NAT Gateway. Update the route table associated with one or more of your private subnets to point Internet-bound traffic to the NAT gateway.

The option that says: In the Security Group, whitelist the Public IP of the Internet Gateway. Route the user payment requests through the Internet Gateway. Update the route table associated with one or more of your private subnets to point Internet-bound traffic to the Internet Gateway is incorrect because you cannot whitelist an IP address using a Security Group. You should use a NAT Gateway instead to enable instances in a private subnet to connect to the Internet.

The option that says: Use the Application Load Balancer to route payment requests from the application servers through the Customer Gateway. Update the route table associated with one or more of your private subnets to point Internet-bound traffic to the Customer Gateway is incorrect because what you need here is a NAT Gateway and not a Customer Gateway. The use of an ELB is also not suitable for this scenario.

The option that says: Develop a shell script to automatically assign Elastic IP addresses to the Amazon EC2 instances. Add the script in the User Data of the EC2 instances, which automatically adds the Elastic IP address to the Network Access List upon launch. Update the route table associated with one or more of your private subnets to point Internet-bound traffic to a VPC Endpoint is incorrect because neither the required NAT Gateway nor the NAT instance is mentioned in this option. Moreover, a VPC endpoint simply enables you to privately connect your VPC to supported AWS services and VPC endpoint services powered by PrivateLink. This doesn't allow you to connect to the public Internet.

References:

<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>

<https://docs.aws.amazon.com/AmazonVPC/latest/UserGuide/vpc-nat-comparison.html>

Check out this Amazon VPC Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-amazon-vpc/>

☐ In the Security Group, whitelist the Public IP of the Internet Gateway. Route the user payment requests through the Internet Gateway. Update the route table associated with one or more of your private subnets to point Internet-bound traffic to the Internet Gateway.

☐ Use the Application Load Balancer to route payment requests from the application servers through the Customer Gateway. Update the route table associated with one or more of your private subnets to point Internet-bound traffic to the Customer Gateway.

Q45) A global cloud-based payment processing system is hosted in AWS which accepts credit card payments as well as cryptocurrencies such as Bitcoin. It is deployed in AWS which uses EC2, DynamoDB, S3, and CloudFront to process the payments. Since they are accepting credit card information from the users, they are required to be compliant with the Payment Card Industry Data Security Standard (PCI DSS). It was found that the credit card numbers are not properly encrypted on the recent 3rd-party audit and hence, their system failed the PCI DSS compliance test. You were hired by the company to solve this issue so they can release the product in the market as soon as possible. In addition, you also have to improve performance by increasing the proportion of your viewer requests that are served from CloudFront edge caches instead of going to your origin servers for content.

Which of the following is the BEST option to protect and encrypt the sensitive credit card information of the users and improve the cache hit ratio of your CloudFront distribution?

☐ Use a custom SSL in the CloudFront distribution. Configure your origin to add User-Agent and Host headers to your objects to increase your cache hit ratio.

☐ Modify the CloudFront distribution to use Signed URLs. Configure your origin to add a Cache-Control max-age directive to your objects, and specify the longest practical value for max-age to increase your cache hit ratio.

☒ Secure end-to-end connections to the origin servers in your CloudFront distribution by using HTTPS and field-level encryption. Set up your origin to add a Cache-Control max-age directive to your objects. Apply the longest practical value for max-age to increase your cache hit ratio.

Explanation:-You can already configure CloudFront to help enforce secure end-to-end connections to origin servers by using HTTPS. Field-level encryption adds an additional layer of security along with HTTPS that lets you protect specific data throughout system processing so that only certain applications can see it. Field-level encryption allows you to securely upload user-submitted sensitive information to your web servers. The sensitive information provided by your clients is encrypted at the edge closer to the user and remains encrypted throughout your entire application stack, ensuring that only applications that need the data—and have the credentials to decrypt it—are able to do so.

To use field-level encryption, you configure your CloudFront distribution to specify the set of fields in POST requests that you want to be encrypted, and the public key to use to encrypt them. You can encrypt up to 10 data fields in a request.

You can improve performance by increasing the proportion of your viewer requests that are served from CloudFront edge caches instead of going to your origin servers for content; that is, by improving the cache hit ratio for your distribution. To increase your cache hit ratio, you can configure your origin to add a Cache-Control max-age directive to your objects, and specify the longest practical value for max-age. The shorter the cache duration, the more frequently CloudFront forwards another request to your origin to determine whether the object has changed and, if so, to get the latest version.

Hence, the correct answer is: Secure end-to-end connections to the origin servers in your CloudFront distribution by using HTTPS and field-level encryption. Set up your origin to add a Cache-Control max-age directive to your objects. Apply the longest practical value for max-age to increase

your cache hit ratio.

The option that says: Use a custom SSL in the CloudFront distribution. Configure your origin to add User-Agent and Host headers to your objects to increase your cache hit ratio is incorrect because although it provides secure end-to-end connections to origin servers, it is better to add field-level encryption to protect the credit card information.

The option that says: Modify the CloudFront distribution to use Signed URLs. Configure your origin to add a Cache-Control max-age directive to your objects, and specify the longest practical value for max-age to increase your cache hit ratio is incorrect because a Signed URL provides a way to distribute private content but it doesn't encrypt the sensitive credit card information.

The option that says: Set up an origin access identity (OAI) and add it to the CloudFront distribution. Configure your origin to add User-Agent and Host headers to your objects to increase your cache hit ratio is incorrect because OAI is mainly used to restrict access to objects in S3 bucket, but not provide encryption to specific fields.

References:

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/field-level-encryption.html#field-level-encryption-setting-up>

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/cache-hit-ratio.html>

Check out this Amazon CloudFront Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-amazon-cloudfront/>

● Set up an origin access identity (OAI) and add it to the CloudFront distribution. Configure your origin to add User-Agent and Host headers to your objects to increase your cache hit ratio.

Q46) A national university has launched its serverless online learning portal using Lambda and API Gateway in AWS that enables its students to enroll, see their grades online as well as manage their class schedule. The portal abruptly stopped working after a few weeks and lost all of its data. The university hired a DevOps consultant and based on the investigation, the outage was due to an SQL injection vulnerability on the portal's login page in which the attacker simply injected the malicious SQL code. The consultant also emphasized the system's inability to track historical changes to the rules and metrics associated with their firewall.

Which of the following is the MOST suitable and cost-effective solution to avoid another SQL Injection attack against their infrastructure in AWS?

● Configure the Network Access Control List of your VPC to block the IP address of the attacker and then create a CloudFront web distribution.

Configure AWS WAF to add a web access control list (web ACL) in front of the CloudFront distribution to block requests that contain malicious SQL code. Track the changes to your web access control lists (web ACLs) such as the creation and deletion of rules including the updates to the WAF rule configurations using AWS Config.

✔ Add a web access control list in front of the API Gateway using AWS WAF to block requests that contain malicious SQL code. Track the changes to your web access control lists (web ACLs) such as the creation and deletion of rules including the updates to the WAF rule configurations using AWS Config.

Explanation:-AWS WAF is a web application firewall that helps protect your web applications from common web exploits that could affect application availability, compromise security, or consume excessive resources. With AWS Config, you can track changes to WAF web access control lists (web ACLs). For example, you can record the creation and deletion of rules and rule actions, as well as updates to WAF rule configurations.

AWS WAF gives you control over which traffic to allow or block to your web applications by defining customizable web security rules. You can use AWS WAF to create custom rules that block common attack patterns, such as SQL injection or cross-site scripting, and rules that are designed for your specific application. New rules can be deployed within minutes, letting you respond quickly to changing traffic patterns. Also, AWS WAF includes a full-featured API that you can use to automate the creation, deployment, and maintenance of web security rules.

AWS Config is a service that enables you to assess, audit, and evaluate the configurations of your AWS resources. Config continuously monitors and records your AWS resource configurations and allows you to automate the evaluation of recorded configurations against desired configurations. With Config, you can review changes in configurations and relationships between AWS resources, dive into detailed resource configuration histories, and determine your overall compliance against the configurations specified in your internal guidelines. This enables you to simplify compliance auditing, security analysis, change management, and operational troubleshooting.

In this scenario, integrating WAF in front of the API Gateway and using AWS Config are the ones that should be implemented in order to improve the security of the online learning portal. Hence, the correct answer is: Add a web access control list in front of the API Gateway using AWS WAF to block requests that contain malicious SQL code. Track the changes to your web access control lists (web ACLs) such as the creation and deletion of rules including the updates to the WAF rule configurations using AWS Config.

The option that says: Add a web access control list in front of the Lambda functions using AWS WAF to block requests that contain malicious SQL code. Track the changes to your web access control lists (web ACLs) such as the creation and deletion of rules including the updates to the WAF rule configurations using AWS Firewall Manager is incorrect because you have to use AWS WAF in front of the API Gateway and not directly to the Lambda functions. AWS Firewall Manager is primarily used to manage your Firewall across multiple AWS accounts under your AWS Organizations and hence, it is not suitable for tracking changes to WAF web access control lists. You should use AWS Config instead.

The option that says: Configure the Network Access Control List of your VPC to block the IP address of the attacker and then create a CloudFront web distribution. Configure AWS WAF to add a web access control list (web ACL) in front of the CloudFront distribution to block requests that contain malicious SQL code. Track the changes to your web access control lists (web ACLs) such as the creation and deletion of rules including the updates to the WAF rule configurations using AWS Config is incorrect because even though it is valid to use AWS WAF with CloudFront, it entails an additional and unnecessary cost to launch a CloudFront distribution for this scenario. There is no requirement that the serverless online portal should be scalable and be accessible around the globe hence, a CloudFront distribution is not relevant.

The option that says: Launch a new Application Load Balancer and set up AWS WAF to block requests that contain malicious SQL code. Place the API Gateway behind the ALB and use the AWS Firewall Manager to track changes to your web access control lists (web ACLs) such as the creation and deletion of rules including the updates to the WAF rule configurations is incorrect because launching a new Application Load Balancer entails additional cost and is not cost-effective. In addition, AWS Firewall Manager is primarily used to manage your Firewall across multiple AWS accounts under your AWS Organizations. Using AWS Config is much more suitable for tracking changes to WAF web access control lists.

References:

<https://aws.amazon.com/waf/>

<https://docs.aws.amazon.com/waf/latest/developerguide/what-is-aws-waf.html>

<https://aws.amazon.com/config/>

Check out this AWS WAF Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-waf/>

● Add a web access control list in front of the Lambda functions using AWS WAF to block requests that contain malicious SQL code. Track the changes to your web access control lists (web ACLs) such as the creation and deletion of rules including the updates to the WAF rule configurations using AWS Firewall Manager.

● Launch a new Application Load Balancer and set up AWS WAF to block requests that contain malicious SQL code. Place the API Gateway behind the ALB and use the AWS Firewall Manager to track changes to your web access control lists (web ACLs) such as the creation and deletion of rules including the updates to the WAF rule configurations.

Q47) An educational startup has developed an e-learning platform hosted in AWS, where they sell their online courses. The CTO

is planning to release an online forum that will allow students with each other and post their questions. This new feature requires a new DynamoDB table named Thread in which the partition key is ForumName and the sort key is Subject. Below is a diagram that shows how the items in the table must be organized:

The forum must also find all of the threads that were posted in a particular forum within the last three months as part of the system's quarterly reporting.

Which of the following is the MOST suitable solution that you should implement?

- Configure the e-learning platform to use a Query operation for the entire Thread table. Discard any posts that were not within the specified time frame.
- Refactor the e-learning platform to do a Scan operation in the entire Thread table. Discard any posts that were not within the specified time frame.
- Create a new DynamoDB table with a global secondary index. Refactor the e-learning platform to use a Query operation for search and to utilize the LastPostDateTime attribute as the sort key.
- ✔ Create a new DynamoDB table with a local secondary index. Refactor the e-learning platform to use the Query operation for search and utilize the LastPostDateTime attribute as the sort key.

Explanation:-DynamoDB supports two types of secondary indexes:

- Global secondary index — an index with a partition key and a sort key that can be different from those on the base table. A global secondary index is considered "global" because queries on the index can span all of the data in the base table, across all partitions.

- Local secondary index — an index that has the same partition key as the base table, but a different sort key. A local secondary index is "local" in the sense that every partition of a local secondary index is scoped to a base table partition that has the same partition key value.

A local secondary index maintains an alternate sort key for a given partition key value. A local secondary index also contains a copy of some or all of the attributes from its base table; you specify which attributes are projected into the local secondary index when you create the table. The data in a local secondary index is organized by the same partition key as the base table, but with a different sort key. This lets you access data items efficiently across this different dimension. For greater query or scan flexibility, you can create up to five local secondary indexes per table.

Suppose that an application needs to find all of the threads that have been posted within the last three months. Without a local secondary index, the application would have to Scan the entire Thread table and discard any posts that were not within the specified time frame. With a local secondary index, a Query operation could use LastPostDateTime as a sort key and find the data quickly.

In the provided scenario, you can create a local secondary index named LastPostIndex to meet the requirements. Note that the partition key is the same as that of the Thread table, but the sort key is LastPostDateTime as shown in the diagram below:

Hence, the most effective solution in this scenario is to: Create a new DynamoDB table with a local secondary index. Refactor the e-learning platform to use the Query operation for search and utilize the LastPostDateTime attribute as the sort key.

The option that says: Refactor the e-learning platform to do a Scan operation in the entire Thread table. Discard any posts that were not within the specified time frame is incorrect because although this option is valid, this solution would consume a large amount of provisioned read throughput and take a long time to complete. This is not a scalable solution and the time it takes to fetch the data will continue to increase as the table grows.

The option that says: Configure the e-learning platform to use a Query operation for the entire Thread table. Discard any posts that were not within the specified time frame is incorrect because using the Query operation is not sufficient to meet this requirement. You have to create a local secondary index when you create the table to narrow down the results and to improve the performance of your application.

The option that says: Create a new DynamoDB table with a global secondary index. Refactor the e-learning platform to use a Query operation for search and to utilize the LastPostDateTime attribute as the sort key is incorrect because using a local secondary index is a more appropriate solution to be used in this scenario. Take note that in this scenario, it is still using the same partition key (ForumName) but with an alternate sort key (LastPostDateTime) which warrants the use of a local secondary index.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/SecondaryIndexes.html>

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/LSI.html>

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-amazon-dynamodb/>

Tutorials Dojo's AWS Certified DevOps Engineer Professional Exam Study Guide:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-certified-devops-engineer-professional/>

Q48) A PHP web application is uploaded to the AWS Elastic Beanstalk of the company's development account to automatically handle the deployment, capacity provisioning, load balancing, auto-scaling, and application health monitoring. Amazon RDS is used as the database, and it is tightly coupled to the Elastic Beanstalk environment. A DevOps Engineer noticed that if you terminate the environment, its database goes down as well. This issue prevents you from performing seamless updates with blue-green deployments. Moreover, this poses a critical security risk if the company decides to deploy the application in its production environment.

How can the DevOps Engineer decouple the database instance from the environment with the LEAST amount of data loss?

- Decouple the Amazon RDS instance from your Elastic Beanstalk environment using the Canary deployment strategy. Take an RDS DB snapshot of the database and then set up a new Elastic Beanstalk environment with the necessary information to connect to the Amazon RDS instance.
- Decouple the Amazon RDS instance from your Elastic Beanstalk environment using a blue/green deployment strategy. Enable deletion protection and take an RDS DB snapshot of the database. Set up a new Elastic Beanstalk environment with the necessary information to connect to the Amazon RDS instance and delete the old environment.
- ✔ Decouple the Amazon RDS instance from your Elastic Beanstalk environment using the blue/green deployment strategy to decouple. Take an RDS DB snapshot of the database and enable deletion protection. Set up a new Elastic Beanstalk environment with the necessary information to connect to the Amazon RDS instance. Before terminating the old Elastic Beanstalk environment, remove its security group rule first before proceeding.

Explanation:-AWS Elastic Beanstalk provides support for running Amazon Relational Database Service (Amazon RDS) instances in your Elastic Beanstalk environment. This works great for development and testing environments. However, it isn't ideal for a production environment because it ties the lifecycle of the database instance to the lifecycle of your application's environment.

If you haven't used a DB instance with your application before, try adding one to a test environment with the Elastic Beanstalk console first. This lets you verify that your application is able to read environment properties, construct a connection string, and connect to a DB instance before you add Amazon Virtual Private Cloud (Amazon VPC) and security group configuration to the mix.

To decouple your database instance from your environment, you can run a database instance in Amazon RDS and configure your application to connect to it on launch. This enables you to connect multiple environments to a database, terminate an environment without affecting the database, and perform seamless updates with blue-green deployments.

To allow the Amazon EC2 instances in your environment to connect to an outside database, you can configure the environment's Auto Scaling group with an additional security group. The security group that you attach to your environment can be the same one that is attached to your database instance, or a separate security group from which the database's security group allows ingress.

You can connect your environment to a database by adding a rule to your database's security group that allows ingress from the autogenerated security group that Elastic Beanstalk attaches to your environment's Auto Scaling group. However, doing so creates a dependency between the two

security groups. Subsequently, when you attempt to terminate the environment, Elastic Beanstalk will be unable to delete the environment's security group because the database's security group is dependent on it.

Hence, the correct answer is: Decouple the Amazon RDS instance from your Elastic Beanstalk environment using the blue/green deployment strategy to decouple. Take an RDS DB snapshot of the database and enable deletion protection. Set up a new Elastic Beanstalk environment with the necessary information to connect to the Amazon RDS instance. Before terminating the old Elastic Beanstalk environment, remove its security group rule first before proceeding.

The option that says: Decouple the Amazon RDS instance from your Elastic Beanstalk environment using a blue/green deployment strategy. Enable deletion protection and take an RDS DB snapshot of the database. Set up a new Elastic Beanstalk environment with the necessary information to connect to the Amazon RDS instance and delete the old environment is incorrect because although the deployment strategy being used here is valid, the existing security group rule is not yet removed which hinders the deletion of the old environment.

The option that says: Decouple the Amazon RDS instance from your Elastic Beanstalk environment using the Canary deployment strategy. Take an RDS DB snapshot of the database and enable deletion protection. Set up a new Elastic Beanstalk environment with the necessary information to connect to the Amazon RDS instance and delete the old environment is incorrect because there is no Canary deployment configuration in Elastic Beanstalk. This type of deployment strategy is usually used in Lambda.

The option that says: Decouple the Amazon RDS instance from your Elastic Beanstalk environment using the Canary deployment strategy. Take an RDS DB snapshot of the database and then set up a new Elastic Beanstalk environment with the necessary information to connect to the Amazon RDS instance is incorrect because you should use a blue/green deployment strategy instead. This will also cause data loss since the deletion protection for the database is not enabled. Moreover, there is no Canary deployment configuration in Elastic Beanstalk.

References:

<https://aws.amazon.com/premiumsupport/knowledge-center/decouple-rds-from-beanstalk/>

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/AWSHowTo.RDS.html>

Check out this AWS Elastic Beanstalk Cheat Sheet:

<https://tutorialsdojo.com/aws-cheat-sheet-aws-elastic-beanstalk/>

Elastic Beanstalk vs CloudFormation vs OpsWorks vs CodeDeploy:

<https://tutorialsdojo.com/aws-cheat-sheet-elastic-beanstalk-vs-cloudformation-vs-opsworks-vs-codedeploy/>

● Decouple the Amazon RDS instance from your Elastic Beanstalk environment using the Canary deployment strategy. Take an RDS DB snapshot of the database and enable deletion protection. Set up a new Elastic Beanstalk environment with the necessary information to connect to the Amazon RDS instance and delete the old environment.

Q49) You have developed an app that lets users quickly upload and share image snippets across various platforms such as mobile app, desktop web browser, and third party instant messaging apps. Your app relies on a Lambda function that detects each request's User-Agent and automatically sends a corresponding image resolution based on the device. You want to have a consistent and fast experience for all your users around the world. Which of the following options will you implement?

- Deploy your function on Amazon API Gateway and set up an Edge-optimized API endpoint.
- Upload your function on S3 and set it as CloudFront origin to run it.
- Create your function on AWS Lambda and set it as a CloudFront origin.
- ✔ Create your function on Lambda and deploy it with Lambda@Edge.

Explanation:-Lambda@Edge runs your code in response to events generated by the Amazon CloudFront content delivery network (CDN).

Lambda@Edge lets you run Node.js Lambda functions to customize content that CloudFront delivers, executing the functions in AWS locations closer to the viewer. The functions run in response to CloudFront events, without provisioning or managing servers.

Lambda@Edge is a feature of Amazon CloudFront that lets you run code closer to users of your application, which improves performance and reduces latency. Lambda@Edge lets you use CloudFront triggers to invoke a Lambda function. When you associate a CloudFront distribution with a Lambda function, CloudFront intercepts requests and responses at CloudFront edge locations and runs the function. Lambda functions can improve security or customize information close to your viewers, to improve performance.

Hence, the correct answer is: Create your function on Lambda and deploy it with Lambda@Edge.

The option that says: Create your function on AWS Lambda and set it as a CloudFront origin is incorrect because in AWS, you don't usually set a Lambda function as the origin for your CloudFront distribution. You have to use Lambda@Edge instead.

The option that says: Upload your function on S3 and set it as CloudFront origin to run it is incorrect because although you can set S3 as the origin of your CloudFront distribution, you won't be able to trigger the function since S3 has no compute capability.

The option that says: Deploy your function on Amazon API Gateway and set up an Edge-optimized API endpoint is incorrect because you can't run your function using API Gateway alone. You have to integrate your Lambda functions with API Gateway first. Moreover, The Edge-optimized API is just a default hostname in API Gateway that is deployed to the specified region while using a CloudFront distribution to facilitate client access typically from across AWS regions.

References:

<https://docs.aws.amazon.com/lambda/latest/dg/lambda-edge.html>

<https://aws.amazon.com/lambda/edge/>

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/lambda-edge-how-it-works-tutorial.html>

Check out this AWS Lambda Cheat Sheet:

<https://tutorialsdojo.com/aws-lambda/>